

Shape Up v 1.7, 2019 edition

Stop Running in Circles and
Ship Work that Matters

by Ryan Singer

Jason Fried 的前言

致谢

简介

成长的烦恼

六周周期

塑造工作

赋予团队全面责任

针对风险

本书的组织方式

塑造的原则

线框图太具体

文字太抽象

案例研究：点阵日历

特性 1：粗糙

特性 2：已解决

特性 3：已界定

谁来进行塑造

两个轨道

塑造的步骤

设定边界

设定期望

固定时间，可变范围

“好”是相对的

响应原始想法

缩小问题范围

案例研究：定义“日历”

当心抓包

边界已经建立

寻找元素

以正确的速度前进

面包板

例子

粗笔素描

元素是输出

设计师的空间

尚未可交付

没有传送带

风险和兔子洞

不同类别的风险

寻找兔子洞

案例研究：填补漏洞

宣布超出范围

精简

向技术专家展示

降低风险，准备撰写

写一个提案

要素 1. 问题

要素 2. 兴趣

要素 3. 解决方案

帮助他们看到

嵌入式素描

注释的粗笔素描

要素 4. 兔子洞

要素 5. 不做

实例

准备好展示

Basecamp 中的实施方式

投注，而不是积压

没有积压列表

一些潜在的投注选项

分散的列表

重要的想法会回来

投注桌

六周周期

冷却期

团队和项目规模

投注桌

押注的意义

不间断的时间

断路器

关于 bug 呢？

保持清洁

下注吧

观察你所在的位置

现有产品

新产品

研发模式

生产模式

清理模式

例子

点阵日历

新产品：HEY

实验性功能：Hill Charts

需要询问的问题

问题是否重要？

期望是否合理？

解决方案是否有吸引力？

现在是否合适？

是否有合适的人员可用？

发布启动消息

移交责任

分配项目，而非任务

完成意味着部署

启动

取向

想象的 vs 发现的任务

完成一个部分

整合一个片段

案例研究：项目中的客户

程序员不需要等待

像素完美屏幕之前的可承受能力

为下一步编写足够的程序

从中间开始

绘制范围

按结构组织，而不是按人员

范围图

项目的语言

案例研究：消息草稿

发现范围

如何知道范围是否正确

分层蛋糕

冰山

杂烩

用~标记好做的事项

展示进展

那些不存在的任务

估算不显示不确定性

工作就像一座山

山上的范围

不需要询问的状态

没有人说“我不知道”

重构范围的提示

向山顶努力

按正确的顺序解决问题

决定何时停止

与基线比较

限制激发权衡

范围会像草一样生长

裁减范围并不是在降低质量

裁减范围

QA 是为了边缘情况

何时延长项目

迈出下一步

让风暴过去

保持无债状态

反馈需要被塑造

结论

关键概念

联系我们

如何在 Basecamp 中实施 Shape Up

用于塑形的 Basecamp 团队

周期项目的 Basecamp 项目

用于作用域的待办事项清单

在 Hill 图上跟踪作用域

调整到你的尺寸

基本真理 vs. 具体实践

小到可以临时应付

大到可以专业化

如何开始塑造形势

选项 A：进行为期六周的实验

选项 B：从塑造开始

选项 C：从周期开始

首先解决发货问题

Jason Fried 的前言

团队的工作方式对于团队能够完成的事情有着巨大的影响。流程、方法、实践、方式、纪律、信任、沟通风格、速度。方式——就是如何——是基础和根本。

你经常会听到人们说“执行就是一切”，但这并不完全正确。事实上，这往往是相当错误的。

当涉及到项目工作，特别是软件开发，以错误的方式执行某事可能会摧毁士气，压垮团队，侵蚀信任，损害长期进步的机制。是的，它“完成”了，但代价是什么？通过做这件事，我们对自己做了什么？我们真的必须再次这样做，一个月又一个月，一年又一年吗？

你参与过多少个你想要重新做的项目？有多少项目延期，最终爆发出问题，让人筋疲力尽？有多少项目本质上是不合理期望的集合？有多少项目让团队内部产生分歧，让从施工者到利益相关者都感到沮丧，最终宁愿灭亡也不愿交付？

有时执行就是一切——一切都是错误的。那么正确的执行又是什么样子呢？

在过去几年里，人们对我们在 Basecamp 的工作方式表现出了更高的好奇心。人们经常问我们如何以如此小的团队在如此短的时间内以如此高水平的质量完成如此多的工作。以及我们如何让我们的团队长期合作。

首先，我们不做瀑布、敏捷或 Scrum。其次，我们不用便利贴贴满墙壁。第三，我们不做每日站立会议、设计冲刺、开发冲刺，或者任何与在最后感到疲惫不堪有关的隐喻。没有积压任务、没有看板、没有速度追踪，这些通通没有。

我们有完全不同的方法。这是在近 15 年的不断试验、摸索、记录、迭代、完善中独立发展起来的。我们塑造了自己的方式。

博客文章、研讨会和偶尔的会议演讲为人们提供了我们独特过程的一瞥，但我们从未将其公之于众。这本书就是要做到这一点。

现在我们的过程已经完全形成、记录完备、准备就绪，我们愿意与所有对学习新方法感到好奇的人分享。探险家、先驱，那些不在乎别人在做什么的人。那些希望比其他人做得更好的人。

不要把它看作一本书。把它看作一盏手电筒。你和你的团队在黑暗中摸索了足够长的时间。现在你有了一种明亮而强大的东西，可以帮助你找到一条新路。

我们希望你觉得它有趣、启发，最重要的是，有帮助。

谢谢阅读。

致谢

Jason Fried 和 David Heinemeier Hansson, Basecamp 的创始人, 为本书播下了许多种子。这本书受到他们的价值观、Basecamp 的文化以及长达十五年的合作试错的启发。

Bob Moesta 和 Chris Spiek 作出了重要贡献。没有他们的帮助, 这本书就不可能问世。

Yaneer Bar-Yam 在新英格兰复杂系统研究所的讲座帮助我构建了这一方法。

Basecamp 的专业设计师和程序员多年来不断尝试、测试和改进这些技术, 以便将真实项目交付。他们的努力使这本书成为了实践的书, 而非理论的书。

简介

这本书是关于我们在 Basecamp 如何进行产品开发的指南。它也是一个装满了技术的工具箱, 你可以按照自己的方式应用到你自己的流程中。

无论你是创始人、CTO、产品经理、设计师还是开发人员, 你可能会因为一些所有软件公司都必须面对的共同挑战而来到这里。

成长的烦恼

随着软件团队的壮大, 一些常见的挣扎出现了:

- 团队成员觉得项目似乎永无止境。
- 产品经理无法找到时间对产品进行战略性思考。
- 创始人问自己：“为什么我们不能像早期那样快速推出功能？”

在 Basecamp，我们亲身经历了这些挑战，当我们从四个人发展到五十多人。

Basecamp 在 2003 年开始，我们为自己建立了一个工具。那时我们是一家为客户设计网站的咨询公司。信息在客户、设计师和项目经理之间的传递中会丢失。我们希望 Basecamp 成为一个集中的地方，所有相关方都能看到工作、讨论并知道接下来该做什么。事实证明很多公司都有这种“信息流失”的问题。今天，各行各业的数百万人依赖 Basecamp 作为他们的共享真相来源。

我们三个人建立了第一个版本。Jason Fried，Basecamp 的创始人，领导了设计工作。他的联合创始人 David Heinemeier Hansson 负责编程（并创造了著名的 Web 框架 Ruby on Rails）。当时我是一个专注于可用性和用户界面的网页设计师。我执行了 Jason 对应用程序关键功能的设计方向，并与他合作填补了概念的细节。

从 2003 年 7 月的第一个原型到 2004 年 2 月的发布，David 每周只工作十个小时。我们知道，除非我们非常有目的地利用这十个小时，否则我们不会取得任何进展。在这些限制下，我们对“限定时间预算内的范围”进行了强调。

随着业务的增长，我开始扩展我的技能。与 David 和 Ruby on Rails 一起工作使我能够接触到编程世界。我学会了程序员用于驯服复杂性的技术：例如模块化、抽象级别和关注点分离。一只脚踏在设计世界，另一只脚踏在编程世界，我想知道我们是否可以将这

些软件开发原则应用到我们设计和管理产品的方式中。

这个想法的第一个测试是在 2009 年进行的。那时，我们已经聘用了几名程序员，并提供了四个独立的软件即服务产品。我们希望将这些产品捆绑在一起，成为一个具有单一登录和统一计费的无缝套件。这是一个庞大的技术工程，具有具有风险的用户界面流程。除了确保基础架构正确外，我们还必须在客户进入产品时打断他们，并让他们因难以解释的原因更改他们的用户名和密码。我在这个项目中兼任设计师和产品经理，我原型设计了本书中描述的装样板和范围映射技术，以管理复杂性。

我们取得了非常好的成果，以至于我们决定在 2012 年再次应用相同的技术，重新设计 Basecamp，为 2.0 版本。再次，需要管理的工作非常庞大，但整个过程出奇的顺利。

到了 2015 年，我们已经有了一个核心团队，他们经历了这些经验，并取得了令人瞩目的进展。但是我们发现很难向新员工解释我们正在做的事情。我们的产品团队已经扩大了四倍，每个人都在远程工作。这使得很难传达我们的直觉。我们需要一种语言来描述我们正在做什么，并且需要更多的结构来保持在我们的新规模上继续这样做。

为了管理这种新的能力，我们从临时的项目长度转变为重复的周期。（经过一些实验，找到了正确的周期长度：六周。稍后再详谈。）我们正式化了我们的提出和投注流程。我的角色再次转变，从设计和产品管理到产品策略。我需要新的语言，比如“塑造”这个词，来描述我们在承诺项目给团队之前进行的前期设计工作，以设定边界和减少风险。

就在我们开始更好地向自己描述我们的工作方式时，越来越多的朋友和同行开始来找我们询问我们是如何做到的。最后，Jason 有一天拉着我说，我认为你应该写一本关于这个的书。

这就是结果。你可以将这本书看作是两本书合在一起。首先，它是一本基本真理的书。我希望它能给你更好的语言来描述和应对每当你进行产品开发时出现的风险、不确定性和挑战。其次，这本书概述了我们在我们目前规模上对我们的产品取得有意义进展所使用的具体流程。

以下是这本书中主要观点的简要概述。

六周周期

首先，我们在**六周周期**内工作。六周足够长，可以从头到尾构建出有意义的东西，也足够短，以至于每个人从一开始就感受到截止日期的临近，因此他们会明智地利用时间。我们大部分的新功能都是在一个六周周期内构建并发布的。

我们的决定是基于在接下来的六周内推动产品前进，而不是微观管理时间。我们不计算小时数，也不质疑每天的花费。我们不进行每日会议。我们不会每两周重新思考我们的路线图。我们的焦点在更高的水平上。我们对自己说：“如果这个项目在六周后交付，我们会感到非常高兴。我们会觉得我们的时间花得很值。”然后我们承诺六周，让团队独立完成工作。

塑造工作

其次，我们会在将工作交给团队之前**塑造工作**。一个小型的高级团队与周期团队并行工作。在我们认为项目已经准备好投注之前，他们会定义解决方案的关键元素。项目的定义在正确的抽象级别上：具体到团队知道该做什么，但又足够抽象，以至于他们有空自行解决有趣的细节。

在塑造过程中，我们更加关注我们的**兴趣**而不是估计。我们不问完成某项工作需要多少时间，而是问：我们想要花费多少时间？这就是塑造的任务：缩小问题的范围，并设计出在我们兴趣范围内的解决方案的大纲。

赋予团队全面责任

第三，我们**赋予一个小型的集成团队完全责任**。他们定义自己的任务，调整范围，并共同构建产品的垂直切片。这与其他方法完全不同，其他方法中，经理们会切割工作，而程序员则充当接票员。

这些概念共同形成了一个良性循环。当团队更加自治时，高级人员就可以花更少的时间来管理他们。花在管理上的时间较少，高级人员可以更好地塑造项目。当项目塑造得更好时，团队就有了更清晰的边界，因此可以更加自治地工作。

针对风险

在整个过程的每一步，我们都针对特定的风险：无法按时交付的风险。这本书不是关于建造错误事物的风险。其他书可以帮助你解决这个问题（我们推荐 [Competing Against Luck](#)）。在重新获得你的交付能力之后，你可以改善你的发现过程。你可以拥有世界上最好的策略，但如果你无法行动，那又有什么好处呢？

这本书是关于陷入困境的风险，陷入上个季度工作的风险，浪费时间在意外问题上，以及无法自由地做明天想做的事情的风险。

在塑造过程中，我们通过将项目承诺到时间范围之前解决开放性问题来降低风险。我们不会将问题交给一个团队，而这个团队仍然有未解决的问题或交错的依赖关系。

在计划过程中，我们通过将我们的投注限制在六周来降低风险。如果一个项目超时，通常情况下它不会得到延期。这个“断路器”确保我们不会在需要重新思考的概念上投入原始兴趣的倍数。

最后，在构建过程中，我们通过早期集成设计和编程来降低风险。我们不会构建许多不相关的部分，并希望它们在最后一刻合并在一起，而是从一开始就构建工作的一个有意义的部分，然后重复。团队将工作从最未知的部分到最不令人担忧的部分进行排序，并尽快集成，以了解什么是有效的，什么是无效的。

本书的组织方式

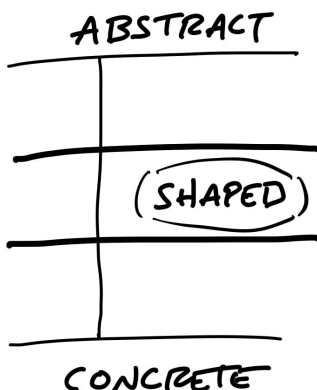
第一部分是关于**塑造**的 —— 我们在安排项目之前对项目进行的预先工作。每一章都解释了流程的具体步骤，从在一个原始想法上设定兴趣，到勾画出一个解决方案，再到撰写一个呈现潜在项目的提案。在此过程中，你将学习到一些具体的技术 —— 比如装样板和粗笔素描 —— 来保持设计在正确的抽象级别上。

第二部分是关于**投注**的 —— 我们如何在提出的项目中进行选择，并决定每次六周要做什么。

第三部分是关于**构建**的 —— 我们对团队的期望和他们用来发现要做什么的特殊实践。我们将看看团队如何找出要做什么，他们如何集成设计和编程，他们如何跟踪已知与未知的内容，最后他们如何做出艰难的决策，以便按时完成项目。

最后，附录为你提供了一些帮助，当你需要在你的公司进行变革时。有一些建议，关于如何尝试你的第一个六周实验，如何调整方法以适应公司的规模，以及如何使用 Basecamp 实施 Shape Up 的具体指导。

塑造的原则



当我们进行塑造工作时，需要在正确的抽象层次上进行：既不要太模糊，也不要太具体。产品经理经常在这两个极端中犯错。

线框图太具体

当设计负责人直接进行线框图或高保真模型时，他们在早期定义了太多的细节。这样做会让设计师缺乏创造力的空间。一位朋友这样描述道：

我会把一个线框图交给我的设计师，然后对她说：“我知道你正在看这个，但这不是我想让你设计的。我希望你重新思考它！”当你给他们这个具体的东西时，要做到这一点是很困难的。

过度详细说明设计也会导致估计错误。尽管看起来有些反直觉，但工作越具体，估计起来就越困难。这是因为使界面“恰到好处”可能需要解决在模型中看不到的隐藏复杂性和实现细节。当范围不可变时，团队无法重新考虑一个设计决策，如果发现成本超出了价值。

文字太抽象

另一方面，太模糊的项目也不可行。当一个项目用几个词定义时，没有人知道它意味着什么。“构建日历视图”或“添加组通知”听起来很合理，但具体指的是什么？团队成员没有足够的信息来进行权衡。他们不知道该包含什么或排除什么。一位曾在这种情况下工作过的程序员说：

你在没有上下文的情况下解决问题。你必须成为一名心灵读者。就像是：“我们看到它时就会知道。”

对于估计而言，过于未具体说明的项目自然会失控，因为没有界定范围来定义超出范围的内容。

案例研究：点阵日历

让我们来看一个如何在适当的细节层次上塑造项目的例子。

我们在没有日历功能的情况下推出了 Basecamp 的第三个版本。它有一个“日程安排”功能，只是简单地按顺序列出事件，没有任何月度、周度或日常网格。

发布后不久，客户开始要求我们在 Basecamp 中“添加一个日历”。我们以前构建过日历，我们知道它们有多复杂。构建一个合适的日历很容易需要六个月甚至更长时间。

日历变得复杂的原因包括：

- 在单元格之间拖放事件以移动它们
- 将多天事件环绕在屏幕边缘
- 不同的月度、周度或日常时间尺度的不同视图
- 拖动事件边缘以更改其持续时间
- 为不同类别的事件进行颜色编码
- 处理桌面和移动设备交互的不同期望

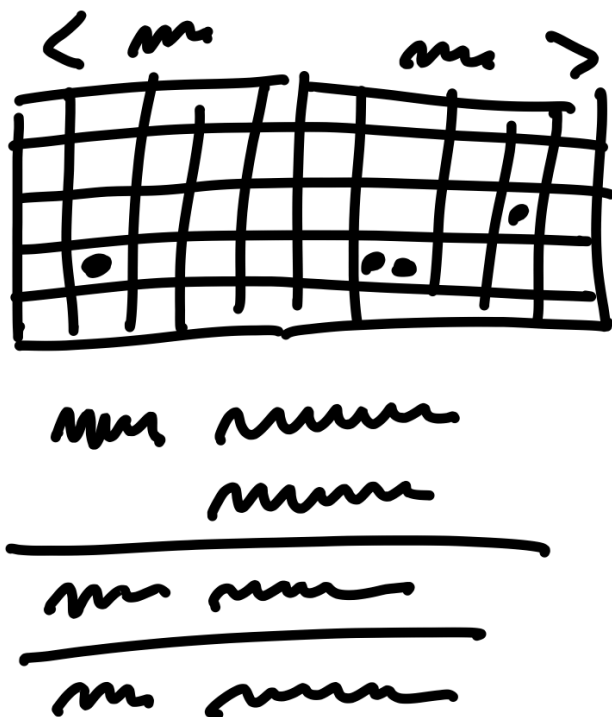
Basecamp 的早期版本有日历功能，但只有大约 10% 的客户使用它们。这就是为什么我们没有对日历投入六个月时间的原因。另一方面，如果我们可以一个六周的周期内做些事情来满足那些在这段时间内给我们写信的客户，我们愿意考虑这样做。

在只有六周的时间内，我们只能构建当人们说“日历”时所想象中的大约十分之一。问题变成了：哪一部分？

我们进行了一些研究（在下一章中讨论），并缩小了我们想要解决的一个用例。我们最终得出了一个有前途的概念，受到了手机上的日历的启发。我们可以构建一个两个月的只读网格视图。任何有事件的一天都会有一个点来表示每个事件。日历下方将显示事件列表，单击带有点的日期将滚动到该日期的事件视图。我们称之为“点阵”。

点阵不是一个功能齐全的日历。我们打算允许在日期之间拖放事件。我们也不打算跨越网格展示多天事件；我们只会重复这些点。对事件进行颜色编码或分类也不会有。由于我们对用例的理解，我们对所有这些权衡都感到满意。

这是我們用来定义解决方案的精细度水平：



点阵概念的粗略草图

请注意草图的粗糙程度以及省略了多少细节。设计师有很大的空间来解释这个概念应该如何看起来和感觉起来。

与此同时，请注意这个想法有多么具体。它非常清楚它是如何工作的，需要构建什么，什么在内部，什么在外部。

项目结束时，设计师和程序员创建的完成作品是这样的：

August							September						
SUN	MON	TUE	WED	THU	FRI	SAT	SUN	MON	TUE	WED	THU	FRI	SAT
		1	2	3	4	5						1	2
6	7	8	9	10	11	12	3	4	5	6	7	8	9
13	14	15	16	17	18	19	10	11	12	13	14	15	16
20	21	22	23	24	25	26	17	18	19	20	21	22	23
27	28	29	30	31			24	25	26	27	28	29	30

Wed, Aug 23

Interview with The TaTa Top 

9:30am - 10:00am

Interview with Tim Krepp 

12:00pm - 12:30pm

Add an event

Mon, Aug 28

Interview with Amy Trager 

11:00am - 11:30am

Wed, Aug 30

Interview with Jenifer of Colorstock 

10:00am - 10:30am

Fri, Sep 1

Dr. Bronner's visit 

点阵在推出时的屏幕截图

这个小例子突出了塑造工作的一些特点。

特性 1：粗糙

塑造阶段的工作是粗糙的。每个人都能看出它是未完成的。他们可以看到他们的贡献将放置的空白空间。太早进行太细致的工作会让每个人都专注于错误的细节。当设计师和程序员需要自己的判断和专业知识来解决所有真正的权衡时，他们需要空间。

特性 2：已解决

尽管粗糙且未完成，但已经对进行了思考。解决方案的所有主要元素都在宏观层面上，并且它们彼此连接。工作没有具体到个别任务，但整体解决方案已经呈现出来。尽管仍可能出现意外情况和问题，冰山仍可能出现，但有明确的方向显示该做什么。任何可能出现的问题或潜在风险都已经被排除，以减少项目的风险。

特性 3：已界定

最后，塑造的工作指示了不应该做什么。它告诉团队在哪里停止。有一个具体的预算，团队被允许在项目上花费的时间。在这固定的时间内完成项目需要限制范围并剔除特定的事物。

综合在一起，粗糙程度留出了团队解决所有细节的空间，而解决方案和边界则起到了保护栏的作用。它们减少了风险并引导团队的努力，确保他们不会构建过多、游荡或陷入困境。

谁来进行塑造

塑造是创造性和整合性的。它需要将界面想法与技术可能性和业务优先级相结合。为了做到这一点，您需要将这些技能作为一名通才来体现，或者与一两个其他人合作。

塑造主要是设计工作。塑造的概念是从用户的角度看到的交互设计。它定义了功能的作用、工作方式以及其在现有流程中的位置。

您不需要成为程序员来进行塑造，但您需要有技术素养。您应该能够判断什么是可能的、什么是容易的、什么是困难的。了解系统工作原理将有助于您看到实现您的想法的机会或障碍。

这也是战略性的工作。确定预算并提出解决方案需要您对问题进行批判性思考。我们试图解决什么问题？为什么它很重要？什么算作成功？哪些客户受到影响？做这件事与做其他事情相比的成本是多少？

塑造是一种闭门创造过程。您可能会独自在纸上绘图，或者与紧密的合作者一起在白板前。在您面前会有粗糙的图表，外面的人无法解释。与合作者合作时，您需要迅速行动，坦率发言，并从一个有前途的立场跳跃到另一个。这是那种私人的、粗糙的、早期的工作。

两个轨道

您无法真正安排塑造工作，因为根据其本质，未塑造的工作是风险和未知的。因此我们有两个单独的轨道：一个用于塑造，一个用于构建。在任何六周的周期内，团队都在构建之前塑造过的工作，而塑造者则在为未来周期可能构建的工作而努力。塑造轨道上的工作是保密的，直到对其进行投资的承诺已经做出。这使得塑造者有在工作进行中将工作搁置或放弃的选择。

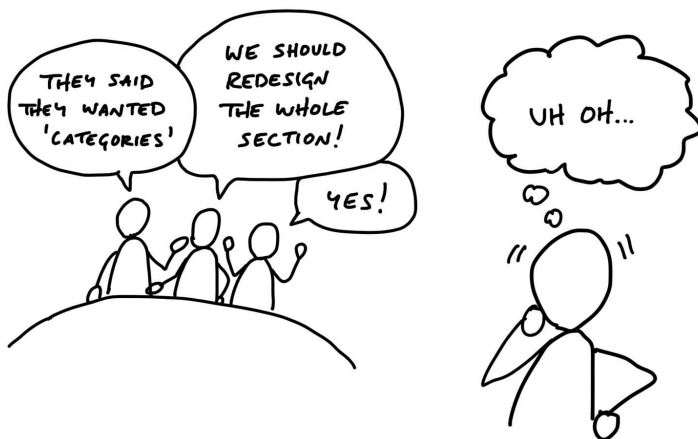
塑造的步骤

塑造有四个主要步骤，我们将在接下来的四章中介绍。

1. **设定界限。** 首先，我们要弄清楚原始想法值多少时间，以及如何定义问题。这为我们提供了塑造的基本界限。

2. **粗略描绘要素。** 然后是创造性的工作，即勾画出解决方案。我们以比线框图更高的抽象层次进行这项工作，以便能够快速移动并探索足够广泛的可能性范围。这一步的输出是一个在预算范围内解决问题的想法，但并没有解决所有细节。
3. **解决风险和潜在风险。** 一旦我们认为我们有了解决方案，我们会仔细研究它，找出可能会让团队陷入困境的漏洞或未解答的问题。我们修改解决方案，将其中的东西剔除，或者在某些棘手的地方指定细节，以防止团队陷入困境或浪费时间。
4. **撰写提案。** 一旦我们认为我们已经塑造得足够可以进行投资，我们会用正式的书面材料称为“提案”将其包装起来。提案总结了问题、限制、解决方案、潜在风险和局限性。提案提交到“投注桌”进行考虑。如果项目被选中，提案可以在启动时向团队解释项目。

设定边界



塑造的第一步是设定我们尝试做什么的边界。我们进行的对话将会完全不同，如果人们认为我们在讨论一个小的改进还是一个重大的重新设计。

关于构建功能的对话总是以 **原始想法** 开始，比如“客户要求群组通知。”在我们都沉浸在讨论解决方案的方式之前，我们应该首先在讨论上设定一些广泛的条款，使其变得高效。

设定期望

有时候一个想法会立即激发我们的兴奋。在这种情况下，我们需要通过检查这个想法是否真的值得我们投入时间来抑制这种兴奋。如果我们不停下来思考这个想法有多有价值，我们可能会很快地投入资源，或者进行长时间的讨论，而最终毫无结果。

其他的想法可能没有那么令人兴奋，感觉更像是一个我们不期望的挑战。客户想要一个日历；我们并不特别想要构建一个，但我们觉得我们需要对请求做出一些回应。

无论是迫不及待还是不愿意深入，明确定义主题值得我们多少时间和精力是有帮助的。如果我们能够控制，这个事情是否值得我们迅速解决？这个大想法是否值得花费整个周期？我们是否会重新设计我们已经有的东西来适应它？如果我们只能将其实现为一个小的调整，我们会考虑吗？

我们称这个为 **期望**。你可以把期望看作是标准团队规模的时间预算。通常我们将期望设置为两种规模：

- **小批量**：这是一个设计师和一两名程序员可以在一两周内完成的项目。我们将这些项目合并成一个**六周**的周期（后面会详细介绍）。
- **大批量**：这个项目需要同样规模的团队整整六周的时间。

在极少数情况下，如果范围太大以至于六周的项目不可想象，我们会尝试通过缩小问题定义来限定范围。如果我们仍然无法缩小范围，我们会分解出一个有意义的部分，使其符合六周的期望。

固定时间，可变范围

期望与估算完全不同。估算从设计开始，以一个数字结束。期望从一个数字开始，以一个设计结束。我们使用期望作为设计过程中的创造性约束。

这一原则称为“固定时间，可变范围”，是成功定义和发布项目的关键。以这本书为例。如果可以随时添加更多内容、解释更多内容或改进已有内容，那么很难出版一本书。当你有最后一周时，你必须做出决定。在还剩一周的时间里，我可以选择是修复错别字还是在章节中添加新的部分。这就是时间、质量和范围之间的张力。我不想发布一本有着令人尴尬的错别字的书，所以我选择通过去掉额外的部分来减小范围。如果没有固定截止日期的压力，我不会做出这种折衷。如果范围不可变，我就必须包括额外的部分。那么就没有时间来修复质量问题。

我们在过程的每个阶段都应用这一原则，从塑造潜在项目到构建和发布它们。首先，期望限制了我们在塑造过程中设计哪种解决方案。稍后，当我们将工作交给一个团队时，固定的时间限制将推动他们做出关于项目核心和次要部分或不必要部分的决定。

“好”是相对的

“最好”解决方案没有绝对的定义。最好是相对于你的约束条件而言的。没有时间限制，总会有更好的版本。最终的晚餐可能是一顿十道菜。但当你饥饿并且赶时间时，一根热狗就很完美。

我们为期望设定的时间量将引导我们走向不同的解决方案。在精致版本中，我们可以对一整套数据库列进行建模，或者只在简单版本中提供一个平面文本区域。我们可以重新设计主要的登陆页面来适应新功能，或者我们可以将其推迟到一个设计约束更少的屏幕上。我们只能根据我们想要花费多少时间和它有多重要来判断什么是一个“好”的解决方案。

响应原始想法

我们对任何想法的默认回应应该是：“有趣。也许以后会考虑。”换句话说，这是一个非常含混的“不”，留下了所有的选项。我们不会将它放入待办列表。我们给予它空间，这样我们就可以了解它是否真的重要，以及可能的含义。

在第一次接触时，现在说“是”或“否”还为时过早。即使我们对此感到兴奋，我们也不应该做出我们还没有理解的承诺。在形成完善之前，我们需要对这个想法进行研究。如果我们总是对着即将到来的请求说“是”，我们最终会积累一大堆工作。

保持冷静和一点扑克脸是很重要的。我们不想关闭一个我们还不理解的想法。明天可能会有新的信息让我们以不同的方式看待它。另一方面，一开始表现出过多的热情可能会让人觉得这件事情会发生。一旦我们将其与我们想做的其他所有事情联系起来，我们可能

就无法承诺它。

缩小问题范围

除了设定期望，我们通常还需要缩小对问题的理解。

曾经有一个客户要求我们提供更复杂的权限规则。她想要的改变很容易需要花费六周的时间来构建。我们没有直接接受她的请求，而是进行了更深入的挖掘。事实证明，有人归档了一个文件，却不知道该文件对使用系统的其他所有人来说会消失。我们意识到我们可以在归档操作本身上放置一个警告，解释其影响。这只需要一天的时间，而不是一个六周的项目。

另一个例子是上一章的“日历视图”。每个人都知道日历视图是什么。但是详细了解它会揭示大量未知的问题和会极大影响范围的决定。如果我们只想花费六周而不是六个月来构建一个巨大的日历，我们如何缩小范围？

在这种情况下，我们从问“我们能构建什么？”转变为问“到底出了什么问题？”当然，日历听起来很不错。但是请求的推动因素是什么？具体来说，在没有这个他们要求的東西的情况下，什么时候会出现某人当前的工作流程出现问题？

案例研究：定义“日历”

在日历请求的案例中，我们拨打了要求此功能的客户。我们不是问她为什么想要一个日历以及它应该是什么样子，而是问她在什么时候需要一个日历。她告诉我们，她在一个办公室工作，办公室的一面墙上画着一个大的日历。她的办公室同事在日历上标记了他们在几个会议室里与客户会面的时间。有一天她在家工作。一个客户打

电话要求她安排一个会议。她不得不开车去办公室看墙上的日历。途中交通非常糟糕，最终没有一个适合她客户的空闲时间。如果她能够在家用电脑上查看日历上的空闲时间，她就可以节省一小时的交通时间和很多的沮丧。

我们得到的见解不是“电脑化日历”——那是显而易见的。我们得知的是，“查看空闲时间”对于这个使用情景来说是重要的，而不是“日历所做的一切”。

这个故事以及类似的故事，为我们设计提供了一个具体的 **基准**。Basecamp 有一个事件议程视图。它用于列出主要的截止日期和里程碑，但对于资源调度来说并不好用，因为你不能在上面看到空白的时间。我们将需求从“日历所做的一切”缩小到“帮助我查看空闲时间，以便我可以安排事情。”

我们还没有解决方案。但现在我们感觉自己有了一个问题，它足够具体，可以激发一个符合我们期望的想法。这使我们想到了上一章中的更简单的“点格”概念。

如果我们无法找出一个具体的痛点或使用情景怎么办？我们的期望也可以告诉我们多少研究是值得的。如果现在不关键，我们无法理解这个问题，我们会放弃它，转而去做其他事情。也许将来会有新的请求或故事出现，给我们更好地理解问题。

当心抓包

在涉及不清晰的想法时，“重新设计”或“重构”通常是最糟糕的。当有人提出像“重新设计文件部分”这样的想法时，这是一个抓包，而不是一个项目。很难弄清楚它的含义，从哪里开始，到哪里结束。下面是一个更有生产力的起点：“我们需要重新思考文件部分，因

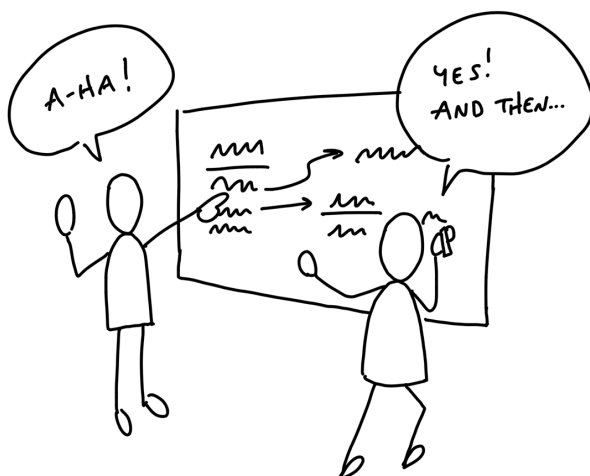
为分享多个文件需要太多步骤。”现在我们可以开始问：什么地方出了问题？在什么情况下有太多步骤？现有设计的哪些部分可以保持不变，哪些部分需要改变？

抓包的一个显著标志是“2.0”标签。我们过去曾犯过一个错误，启动了一个“文件 2.0”项目，而没有真正考虑这意味着什么。我们对改进我们应用程序的一个重大部分感到兴奋。我们知道我们的文件功能存在很多问题，但我们没有问自己我们具体要做什么。这个项目变得一团糟，因为我们不知道“完成”是什么样子。我们通过将项目拆分成小项目来挽救了局面，比如“更好的文件预览”和“自定义文件夹颜色”。我们在每个项目上设定了期望和明确的预期，并成功地发货了。

边界已经建立

当我们拥有了三样东西——一个原始想法、一个期望和一个明确定义的狭窄问题——我们就准备好迈向下一步，定义解决方案的要素。

寻找元素



现在我们已经确定了饥饿感和我们要解决的问题的限制，是时候从语言表达的想法转变为软件解决方案的元素了。解决问题的方法可能有数十种不同的方式。因此，我们需要能够快速行动，涵盖许多不同的想法，而不被拖累。

以正确的速度前进

在这个阶段，有两件事能让我们以正确的速度前进。

首先，我们需要有合适的人员——或者没有人员——在场。要么我们独自工作，要么和一个值得信赖的合作伙伴一起工作，对方能跟上我们的步伐。我们需要能够用简短的语言交流，具有相同的背景知识，并且能够坦诚地跳转不同的想法。

其次，我们需要避免在绘图和素描中陷入错误的细节层面。如果我们从线框图或特定的视觉布局开始，我们会陷入不必要的细节，无法像我们需要的那样广泛探讨。

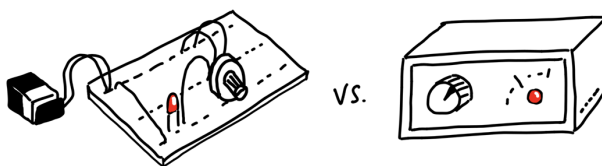
这里的挑战是要足够具体，以便在特定解决方案上取得进展，同时又要不要陷入细节。我们试图回答的问题是：

- 新元素在当前系统中的位置？
- 如何到达它？
- 主要的组件或交互是什么？
- 它会带你去哪里？

为了保持正确的细节层面并记录我们的想法，我们使用手工进行工作，采用一些原型制作技术：面包板和粗笔素描。这些技术使我们能够快速绘制不同版本的整体流程，以便我们可以讨论每种方法的利弊，并且在进行谈论时始终与我们谈论的内容保持一致。

面包板

我们借鉴了电气工程的概念，帮助我们以正确的抽象级别进行设计。面包板是电气工程的原型，具有真实设备的所有组件和布线，但没有工业设计。



决定是否包括指示灯和旋钮是非常不同的，与讨论底盘材料、旋钮应该放在灯的左边还是右边、角落应该有多锐利等等完全不同。

同样，我们可以勾画和讨论界面想法的主要组件和连接，而不指定特定的视觉设计。为此，我们可以使用简单的速记法。我们会绘制三种基本的东西：

1. 地点：这些是你可以导航到的东西，比如屏幕、对话框或弹出的菜单。
2. 作用：这些是用户可以操作的东西，比如按钮和字段。我们认为界面上的文字也是一种作用。阅读它是一种行为，为用户提供后续行动的信息。
3. 连接线：这些显示作用如何将用户从一个地方带到另一个地方。

我们将使用文字来描述每样东西，而不是图片。重要的是我们所确定的组件及其连接。它们使我们能够展开一个想法，并判断行动的顺序是否符合我们试图解决的用例。

例子

假设我们的产品是一个发票工具。我们正在考虑添加一个新的“自动付款”功能，以使我们的客户的客户能够自动支付未来的发票。

如何打开自动付款？涉及什么？我们可以选择一个起点，比如说客户着陆在一个发票上。这是我们的第一个地点。我们通过写出地点的名称并在下面加下划线来绘制它。

INVOICE

在发票上，我们想着可以添加一个新按钮“打开自动付款”。这是一个作用。作用出现在该地点的下方。

INVOICE
TURN ON
AUTOPAY

这个按钮要放在哪里？放在一个设置自动付款的地方。我们不必指定它是一个单独的屏幕还是一个弹出模态窗口，或者其他什么。从连接的角度来看，它都是一样的。让我们画一条连接线从按钮到“设置自动付款”屏幕。

INVOICE
TURN ON
AUTOPAY

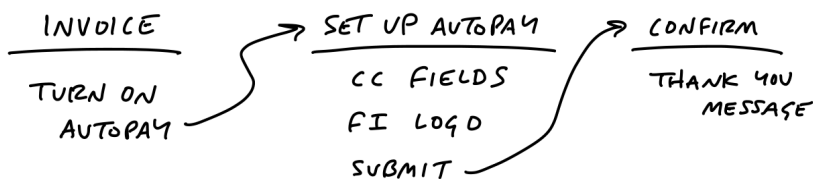


SET UP AUTOPAY

现在我们可以讨论该屏幕上应该有什么。我们是否在这里要求信用卡？有没有存档的卡？其他付款方式呢？

仅仅弄清楚在线条下面写什么就开始引发关于要构建什么的讨论和辩论。

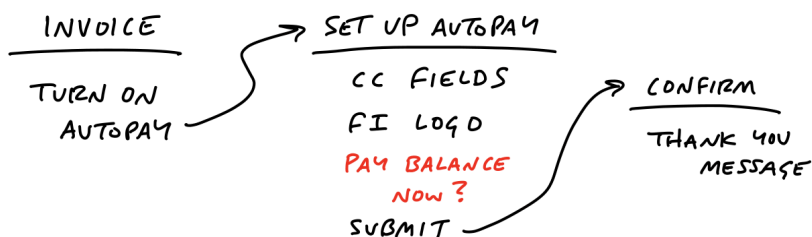
在我们深入思考后，我们决定在这里要求信用卡详细信息，并显示金融机构的标志（这是这个特定产品领域的一个方面）。



非常直截了当。但等等——我们是否实际支付了原始发票？嗯。现在我们既有功能性的问题，也有界面的问题。启用自动付款到底做了什么？它仅适用于未来还是使用自动付款首次付款也支付了当前的发票？我们在开始通过面包板的几个字和箭头引发了更深层次的问题和讨论。

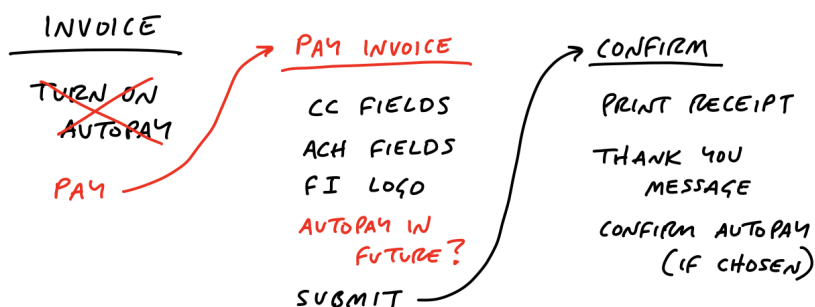
由于我们使用的是如此轻量级的标记法，而且我们没有被线框图所束缚，我们可以迅速地跳转并考虑不同的可能性。

我们可以在设置屏幕上添加一个选项...



但是现在我们在复杂化确认屏幕的责任。我们需要展示支付余额时的收据。确认页面是否应该有一个条件，有时显示刚刚支付的金额的收据？

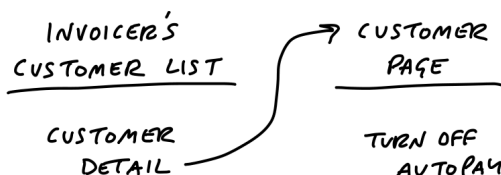
还有完全不同的方法。与其在发票上开始，不如在付款时提供自动付款选项。这样就不会有关于当前金额是否已经支付的歧义了。我们可以在现有的付款确认页面上额外添加一个“启用自动付款”提示。



将这些想法勾勒出来提醒了我们当前的付款表单也支持 ACH，而不仅仅是信用卡。我们讨论并确认我们也可以使用 ACH。

启用自动付款后怎么办？客户如何关闭？到目前为止，系统中的许多客户都没有用户名或密码。他们通过令牌化的链接逐个支付发票。一个自然的假设是现在客户有了类似自动支付的东西，他们需要用户名和密码，并且需要一个管理它的着陆地方。

在这种情况下，团队决定在当时增加用户名/密码流程的范围太大。从战略上反思他们对客户的了解，他们认为如果发票方的客户必须联系发票方请求关闭自动付款，那也是可以接受的。在这种情况下，我们可以为发票方已经提供的客户详细信息页面添加一个单个选项来禁用自动付款。我们呈现的流程如下：



这个例子说明了在面包板阶段所要追求的思考水平和速度。写出这些流程使我们面对了最初没有考虑到的问题，并激发出设计想法，同时没有让我们陷入不重要的视觉选择。

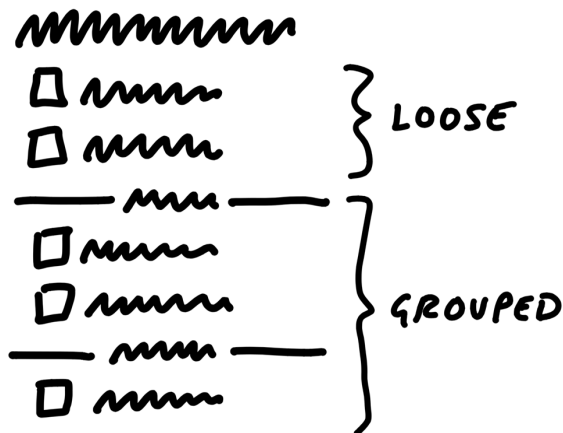
粗笔素描

有时我们心目中的想法是视觉的。面包板方法可能就无法满足要求，因为元素的二维布局是最基本的问题。在这种情况下，我们仍不希望在线框图或不必要的细节上浪费时间。因此我们会使用粗笔素描。

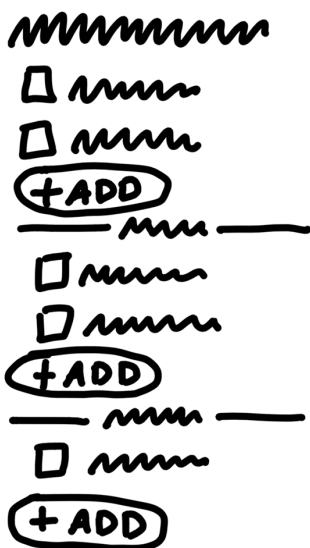
粗笔素描是用宽笔画出的素描，难以添加细节。最初我们用较粗的油性记号笔在纸上做这个，现在我们会 iPad 上使用设置为大直径的笔来做。

这是一个例子。我们经常在 Basecamp 的待办事项清单中创建一些假的待办事项作为分隔符。我们会创建一个项目，比如“—— 需要测试 ——”，并将项目放在下面。我们想要在我们的待办工具中添加某种官方分隔符功能，以将解决方法变成待办事项列表的第一类功能。

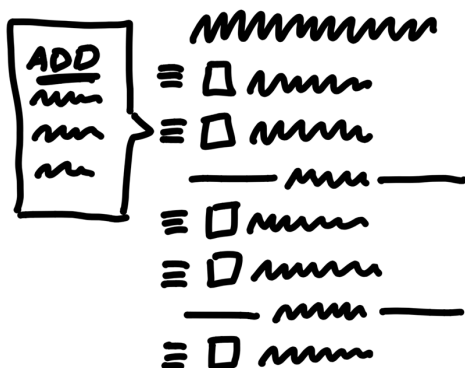
我们必须弄清楚添加分隔符的影响。我们粗略地构想了一个想法，即添加一个分隔符可以将列表分为分隔符上面的“松散”待办事项和下面的“分组”待办事项。添加后续分隔符将在“松散”项目的下方添加更多的分组。



我们可以通过每个分组内的某个作用添加项目，包括顶部的“松散”组。



我们有点担心添加按钮可能会破坏列表的整体感，而且各个组可能会和页面上的列表过分分离。我们讨论了将“添加”作用放在我们已有的每个待办事项项的左边的菜单中的可能性。



这种标记法没有面包板那么约束性，但也有缺点。我们可能会绘制一个侧边栏，并对这样一个布局元素产生依赖，尽管它并不是一个核心元素。但只要我们留心，我们仍然要比过早制作线框图要好得多。

将粗笔素描称为一种技术或工具可能看起来有点愚蠢。之所以这样称呼它们，是因为我们很容易跳到错误的细节层面。给这个粗糙的早期阶段起个名字，并使用一个特定的工具来做这个阶段有助于将我们的创意过程进行分割，并确保我们在尚未充分调查领域的情况下不会跳到某个具体想法的细节。

元素是输出

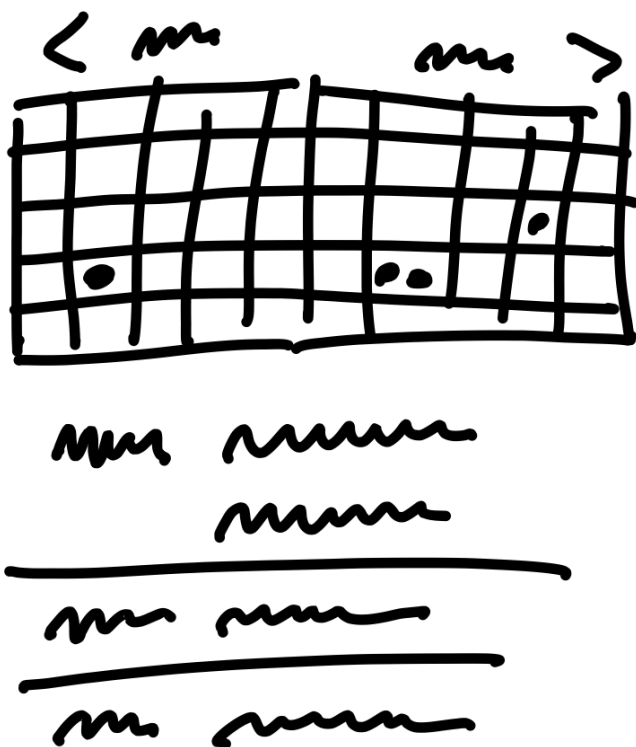
以自动支付的例子来说，我们最终得到了一些清晰的元素：

- 在现有的“支付发票”屏幕上，一个新的“使用此自动支付？”复选框
- 在发票方这边有一个“禁用自动支付”的选项

对于待办事项分组项目，元素是：

- 在第一组上方的松散待办事项直接属于父级
- 在松散待办事项下面出现的分组待办事项
- 我们想尝试在每个部分内添加一个“添加”作用，但如果在视觉上不起作用，我们也可以依靠现有的操作菜单将待办事项插入到相应位置。

同样，在我们为日历网格上的事件渲染简化解决方案进行素描时，我们使用了粗笔标记法。



这使我们能够确定解决方案的主要元素：

- 一个两列的月度日历网格
- 事件用圆点表示，没有跨度的条状图案
- 在圆点下面滚动的事件列表，当你点击一个圆点时，一个事件会被带到视图中

这些元素清单与“月度日历”相比非常狭窄和具体。这正是我们希望通过塑造过程来完成的缩小范围的工作。

设计师的空间

稍后，当需要设计师参与时，您不希望不得不说“我知道我画成这样，但请忽略那个……”。无论您说什么，任何具体的模型都会对其他人在您之后的举动产生偏见——尤其是如果您的职位高于他们。他们会把初始模型中的每一个细节都视为指导，即使您并不是这样打算的。

在正确的“抽象层次”上工作不仅可以确保我们以正确的速度前进，也为后期的创造力留下了重要的空间。

通过留出细节，面包板和粗笔方法为项目的后续阶段留出了设计师的空间。

这是塑造过程的一个主题。我们让项目变得更具体、更具体，但仍然留下了许多决策和选择的空间。这不是一个规范。这更像是游戏的边界和规则。一旦玩游戏的时候，它可以朝无数不同的方向发展。

尚未可交付

塑造的这一步仍然非常私人化。在这一点上的工件——挂在墙上或在您的笔记本上——对于没有和您在一起的人来说几乎是无法理解的，这是很正常的。

我们已经从一个模糊的想法，比如“自动支付”或“待办事项分组”，转变为了一个具体的方法和一些具体的元素。但是我们所拥有的形式仍然非常粗糙，大部分还是概要。

我们已经找到了解决问题的方法。但在我们考虑将其安全地交给团队进行成功构建之前，可能还有一些重要的未知因素或需要解决的问题。

下一步是进行一些压力测试和降低风险。我们想要检查可能会妨碍项目在我们为其设定的固定时间安排内发货的漏洞和挑战。

之后，我们将看看如何将塑造的概念整合成一份推介文书。

没有传送带

另外请记住，在这个阶段，我们可以放弃这个项目。我们还没有对其下注。我们还没有对它做出任何承诺或保证。我们所做的是通过使其更具可操作性增加了 **原始想法** 的价值。当需要分配资源的时候，我们已经更接近一个好的选择。

风险和兔子洞



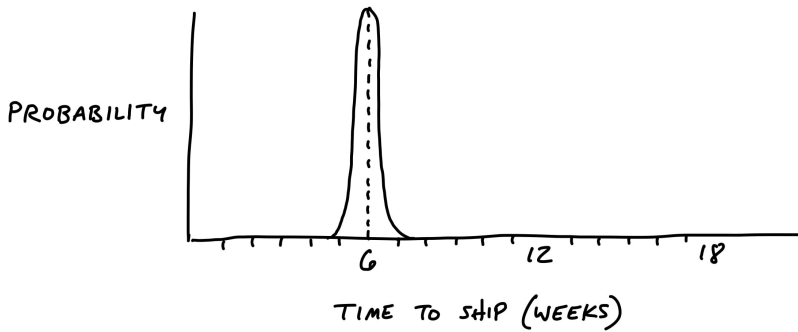
记住，我们正在为一个固定的时间窗口塑造工作。我们可能相信根据我们的经验，我们在前一章中详细制定的元素可以在项目期限（六周）内完成。但我们需要更仔细地审视，因为概念中只需要一个漏洞就足以使项目偏离轨道。假设我们对项目进行了押注，团队接手了。如果他们遇到一个需要两周时间解决的意外问题，他们就花掉了预算的三分之一！

更糟糕的是，有时候你会遇到不仅会延误项目的问题，还没有明显解决方案的问题。我们曾经押注于一个重新设计在 Basecamp 主屏幕上与客户呈现项目的方式的项目。我们假设设计师会想出解决方案；我们没有在塑造阶段进行工作来验证可行的方法是否存在。一旦项目启动，情况变得比我们预期的要困难得多。我们当中没有人能在我们预算的六周内找到合适的设计解决方案。我们最终放弃了项目，并在以后重新考虑了它。

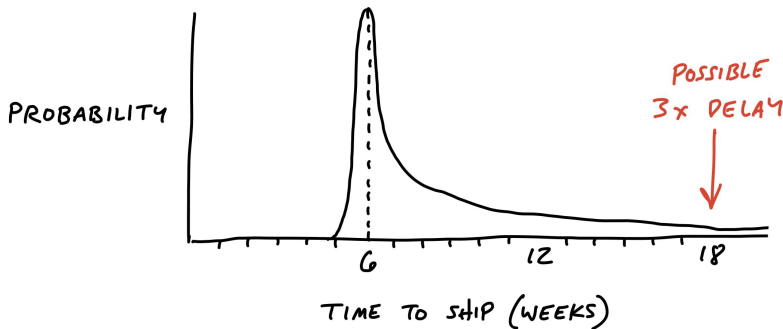
当然，总会有未知因素。这就是为什么我们在第三部分应用了许多实践，让团队按照正确的顺序解决问题，为意外情况留出空间。但这并不意味着我们不应该在押注项目之前寻找可以提前找到并消除的陷阱。在我们考虑对项目进行押注之前，一个塑造好的项目应尽可能没有漏洞。

不同类别的风险

在风险方面，塑造好的工作看起来像一个薄尾概率分布。它有一点可能需要额外的一周时间，但除此之外，解决方案的元素足够明确定义和熟悉，以至于没有理由它应该拖得比那更长。



然而，如果在塑造中存在任何兔子洞——技术上未知的因素，未解决的设计问题，或者误解的相互依赖——项目可能需要多次原本的周期才能完成。右尾会拉长。



我们希望从项目中消除未知因素和棘手的问题，这样我们的概率就会尽可能地薄尾。这意味着项目由相互独立、理解良好的部分组成，它们以已知的方式组装在一起。

寻找兔子洞

详细制定解决方案是一个快速移动的、探索性的过程。这更多地涉及广度而不是深度。在这一步，我们要放慢脚步，批判性地审视我们得出的结果。我们有没有遗漏任何事情？我们是否对技术做出了不公平的假设？

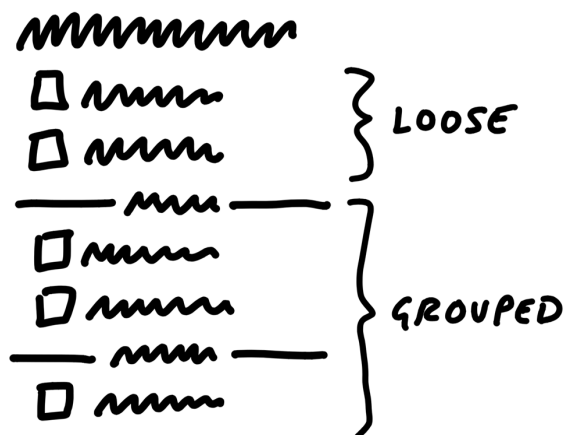
分析解决方案的一种方法是慢动作地通过一个使用案例。根据我们勾画的解决方案，用户究竟如何从起点到终点？减慢速度并将其演示出来可以揭示可能存在的空白或缺失的部分，这些都需要我们进行设计。

然后，我们还应该质疑我们认为已经解决的每个部分的可行性。我们问自己诸如：

- 这是否需要我们以前从未做过的新技术工作？
- 我们是否对部件如何组合做出了假设？
- 我们是否假设存在一个我们自己无法想出的设计解决方案？
- 我们是否应该提前解决一个难题，以免团队陷入困境？

案例研究：填补漏洞

例如，当我们定义“待办事项组”项目时，我们介绍了待办清单中的分隔符的概念：

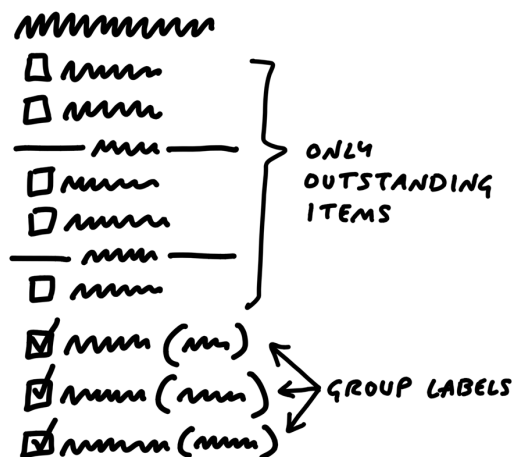


我们喜欢分隔符的概念，松散的与分组的待办事项的逻辑对我们来说是有道理的。但当我们仔细观察后，我们意识到我们没有解决如何显示已完成项目的问题。在现有的设计中，最近完成的几个项目会显示在列表下方。现在，我们应该在每个组的底部呈现已完成的项目吗？还是应该继续在底部显示已完成的项目，并在已完成项目部分内重复相同的分隔符集？我们是否应该重新考虑如何完全处理已完成的项目？

这是概念中的一个漏洞。如果我们不加以解决，我们就会把一个深层次的设计问题推给团队，并不合理地要求他们在截止日期内找到解决方案。这不负责任，我们不应该给团队一个错综复杂的相互依赖的问题，然后要求他们在短暂的固定时间窗口内解决它。

我们根据经验知道，改变已完成的待办事项呈现方式会在用户体验、导航和性能方面产生许多复杂的影响。为了消除项目中的不确定性，我们决定在塑造的概念中规定一个解决方案。我们将保持已完成的项目与以前的工作方式完全相同。我们不会对它们进行分组或分段，而是只需在每个已完成项目后附加组的名称。这可能会有

点混乱，但我们权衡了这一折衷：它极大地简化了问题，而且我们仍然可以在组的详细页面上显示该组的已完成项目。



当你在压力下工作时，做出这种折衷是很困难的。有很多理由可以证明使用不同的设计或深刻重新考虑已完成的待办事项会更好。为什么不尝试在每个组内渲染它们？设计师可能会合理地想：“也许如果我再稍微调整样式，它们就能更好地融入。”他们可能会在他们所剩无几的几周时间内走进死胡同。

作为塑造者，我们考虑的不是最终设计，而是基本的质量和风险。通过这种折衷的概念，我们可以保留使项目有价值的所有元素——未完成项目的组——并减少大量的风险。

接下来，在我们为这个项目撰写提案时，我们将指出这个特定的“修补”作为概念的一部分。这样，以后的任何人都不会因此而受到困扰。

宣布超出范围

由于团队中的每个人都希望做出最好的工作，当然他们会寻找所有的用例并认为它们是必要的。随着团队对 [范围限制](#)（见 [决定何时停止](#)）变得更加熟悉，情况会有所改善。但明确指出任何你特意不支持的情况仍然是一个好主意，以确保项目完全在预算范围内。

例如，我们曾对在 Basecamp 中通知群组成员的想法进行了研究。与其一个一个地勾选五个程序员，你只需点击“程序员”，他们就会被选择进行通知。当我们观察产品时，我们看到了许多地方可能需要这种行为。如果我们让你在发布消息时选择一个群组，为什么在指定待办事项或在聊天室提及人员时不能选择一个群组呢？

我们决定，项目的核心价值是缩小要通知的消息对象范围。我们明确标记了项目中的其他情况为“超出范围”，并集中精力处理我们想要的优势：加快发布消息的流程。

精简

我们在草拟阶段激动人心的解决方案中，可能有一些并不是真正必要的部分。当我们设计“待办事项组”功能时，我们认为为不同组标记颜色可能会很棒。毫无疑问，页面上用颜色标记的组标签看起来更有趣，而且这个功能可能也更有用。但我们决定将其标记为不必要，并从项目的核心中将其剔除。我们可以将其作为一个好的补充提及给团队，但每个人都应该以该功能有价值的假设开始。

向技术专家展示

到目前为止，塑造工作一直是一项闭门活动。在你准备好将想法更广泛地分享之前，你可能需要对一些你不太确定的概念部分进行调查。也许有一个技术上的假设，你需要向更了解代码的人验证一下。或者你想确保使用数据不会与你对当前客户行为的假设相矛盾。

这是一个很好的时机，可以找一些技术专家，向他们介绍这个想法。告诉他们这只是一个想法。这是你正在塑造的一个潜在押注，而不是即将推出的东西。氛围要友好一些：“这是我正在考虑的一个东西……但我还没有准备好向任何人展示……你觉得怎么样？”

小心简单的问题：“这个可能吗？”在软件中，一切都是可能的，但没有什么是免费的。我们想知道的是它在我们正在塑造的周期内是否可行。与其问“做 X 是否可能？”不如问“在 6 周内做 X 可能吗？”这是一个非常不同的问题。

谈论一下这个解决方案是如何在预算范围内的一个好解决方案，这样他们就会成为保持项目规模的合作伙伴。并强调你正在寻找可能炸毁项目的时间炸弹。这不仅仅是一个“你觉得怎么样”的对话——我们真的在寻找可能在项目承诺给团队后炸毁项目的时间炸弹。

试着让白板保持湿润。与其撰写文档或制作幻灯片，不如邀请他们来到白板前，重新绘制之前制定出来的元素，从头开始建立概念。完全坚持你已经制定出来的概念，以便获得对你已经完成的工作的反馈。然后一旦你覆盖了你已经完成的工作，打开讨论并邀请他们提出修改意见。在看到这个概念后，他们是否有任何关于如何大幅简化或以不同方式处理问题的见解？

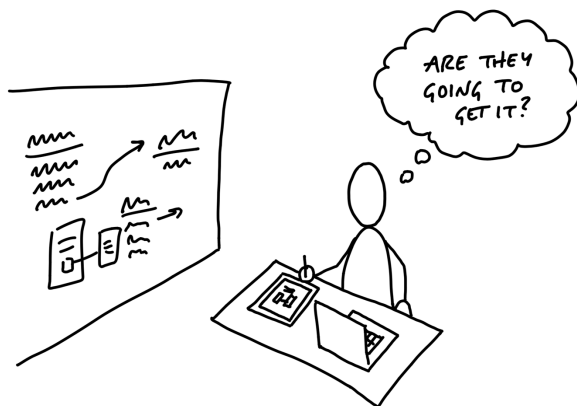
根据对话的进行方式，你可能要么验证了你的方法，要么发现了一些问题，需要你重新进行塑造。

降低风险，准备撰写

在这个阶段结束时，我们已经有了解决方案的元素、填补潜在兔子洞的补丁，以及我们宣布超出范围的区域。我们已经从一个潜在风险的粗略形式的解决方案，转变成了一个坚实的想法，我们希望以后能够在它上面进行押注。

这意味着我们已经准备好从私下塑造和从内部获得反馈的阶段转变到在 **押注桌** 上展示这个想法。为此，我们将以一种形式将其写出来，以便与较少背景的人能够理解和评估。这个“提案”将是我们游说获取资源、收集更广泛的反馈（如果需要），或者简单地记录这个想法以备将来时机更加成熟的文档。

写一个提案



现在我们已经有了解决方案的要素，并且我们已经降低了我们的概念风险，以至于我们相信这是一个可以交给团队的好选择。但是这个概念仍然停留在我们的头脑中，或者只是在白板上或者我们的笔记本上一些难以理解的画面。现在我们需要把这个概念表达出来，让其他人能够理解、消化和回应。

这就是我们说“好的，现在可以写成提案了”的地方。在这一章中，我们将详细介绍提案的要素，并展示 Basecamp 真实项目中的一些完整实例。

提案的目的是提出一个良好的潜在赌注。它基本上是一个演示。提案的要素包括我们需要捕捉到目前为止所做的工作，并以一种能够让安排项目的人员做出明智抉择的形式呈现出来。

在提案中，我们始终希望包括五个要素：

1. **问题** —— 原始想法、使用案例，或者我们看到的激发我们解决这个问题的某些东西
2. **兴趣** —— 我们想花费多少时间以及这如何限制解决方案
3. **解决方案** —— 我们想出的核心要素，以一种易于他人立即理解的形式呈现
4. **兔子洞** —— 值得注意的解决方案细节，以避免问题
5. **不做** —— 从概念中明确排除的任何事物：故意不涵盖的功能或使用案例，以适应兴趣或使问题可行

要素 1. 问题

同时提出问题和解决方案至关重要。这听起来似乎是一个显而易见的观点，但令人惊讶的是，团队，包括我们自己在内，经常直接跳到解决方案，假设构建这个东西的好处是显而易见的。

直接深入“要构建什么”——解决方案——是危险的。如果没有问题，就没有办法讨论这个解决方案是否好还是坏。例如，“在 iPad 应用程序中添加标签”可能会吸引 UI 设计师，但有什么能阻止讨论演变成对不同 UI 方法的长时间辩论呢？没有具体的问题，就没有测试适用性的基础，判断一个解决方案是否比其他解决方案更好。

明确问题还让我们在以后提出或进行赌注时能够更清晰地交谈。解决方案可能是完美的，但是如果问题只发生在已知对产品不太合适的客户身上怎么办？我们可能会花六周时间设计一个巧妙的解决方案，但只有低留存率的小部分已知客户受益。我们希望能够将有关需求的讨论与解决方案分离开，这样我们就不会花时间在对不适合的人群没有好处的好解决方案上。

你必须对问题的解释有多深入取决于你与阅读写作的人分享了多少背景信息。最好的问题定义包括一个具体的故事，表明现状为什么不奏效。这为你提供了一个**基准**，以便测试适应性。人们能够将解决方案与这个具体问题——或者如果有辩论的话，其他解决方案——进行权衡，并判断新解决方案替换后，这个故事的結果是否更好。

要素 2. 兴趣

你可以将兴趣看作是问题定义的另一部分。我们不仅想解决这个问题使用案例，还想想出一个方法，在六周内完成，而不是三个月，或者——在 **小批量** 项目的情况下——两周，而不是整整六周。

在提案中陈述兴趣可以防止无效的讨论。总是会有更好的解决方案。问题是，如果我们现在只关心在这上面花两周的时间，那么这个具体的解决方案看起来如何？

任何人都可以建议昂贵和复杂的解决方案。要想得到一个在小时间框内适合的简单想法，需要工作和设计洞察力。陈述兴趣并把它作为一个约束将每个人都变成了这个过程的合作伙件。

要素 3. 解决方案

与没有问题的解决方案一样，有时公司会赌上没有解决方案的问题。“我们真的需要让在消息部分更容易找到东西。客户对此抱怨。”

这不足以提出或赌注。没有解决方案的问题是未成形的工作。把它交给团队意味着把研究和探索推到错误的层面，这里的技能、时间限制和风险概况（薄尾与重尾）都不匹配。

如果没有解决方案，有人应该回去做形状工作上的形状追踪。只有当问题、兴趣和解决方案汇集在一起时，它才准备好赌注。然后你可以仔细审查问题和解决方案之间的匹配度，并判断它是否是一个好的赌注。

帮助他们看到

在要素阶段，以正确的抽象级别勾画出想法非常关键，这样我们才不会放慢速度，也不会丢失我们的大脑角落和舌尖上出现的任何想法。

当我们写提案时，我们也需要以正确的细节级别进行绘制。这里的挑战有点不同。我们有时间慢下来，准备一个适当的演示。我们需要保持高层次，但比独自工作或与合作伙伴工作时要增加一些具体性。阅读提案并查看绘图的人如果没有太多上下文，需要“理解”这个想法。

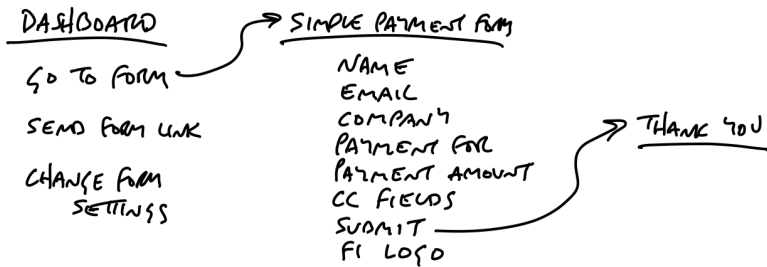
我们需要更多的具体性，但我们不想用线框或高保真模型过度规范化设计。这会限制后来进行工作的设计师。我们还有可能把讨论引向与实际形状工作无关的颜色、比例或布局等主题。

同时，手写面包板有一种“你必须亲自在场才能理解”的质感。对于那些没有逐步观看面包板展开的人来说，它可能看起来像是一团词语和箭头的混合。

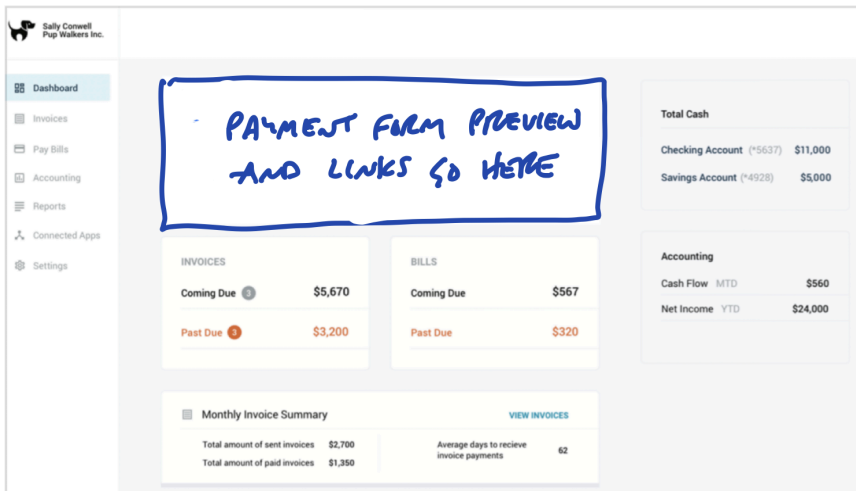
因此，在不过多涉及无关细节的情况下，我们需要一些技巧来帮助人们看到这个想法。

嵌入式素描

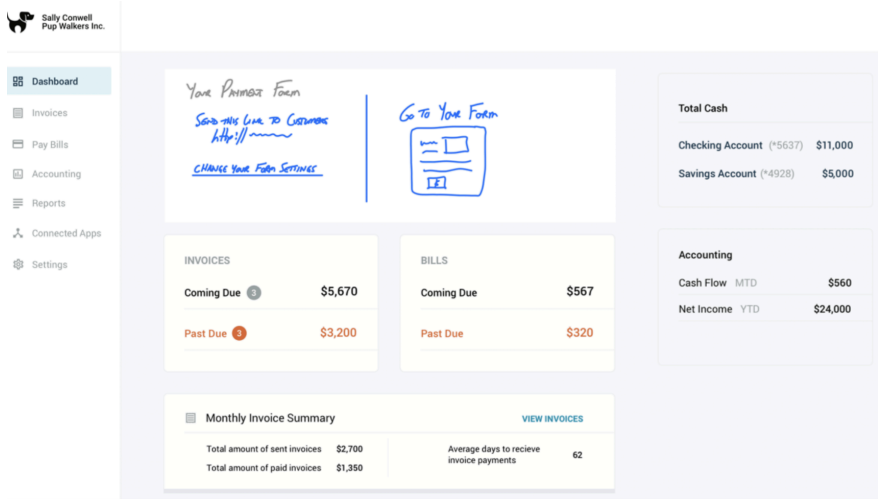
假设你的面包板从形状会话中看起来像这样：



人们可能会很难想象这些新的辅助功能在仪表板上的位置。我们可以在仪表板上草绘一个新的框，使其更清晰：



但我们仍然要求人们想象太多。在这里，进入粗笔细节是值得的。



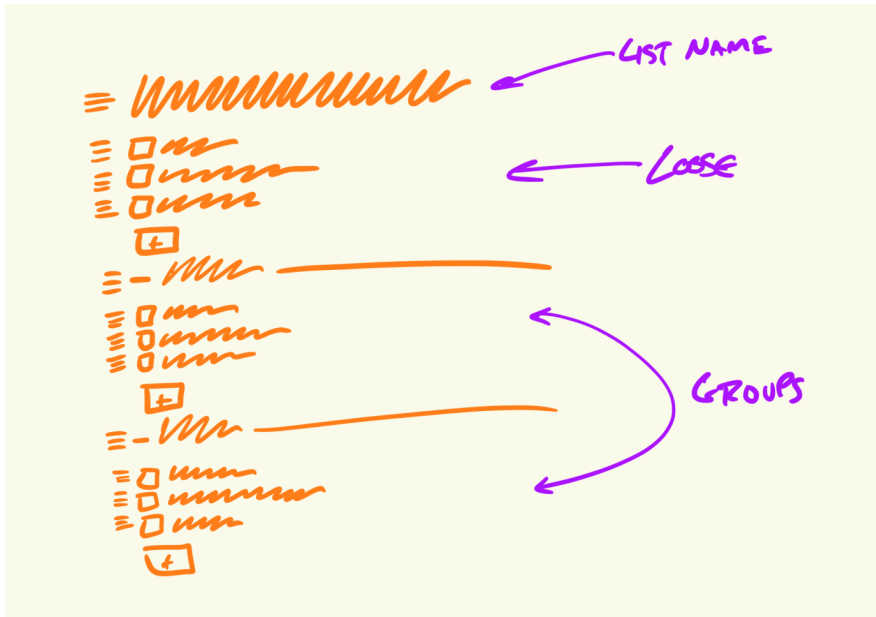
这样可以更容易地看到这些元素，并评估这个功能在仪表板上的表现是否清晰。缺点是我们已经陷入了一些布局决策，这是最好避免的。设计师应该随意找到与框架不同的设计。我们会在提案中添加一个免责声明，提醒设计师他们应该采取的自由度。

这是一个有选择地进入更多视觉细节的例子，因为我们需要它来推销概念。幸运的是，在概念的其他部分，我们不需要做出太多的视觉决策。这是设计的一个“关键”部分，每个人都必须以具体的方式看到，才能“理解”它。

注释的粗笔素描

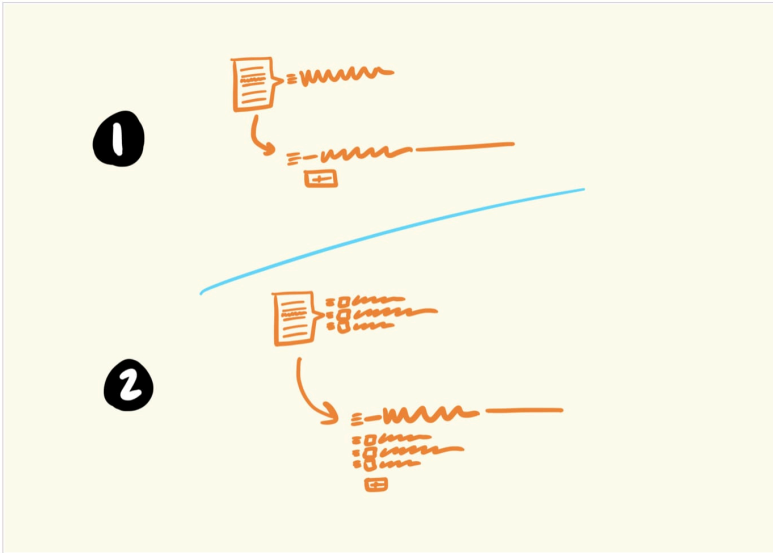
有时候，想法本身是有视觉效果的，或者表达起来有点复杂。粗笔素描在提案中可能非常有效；你只需要更注意地标记它们。

在 iPad 上重新绘制素描——仍然使用粗笔尺寸——效果很好。你可以使用不同的颜色将标签与素描的实际部分分开。



或者你可以添加一些标注，以便讨论特定的元素。

You can create a divider in two ways...



Two ways to make a divider - 152 KB

要素 4. 兔子洞

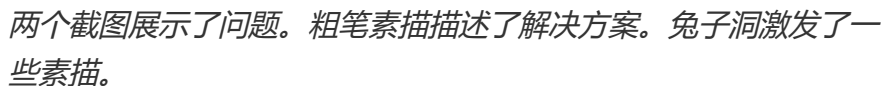
有时候解决一个兔子洞只需要几行文字。例如，在上面的付款表单项目中，塑形者想要强调如何创建 URL 的特定解决方案。这种 URL 在项目的 v1 中永远不会出现在自定义域上。这种东西虽然不是概念的核心，但把它明确表示出来，就填补了一个潜在的兔子洞。

要素 5. 不做

最后，如果在这个概念中有任何我们 不 做的事情，最好在这里提及。在付款表单项目的情况下，团队事先决定不允许对表单进行任何 WYSIWYG 编辑。用户只能在单独的“自定义”页面上提供标志和自定义标题文本。在某些人看来，WYSIWYG 可能更好，但考

实例

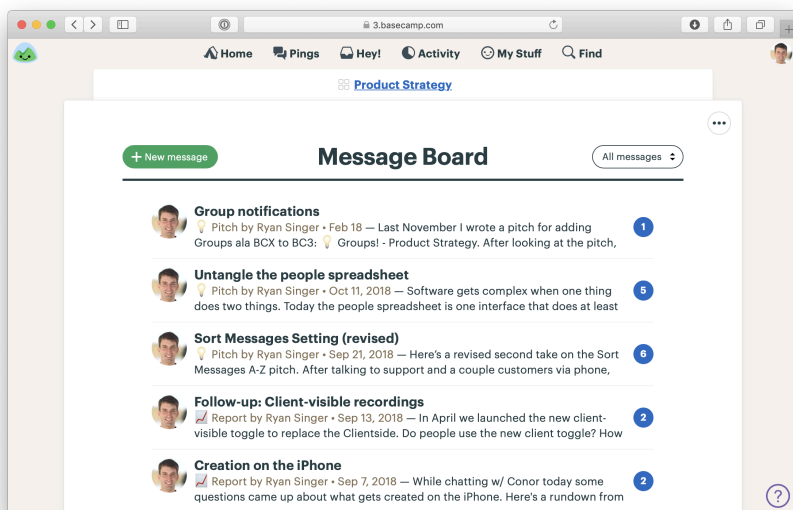
这个关于将待办事项分组在一起的提案开始时展示了人们在当前设计中正在使用的一种解决方法。然后它勾画出了启用可选待办事项分组的所有主要想法。



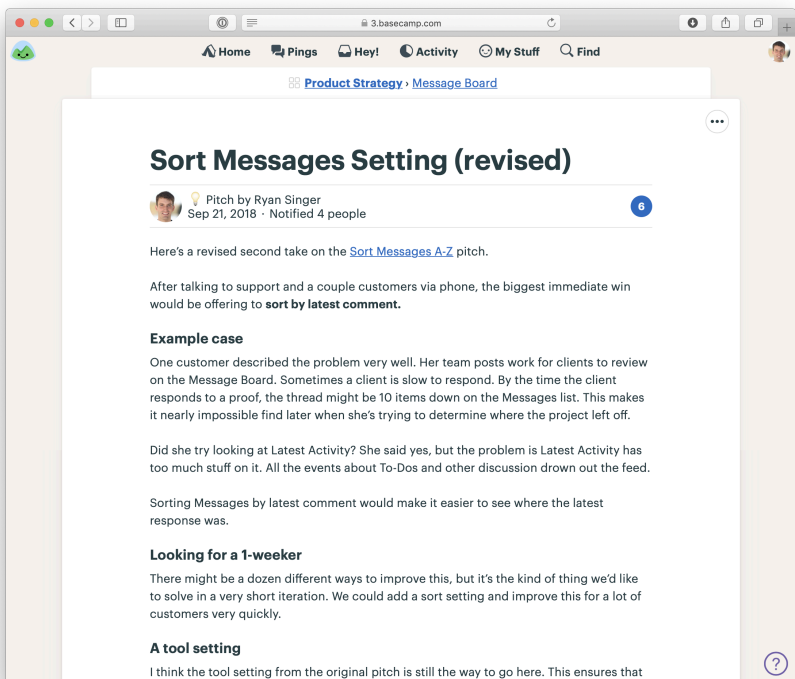
59 / 163

Basecamp 中的实施方式

我们在 [Basecamp](#) 中将提案发布为消息。我们创建了一个名为 *Pitch* 的消息分类，以便我们可以轻松找到它们。提案发布到一个名为 *Product Strategy* 的团队，只有在 [投注桌](#) 上的人才能访问。

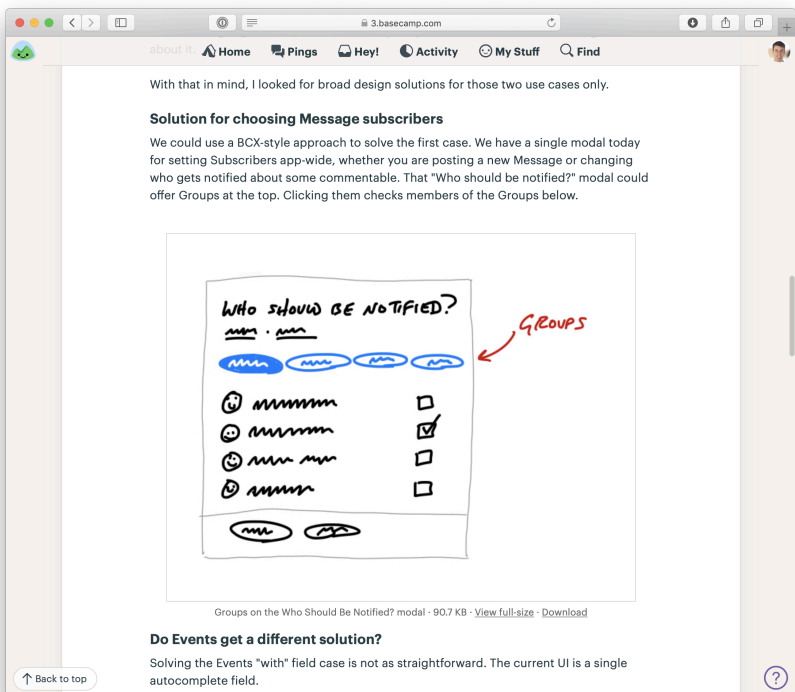


在 Basecamp 团队的产品策略消息板上的提案



作为消息的提案。注意一周的 **需求**。这是一个 **小批量** 项目。

当我们需要在提案中包含 **粗线条素描** 时，我们会在 iPad 上画出来（使用 [Notability](#)），然后截图。Basecamp 的文本编辑器使插入图像并加上标题变得很容易，这样它们在提案流程中就能够有意义。



在提案中间画出的 iPad 素描

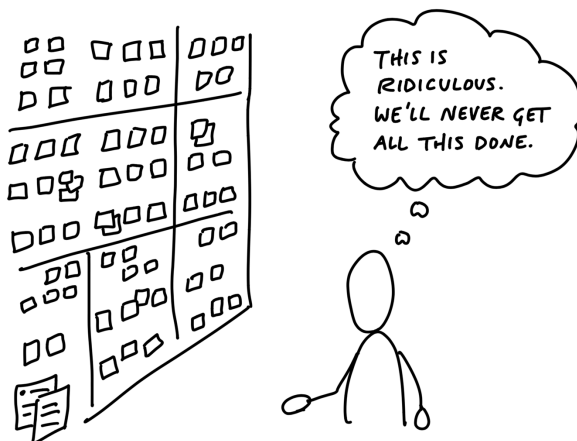
人们会对提案进行异步评论。不是为了说“是”或“否”——这是在投注桌上发生的——而是为了指出问题或补充缺失的信息。



我们的 CTO 对提案进行了技术层面的回应。

在下一章中，我们将更详细地了解投注过程，看看提案到底会经历怎样的流程，以及我们如何将它们变成计划中的项目。

投注，而不是积压



现在我们已经写了一个提案，它应该去哪里？不应该放到一个积压列表里。

没有积压列表

积压列表是一个我们不需要承担的沉重负担。数十甚至数百个任务堆积起来，我们都知道我们永远没有时间去完成。这个不断增长的堆栈让我们觉得好像我们一直落后，尽管事实并非如此。仅仅因为某人在一个季度前认为某个想法很重要，并不意味着我们需要一遍又一遍地去审视它。

积压列表也是浪费时间的大户。不断审查、整理和组织旧想法所花费的时间阻止了每个人在当下真正重要的及时项目上取得进展。

一些潜在的投注选项

那么我们该怎么办呢？在每个六周周期之前，我们都会举行一个**投注会议**，在这里利益相关者决定下一个周期要做什么。在投注会议上，他们会查看过去六周的提案，或者任何某人故意重新提出并游说的提案。

其他什么都不会摆上桌面。没有一长串要审查的想法。不会花时间整理一个积压的旧想法列表。只有一些精心制定、风险降低的选项需要审视。这些提案是潜在的投注选项。

由于只有少数选择和六周的周期，这些会议是不经常的、短暂的，而且高效率的。

如果我们决定对一个提案进行投注，它就会进入下一个周期进行开发。如果不投注，我们就放手让它离开。我们没有必要追踪或保留任何东西。

如果这个提案很棒，但时间不合适呢？任何想要再次支持它的人都可以独立地追踪它，然后在六周后再次游说。

分散的列表

我们不必在一个沉重的积压列表和不记得过去任何事情之间做选择。每个人仍然可以独立地跟踪提案、错误、请求或他们想要独立完成的事情，而无需一个中央积压列表。

支持团队可以保留一个经常出现的请求或问题列表。产品团队会跟踪一些他们希望在未来周期能够塑造的想法。程序员会在有空时维护一个他们想要修复的错误列表。没有一个中央的积压列表，这些列表也不是投注过程的直接输入。

部门之间定期但不频繁的一对一会议有助于交叉传播下一个要做什么的想法。例如，支持团队可以告诉产品团队他们正在看到的顶级问题，然后产品团队可以独立跟踪，作为未来需要塑造的潜在项目。也许产品团队只解决其中一个顶级问题。然后，在未来的一对一会议中，支持团队可以再次游说一些尚未得到关注的事情。

这种方法分散了优先考虑和追踪工作的责任，并使其可管理。来自不同部门的人可以支持他们认为重要的任何事情，并使用适合他们的方法来追踪这些事情，或者不追踪。

这样，对话总是新鲜的。任何被重新提出的事情都是在一个背景下、由一个人、带着一个目的。一切都是相关的、及时的，与当下紧密相连。

重要的想法会回来

很容易高估想法。事实上，想法很廉价。它们经常出现并积累成一大堆。

真正重要的想法会再次出现在你面前。你上一次忘记了一个真正伟大、鼓舞人心的想法是什么时候？如果不是那么有趣——也许是顾客偶尔遇到的一个错误——当顾客再次抱怨或新顾客遇到这个问题时，它会再次引起你的注意。如果你只听到一次，再也没有听到，

也许它并不是真正的问题。如果你一直听到，你会有动力塑造一个解决方案，并在下一个周期的投注时间上游说。

投注桌



现在我们有一些很有潜力的提案，是时候决定安排哪些项目了。

六周周期

如果我们无法轻松确定谁有空并且有多长时间，那么承诺时间和人员就会变得困难。当人们由于项目重叠而在不同时间有空时，项目规划就会变成令人沮丧的日历俄罗斯方块游戏。在固定周期内工作会显著简化这个问题。一个周期为我们提供了一个标准的项目大小，用于塑形和安排。

一些公司使用两周周期（也称为“冲刺”）。我们发现两周时间太短，无法完成有意义的工作。更糟糕的是，两周周期由于计划开销而非常昂贵。在两周内完成的工作量不值得围坐在桌旁进行“冲刺计划”的集体时间，也不值得因为需要重组而打破每个人的势头所产生的机会成本。

这使我们尝试了更长的周期。我们希望周期足够长，可以完成整个项目，从头到尾。同时，周期需要足够短，以便从一开始就可以看到结束。人们需要感受到截止日期的迫近，以便进行权衡。如果截止日期在开始时太遥远和抽象，团队自然会游离并且效率低下，直到截止日期开始变得更近并且变得真实。

经过多年的实验，我们最终确定了 **六周**。六周足够长，可以完成有意义的工作，同时又足够短，可以从一开始就看到结束。

冷却期

如果我们连续进行六周周期，就没有时间可以喘口气并考虑接下来要做什么。周期结束是最糟糕的计划时间，因为每个人都忙于完成项目并做最后一刻的决定，以便及时交付。

因此，在每个六周周期之后，我们安排了两周的 **冷却期**。在这段没有安排工作的时间里，我们可以休息，根据需要开会，并考虑下一步该做什么。

在冷却期，项目团队的程序员和设计师可以自由选择要做什么。在努力完成他们的六周项目后，他们享受掌控自己时间的乐趣。他们会用这段时间来修复错误，探索新想法，或者尝试新的技术可能性。

团队和项目规模

除了标准化我们周期的长度外，我们还粗略标准化了我们投注的项目和团队类型。

我们的项目团队由一个设计师和两个程序员或一个设计师和一个程序员组成。他们会在周期的后期加入一个质量保证人员，进行集成测试。

这些团队要么在整个周期内致力于一个项目，要么在周期内工作于多个较小的项目。我们称在整个周期内致力于一个项目的团队为 **大批量** 团队，而致力于一组较小项目的团队为 **小批量** 团队。小批量项目通常每个运行一到两周。小批量项目不是单独安排的。小批量团队需要自己想办法如何安排工作，以便在周期结束之前将它们全部交付。

现在我们有了一种标准的思考能力，我们可以讨论如何决定安排什么。

投注桌

投注桌 是在冷却期举行的会议，利益相关者在会议上决定下一个周期要做什么。要考虑的潜在投注是上一个六周中塑造的新提案，或者可能是一两个特别选择复苏的旧提案。正如我们在上一章中所说，没有“精简”或者积压待办事项需要组织。只有一些值得考虑的好选项。

在 Basecamp，我们的投注桌由 CEO（在我们公司的情况下是产品的最后决定权）、CTO、一名高级程序员和产品战略师（我自己）组成。

高管的时间只能以小片段获得，所以有一种“不浪费时间”的氛围，会议很少超过一两个小时。每个人都有机会提前自己研究这些提案。在会议之前的几周里，临时的一对一对话通常也会建立一些背景。一旦会议开始，就全神贯注于看看哪些选项进入了讨论桌，并做出决策。

会议的输出是一个周期计划。在场的每个人都知道谁有空，业务优先事项是什么，以及我们最近在做什么样的工作。所有这些都反映在关于做什么和安排谁的决策过程中（后文更多解释）。

公司最高层的人都在那里。没有“第二步”来验证计划或获得批准。也没有其他人可以在之后加入干扰或打断安排好的工作。

从最高层获得的认可对于使周期正常运转至关重要。会议很短，选项塑形良好，参与人数少。当这些标准得到满足，投注桌就成为了控制产品方向的地方，而不是资源争夺战或优先级的请求。有了足够长的周期来取得有意义的进展，以及可以真实交付的塑形工作，投注桌给予 C 级人员一种自早期以来就没有的“掌控方向”的感觉。

押注的意义

我们谈论“押注”而非规划，是因为它给人不同的期望。

首先，押注有回报。我们不只是填满时间框，然后完成任务。我们不只是花费两周时间来期待特性的渐进式进展。我们有意地将工作安排进六周的时间框，这样最终会有意义的成果。这个构想定义了具体的回报，使得押注具有价值。

其次，押注是承诺。如果我们押注了六周，那么我们承诺团队将全身心地专注于那件事，没有任何打扰。我们不是试图优化程序员每个小时的时间利用率。我们看重的是六周后整个产品的进展。

第三，明智的押注有风险的上限。如果我们对某事押注了六周，那么最多失去的就是六周。我们不会让自己陷入这样的境地：为了一件不值得的事情花费原本估计的数倍时间。

让我们更仔细地看看后两点。

不间断的时间

如果我们说要用六周时间，却允许团队被拉走做其他事，那其实就算不上是押注。

当你押注，你要信守承诺。我们不允许团队被打断或者被另行安排其他工作。如果有人打断团队进行请求，那就违背了我们的承诺。我们就无法给团队整整六周的时间来完成为期六周的工作。

当有人请求“只需几个小时”或“只需一天”的时候，不要被愚弄。动能和进展是二阶的东西，就像增长或者加速度一样。你无法用一个点来描述它们。你需要一个不间断的点的曲线。当你让某人为了修复一个 bug 或者帮助其他团队而离开一天的时候，你失去的不只是一天。你失去了他们所建立的动能，以及重新获得动能所需的时间。失去错误的小时可能会导致一整天的损失。失去一天可能会导致一周的损失。

如果在这六周内出现了什么问题怎么办？我们仍然不会打断团队并违背承诺。我们最多需要等待的时间就是六周，然后才能着手解决新的问题或者想法。如果这个周期结束了，而那件事仍然是最重要的事情，那我们可以继续押注这件事情的下一个周期。这就是为什么只押注一个周期是如此重要的原因。这样可以让我们保持对这些新问题做出反应的选择。当然，如果真的出现了危机，我们总是可以刹车。但真正的危机是非常罕见的。

断路器

我们将这种不间断的时间与一项严格但极为有效的政策相结合。团队必须在我们押注的时间内完成工作。如果他们没有完成，项目不会默认获得延期。我们故意制造了一个风险，即按照构想的那样，项目可能不会按时完成。听起来很严厉，但对于所有参与者来说都非常有帮助。

首先，它消除了项目失控的风险。我们在项目构想和提出的时候就确定了我们的“食欲”。如果项目只值得用六周时间，那么花费两倍、三倍甚至十倍的时间就是愚蠢的。非常少数项目是属于“不惜一切代价”类型，而且绝对必须立即完成。我们把它看作是确保一个项目不会使系统超负荷的“断路器”。一个需要太长时间的项目永远不会让我们陷入僵局或者妨碍更重要的新项目。

其次，如果一个项目在六周内没有完成，这意味着我们在构想上出了问题。与其在一个错误的方法上投入更多时间，断路器会推动我们重新构想问题。我们可以利用下一个六周的构想过程提出一个新的或更好的解决方案，避免我们在第一次尝试时掉进的任何兔子洞。然后我们会在押注的桌子上审查新的构想，看看它是否真正改

变了我们的成功几率，然后再决定是否再次用六周的时间投入其中。

最后，断路器激励团队更多地对项目负责。接下来的章节中我们会看到，团队被全权负责执行项目。这包括对实施细节进行权衡和选择何处进行范围裁剪。如果不能做出艰难的决定，就无法完成交付。严格的截止日期和未能交付的可能性激励团队定期质疑他们的设计和实施决策如何影响范围。

关于 bug 呢？

如果在六周的周期内团队没有被打扰，我们如何处理出现的 bug 呢？

首先，我们应该退一步，质疑我们对 bug 的假设。

BUG 并没有什么特别之处，使得它们自动比其他事情更重要。仅仅因为某事是一个 BUG，并不意味着我们有借口打断自己或其他人。所有的软件都会有 bug。问题是：它们有多严重？如果我们处于真正的危机——数据丢失、应用程序停滞不前，或者大量客户看到错误的东西——那么我们会立即处理。但是危机是罕见的。绝大多数 bug 可以等待六周甚至更久，许多甚至不需要修复。如果我们试图消除每一个 bug，那么我们永远做不完任何新的事情。如果你必须先修复整个世界，就无法交付任何新的东西。

话虽如此，没有人喜欢 bug。我们仍然希望有办法来处理它们。以下三种策略对我们很有效。

1. **使用冷却期。**问任何程序员是否有他们希望可以回去修复的事情，他们肯定会有一个清单给你。在周期之间的“冷却”期给了他们时间来做这件事。对于绝大多数 bug，等待六周并不算长时间，而每六周中的两周实际上累计起来可以用来进行大量的修复工作。
2. **将其带到押注桌上。**如果一个 bug 太大，无法在冷却期内修复，那它可以在押注桌上争取资源。假设后端进程正在拖慢应用程序，程序员想将其从同步步骤改为异步作业。程序员可以提出修复的理由，并在构想中描述解决方案。然后，押注桌上的人可以做出慎重的决定，而不是打断其他工作。时间应该被用来进行战略性地规划。推迟其他工作来修复一个 bug 和事先决定修复这个 bug 是两回事。
3. **安排一个 bug 消灭日。**一年一次——通常是在假期前后——我们会把整个周期用来修复 bug。我们称之为“bug 消灭日”。假期是一个好时机，因为当人们在旅行或休假的时候，很难完成一个正常的项目。团队可以自我组织来解决最重要的 bug，并解决前端或后端长期存在的问题。

保持清洁

管理能力的关键是每个周期都给自己一个干净的板子。这意味着一次只押注一个周期，并且在没有经过重新构想和考虑的情况下不会携带旧工作的残留。

将来最大化我们的选择是至关重要的。我们不知道接下来的六周会发生什么。我们不知道会出现什么精彩的主意或者紧急请求。

即使我们在周期之上有某种路线图，我们会在脑海中和在侧渠道讨论中保留它。每六周我们都会学到什么是有效的，什么是无效的，什么是重要的，什么是不重要的。保持选项的开放是没有负面影响的，而能够应对意外情况则有巨大的好处。

那些无法在一个周期内完成的项目怎么办？在这种情况下，我们仍然一次只押注六周的时间。假设我们设想了一个需要两个周期才能交付的功能。通过具体构想一个六周的目标，我们将风险大大降低，到时候会有完全建立和运作的东西。如果一切按计划进行，我们会很乐意按照我们在脑海中设想的方式再押注下一个六周的时间。但如果事情并非如此，我们可以定义一个完全不同的项目。或者我们可以暂停多周期的事情，去做一些紧急的事情。重要的是，我们始终要构想每个周期的最终状态，保持对改变方向的选择。

下注吧

观察你所处的位置

根据我们是改进现有产品还是开发新产品，我们会对六周周期中发生的事情设定不同的期望。

这促使我们反思我们在产品开发过程中的位置，并相应地进行下注。

现有产品

当我们为现有产品添加功能时，我们遵循标准的“塑造”流程：塑造工作、下注，并交给团队进行构建。我们期望团队在周期结束时完成并发布被塑造工作的某个版本。

对于现有产品，所有不会发生变化的现有代码和设计定义了新功能将适应的一种空间。塑造和构建就像为已经建好的房子制作家具一样。

新产品

新产品与现有产品不同。在现有产品上添加功能就像为固定尺寸的房间购买沙发，而新产品开发就像弄清楼墙和地基应该放在哪里，以便建筑能够屹立。

我们发现了从零开始构建新产品时的三个工作阶段。在每个阶段中，我们进行塑造的方式以及对团队在周期内如何合作的期望都是不同的。这些阶段会在多个周期内展开，但我们仍然一次只下注一个周期。

研发模式

在新产品的最早阶段，我们的想法只是一个理论或一线希望。我们不知道我们所想象的功能组合在现实中是否能够成型，关于如何在代码中对其进行建模的技术决策也更不明确。

这意味着有很多临时工作。我们可能会决定在建立功能的一半时发现这不是我们想要的，然后尝试另一种方法。

换句话说，我们无法可靠地提前塑造我们想要的东西并说：“这就是我们想要的。我们期望在六周后发布它。”我们必须通过构建来了解我们想要的是什么。

我们称这个阶段为 **研发模式**，并以三种方式进行调整。

1. 我们主要下注于研发新产品想法的一些关键部分的探索时间，而不是对一个良好塑造的提案下注。由于我们期望通过构建来学习，塑造变得模糊许多。
2. 我们的高级人员组成团队，而不是将任务委托给另一个构建团队。David（首席技术官）承担编程角色，并与 Jason（首席执行官兼设计师）或在 Jason 指导下的高级设计师一起工作。这是有两个原因的必要性。首先，当你自己不知道你想要什么时，你就不能委托给其他人。其次，架构决策将决定产品未来的可能性——它们定义了未来功能适应的“空白”。在这个阶段，团队需要保持对产品的愿景，并能够判断设计决策的长期影响。
3. 最后，我们不期望在研发周期结束后发布任何东西。我们的目标是进行探索，而不是发布。在最好的情况下，我们将有一些 UI 和代码提交，作为随后工作的基础。目标是学习什么是有效的，以便我们能够承诺一些承重结构：将定义产品形式的主要代码和 UI 决策。

我们不能仅凭一个研发周期的工作就将任何东西交付给客户。但我们仍然一次只承诺一个周期。我们可能会从第一个周期中学到，我们还没有准备好着手处理该产品。或者我们可能会发现我们的直觉是正确的，产品正在成型。根据情况，我们将逐个周期决定是否继续在研发模式下花费非正式时间。

生产模式

如果经过一些研发周期后我们继续变得更加明朗，最终会达到一个最重要的架构决策已经确定的时刻。产品正在完成那些定义它的几个基本功能，并且为在向客户发布之前必须完成的其他许多工作奠定了基础。

有了这个结构，高级团队可以引入其他人来做贡献。这是进入 **生产模式** 的转变，在这种模式下，我们将在明确的塑造、下注和构建阶段中进行正式周期的工作。生产模式就像在现有产品上工作一样：研发工作所设定的先例使新的贡献者能够确定新功能应该放在何处，以及它如何融入整体。

在生产模式中：

1. 塑造再次变得有意义。塑造的工作描述了我们期望在周期结束时看到的东西。
2. 构建项目的团队不再局限于高级团队。有可能同时对多个团队进行下注（如果有的话）并覆盖更多领域。
3. 发货是目标，而不是探索。但因为产品尚未对客户公开可用，我们对“发货”的定义有所不同。发货意味着合并到主代码库中，并且不期望再次对其进行更改。

由于我们没有在每个周期结束时向客户发货，我们保留了在发布之前从最终版本中删除功能的选项。这意味着我们仍然可以进行实验。我们可以将六周的时间投入到一个功能上，而不知道我们是否会希望将其纳入最终产品。只要我们向构建团队设定期望：我们无法预测我们在最终版本中需要什么，并且我们愿意冒险这个周期来尝试最好的想法。

清理模式

在发布新产品之前的最后阶段，我们抛弃了所有的结构。我们称这个阶段为 **清理模式**。这是一场自由的竞争。我们已经建立了足够多的新产品，以此得知总会有一些我们遗忘的事情，我们错过的事情，细节不对的地方，以及在研发模式和生产模式周期中悄悄爬入的错误。

将手指放在启动按钮附近会让你的头发竖立起来。一切突然变得“真实”。以前被我们忽略的事情现在以新的重要性出现在我们面前。

这就是为什么我们在最后留出一些容量来处理意外情况。在清理模式中：

1. 没有塑造。这个周期更接近于前一章提到的“缺陷修复”。领导层在整个周期中站在舵手的位置，关注重要的事情，并剔除分散注意力的事物。
2. 没有明确的团队边界。每个人都会尽力帮助。
3. 工作会持续“发货”（合并到主代码库）尽可能小的部分。

纪律仍然很重要。我们必须检查自己，以确保我们正在处理的是必须做的事情，而不仅仅是我们心虚要求我们延迟发布。清理阶段不应该持续超过两个周期。

清理也是领导层做出“最终决定”的阶段。V1 上的较小覆盖面意味着要回答的问题更少，支持的内容更少，我们要承诺长期支持的内容也更少。有时我们需要看到所有功能作为一个整体运作，以判断我们可以舍弃什么，以及在将其交付给客户之前可能需要更深入考

虑什么。

例子

点阵日历

我们为现有产品 Basecamp 构建了点阵日历（参见[第2章](#)）。我们塑造了这个项目，押注了六周的时间，一个团队构建了它，然后我们立即将其交付给了客户。

新产品：HEY

2020 年，经过两年的开发，我们推出了一款名为[HEY](#)的新邮件应用和服务。HEY 在开发的第一年处于“研发模式”。由 Jason（CEO）、David（CTO）和 Jonas（高级设计师）组成的团队在最终确定核心之前探索了各种想法。随后进行了近一年的“生产模式”周期，Basecamp 的所有团队完善了 HEY 的功能集。我们最终进行了两轮“清理”，大幅削减了功能集，于 2020 年 7 月推出了产品。

要准确地说，这两种模式之间在第一年之后存在一些重叠。Basecamp 作为一家公司足够大，高级团队可以在“生产模式”下塑造和委派项目，同时在“研发模式”下继续探索新领域。

在 HEY 上的每次押注都是逐个进行的。在最初的几轮研发中，押注者们并不知道他们将在这两年内一直致力于 HEY。逐渐地，他们对这个想法变得自信，并对他们愿意在 HEY 上花费多少个周期有了整体的认识。但他们并没有对这些周期中会涉及什么做出具体

的承诺。而将注意力转回 Basecamp，我们的现有产品，始终是一个备选方案。

实验性功能：Hill Charts

第三个例子展示了一些灰色地带。当我们在 Basecamp 中构建 Hill Charts 功能（参见第 13 章）时，我们不知道它是否会成功。

Basecamp 是一个现有产品，我们觉得押注将这个实验性功能发布给客户的风险太大。因此，我们更像是在一个新产品上进行生产模式的押注。我们塑造了一个刚刚足够功能的第一个版本。我们没有指望在没有进行额外周期的情况下将其交付给客户。这是一种风险：我们押注了一个周期，而不是两个周期。如果它不起作用，我们会放弃它。如果出现更重要的事情，我们可能永远不会安排第二个周期。但在第一个周期之后，我们感到很有信心。我们塑造了一个项目来完善它，决定再押注一个周期，然后将其交付给客户。

需要询问的问题

以下是在押注讨论中可能听到的一些常见问题。

问题是否重要？

就像在提案撰写中一样，我们始终小心地区分问题和解决方案。如果问题不值得解决，解决方案就无关紧要。

当然，任何影响客户的问题都很重要。但我们必须做出选择，因为总会有更多的问题，而时间却总是不够。因此，我们需要权衡这些问题。这个问题现在是否比*那*个问题更重要？

在讨论问题时，与会人员对问题的判断取决于他们的角度、角色和知识。例如，一个问题可能只影响小部分客户，但却给支持带来了不成比例的负担。根据你接触支持的程度以及你关注业务的哪个方面，你可能会对这个问题有不同的权衡。

有时，一个过于复杂或广泛的解决方案可能会引发对问题的质疑。我们是否真的需要在整个应用程序中做这么多改动？我们是否已经足够具体地理解了这个问题？也许有办法缩小范围，以便从变动的 20% 中获得 80% 的利益。

期望是否合理？

当我们将解决方案塑造到一个合理的时间框架时，比如两周或六周，这是件好事。但我们可能仍会就是否值得花费时间进行争论。假设某个利益相关者表示对在某个提案上花费六周的时间不感兴趣。谈判可以从以下几个方向进行：

1. 也许问题没有表述清楚，决策者可以在这个时候加入对话，以改变意见。例如，“是的，这种情况并不经常发生，但当发生时，人们会对此发表评论，这会影响他们对我们的看法。”或者“也许听起来微不足道，但支持人员必须经过 11 个耗时的步骤才能解决问题。”
2. 有时，对时间承诺的拒绝实际上是对其他事情的拒绝。也许他们不喜欢解决方案或技术实现的某些方面。询问“如果我们能在两周内完成，你会怎么想？”可以揭示出问题并不在于时间。CTO 可能会回答：“我不想在应用程序的那个区域引入另一个依赖。”
3. 如果兴趣不够，决策者可能会放弃这个想法。

4. 如果他们相信问题很有吸引力，但他们没有足够的准备来提出问题，决策者可能会回到绘图台，要么制定一个较小版本的方案（以满足更短的期望），要么进行更多的研究。

解决方案是否有吸引力？

问题可能很重要，期望也是合理的，但对解决方案可能存在分歧。

例如，在屏幕上添加界面元素会带来一个看不见的成本：失去了实际空间。在主页的角落放一个按钮可能完美解决问题。但那个空间是有价值的。如果我们现在放弃它，将来就无法使用它。我们是为了解决这个特定问题而卖得太便宜了？

如果有人提出了一个即时的设计解决方案，比如“我们将那个按钮移到一个操作菜单中”，我们可能会进行讨论。但通常情况下，我们不会在押注讨论中过多地进行设计工作或讨论技术解决方案。如果我们发现自己在细节中花费了太多时间，我们会提醒自己“好吧，我们不在这里进行设计”，并回到高层次。

现在是否合适？

我们下一个想要做的项目的类型可能取决于我们最近做过的项目。也许自从我们最后一次推出新功能以来已经过了太长时间。或者也许我们一直在构建太多新功能，感觉是时候解决一些长期存在的客户需求了。或者如果团队在过去的几个周期都在应用程序的同一个区域工作，如果我们计划再做一个同类型的项目，他们的士气可能会下降。

这些都是我们可能会放弃一个项目的理由，即使它形状良好且有价值。项目很棒；只是现在不是合适的时机。

是否有合适的人员可用？

作为押注流程的一部分，我们选择具体的人员在每个团队中扮演哪个角色。也就是说，我们将一个特定的小团队的设计师和一两名程序员与一个项目进行配对。我们有一个由设计师和程序员组成的“核心产品”团队，我们在为每个周期计划团队时从其中选择。团队将在整个周期内与对方合作，然后下一个周期可以是不同的人组合。

不同的项目需要不同的专业知识。也许这个项目需要更多的前端编程。或者另一个项目会引发很多的范围蔓延，因此我们需要一个擅长“范围之锤”的人。

每个人所做的工作类型也是另一个因素。已经连续做了一系列“小批量”项目的人可能更愿意接受一个“大批量”项目，反之亦然。

最后，人员的可用性也总是需要进行一点日历游戏。假期或休假会影响我们在接下来的周期中能够安排哪些项目。

我们看到一些其他公司使用了不同的模式，他们不是将项目分配给人员，而是让团队成员选择他们想要参与的项目。在文化上，我们对会议过于排斥，不愿意增加这一步。但我们听说对一些团队来说，这种方式可以很好地运作，因为项目团队会更有参与感。

发布启动消息

在把押注做出后，押注讨论中的某个人将撰写一条消息，告诉每个人我们在下一个周期中押注了哪些项目，以及谁将参与其中的工作。



Jason 通过[Basecamp](#)消息宣布了下一个周期的押注

移交责任



我们已经下注了，现在是时候启动下一个周期了。团队如何开始？

分配项目，而非任务

我们不是从分配任务给任何人开始。没有人扮演“任务大师”或“架构师”的角色，将项目分割成其他人执行的部分。

一开始就将项目分解为任务，就像将提案通过纸碎机处理一样。每个人只得到了零散的部分。我们希望整个项目在整个过程中保持“完整”，这样我们永远不会失去对更大脉络的把握。

相反，我们相信团队能够承担整个项目，并在提案的范围内工作。团队将自己定义任务，并制定自己的工作方法。他们将拥有完全的自主权，并凭借自己的判断力尽力执行提案。

团队喜欢被赋予更多自由来实现他们认为最好的想法。有才华的人不喜欢被当作“代码猴”或“任务接收者”。

当团队被分配个别任务时，每个人都可以执行自己的小部分，而不必为判断所有部分如何组合而负责。提前规划会使你在途中对实际情况视而不见。

记住：我们并不是给予团队绝对的自由从零开始发明解决方案。我们已经进行了塑造。我们已经设定了边界。现在我们要相信团队来填补提案轮廓，做出真实的设计决策和实现。

在合适的抽象层次上定义项目的努力将会得到回报。凭借他们的才华和对细节的了解，团队将会得出一个比我们试图提前确定最终形式更好的成品。

完成意味着部署

周期结束时，团队将部署他们的工作。对于小批量团队在一个周期内有几个小项目的情况，只要在周期结束前完成，他们可以自行部署每个项目。

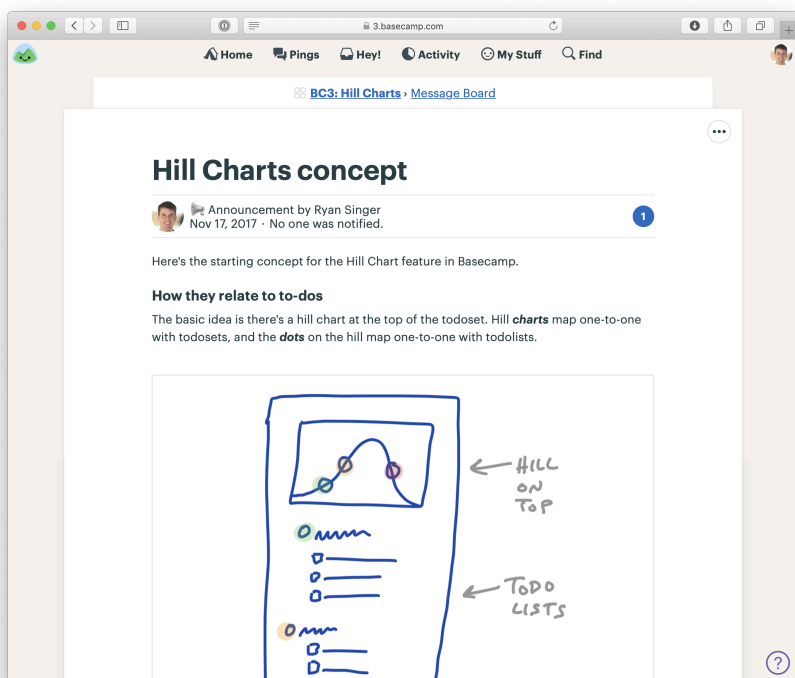
这个限制使我们忠实于我们的赌注，并尊重 **中断器**。项目需要在我们预算的时间内完成；否则，我们的期望和预算都毫无意义。

这也意味着任何测试和质量保证需要在周期内进行。团队将通过划定项目的最基本方面，尽早完成它们，并与质量保证进行协调来适应这一点。（稍后再详细讨论。）

对于大多数项目，我们并不严格要求帮助文档、营销更新或向客户公告的时间安排在周期内完成。这些从风险角度来看是薄尾的（它们永远不会花费我们认为的 5 倍时间），而且大部分由其他团队处理。我们经常会在周期结束后进行这些更新，并发布关于新功能的公告。

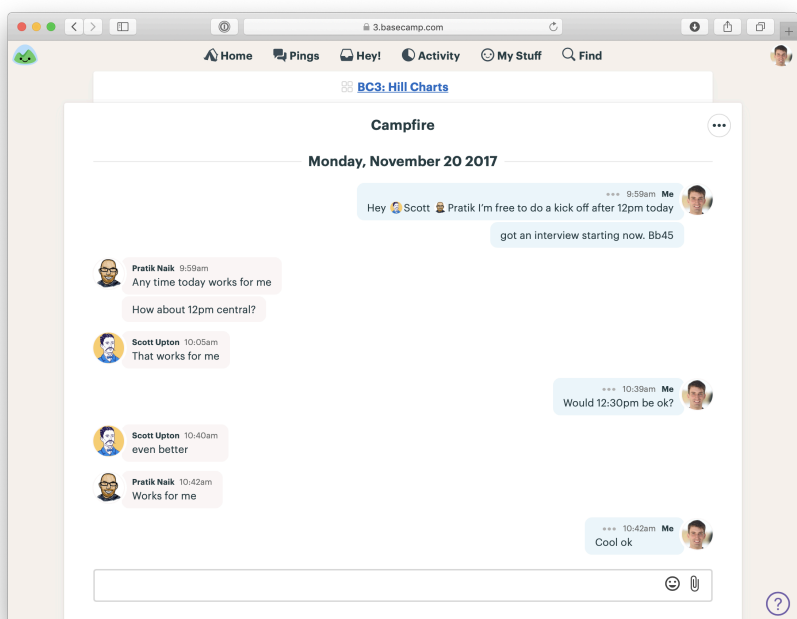
启动

我们通过创建一个新的 [Basecamp](#) 项目并将团队添加进去来启动项目。然后我们将首先在消息板上发布塑造的概念。我们要么发布原始提案，要么发布其简化版本。



[Basecamp](#) 项目上的第一件事是一个包含塑造概念的消息

由于我们的团队是远程的，我们使用 Basecamp 项目中的聊天室安排一个启动电话。



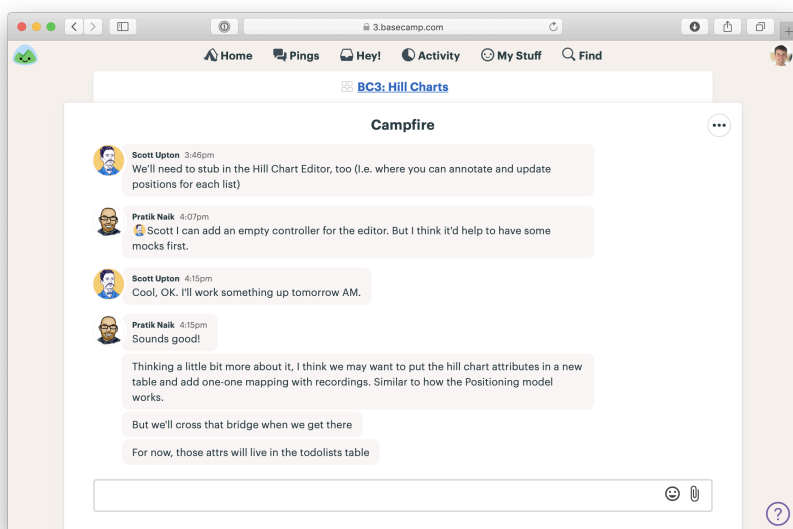
与团队安排一个会议，以了解塑造的工作

这个电话给了团队一个机会，让他们提出任何在写作中不清楚的重要问题。然后，对于项目的大致理解，他们准备好开始工作了。

取向

最初几天的工作看起来不像“工作”。没有人在勾选任务。没有东西被部署。也没有交付物可供查看。通常在团队最初的几天内甚至团队之间的沟通都很少。有时会有一种奇怪的沉默。

为什么？因为每个人都在埋头研究现有系统的运行方式和最佳的起点。每个人都在忙于了解情况和取向。



团队找出从哪里开始

对于经理来说，尊重这个阶段很重要。团队不能只是跳进代码库，立即开始构建新功能。他们必须熟悉相关代码，思考提案，并且走一些短暂的死胡同来找到一个起点。干涉或者太早询问项目状态会伤害项目。这会剥夺团队找到最佳方法所需的时间。探索工作必须发生。要求可见的进展只会使其隐藏起来。赋予团队权力明确地说“我还在摸索如何开始”，这样他们就不必隐藏或伪装这种合法的工作。

一般来说，如果沉默在三天后没有开始打破，那是一个合理的时间来介入并了解发生了什么。

想象的 vs 发现的任务

由于团队得到的是项目而不是任务，他们需要自己想出任务。在这里，我们注意到在项目开始时我们 认为 需要做的任务和在做真正的工作过程中我们 发现 需要做的任务之间的重要区别。

团队自然而然地从一些想象的任务开始——他们认为只需考虑问题就必须做的那些任务。然后，在实际工作中，随着他们动手，他们发现了许多我们事先不知道的其他事情。这些意外的细节构成了项目的真正主体，并有时提出了最艰难的挑战。

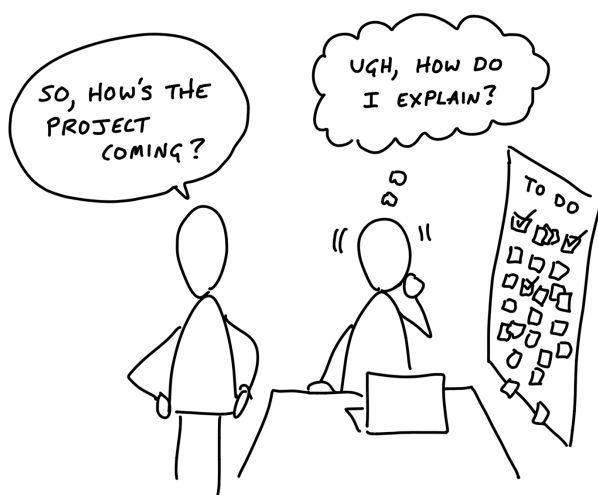
团队通过做真正的工作发现任务。例如，设计师在桌面界面上添加了一个新按钮，然后注意到在移动网页版本上没有明显的位置。他们记录了一个新任务：想办法在移动设备上显示按钮。或者设计的第一遍视觉层次很好，但设计师意识到需要在打乱布局的地方添加更多解释性文本。两个新任务：更改布局以容纳解释性文本；编写解释性文本。

通常一个任务会在做某事的过程中出现。假设程序员正在进行数据库迁移。在查看模型以了解关联时，她可能会遇到需要稍后更新的一个方法。她会想要记录一个任务，以便稍后更新该方法。

要真正弄清楚需要做什么，就要开始真正做工作。 这并不意味着团队开始构建任何东西。他们需要选择一个有意义的东西来首先构建。项目的核心部分，同时又足够小，可以在几天内完成端对端的工作 UI 和工作代码。

在接下来的章节中，我们将看看团队如何选择目标，并共同努力使完全集成的试验工作起来。

完成一个部分



团队开始熟悉情况后，他们开始发现并跟踪构建项目所需的任务。在这个早期阶段，重要的是他们不要创建在最后时刻应该汇聚在一起的各个部分的总体计划。如果团队完成了很多任务，但没有“一件东西”可以点开并尝试，就很难感受到进展。团队可以做很多工作，但由于他们还没有任何实质性的成果，所以会感到不安全。很多事情都做完了，但没有真正做完。

相反，他们应该早期就努力做出可触摸且可以演示的东西——在第一周左右。这需要在项目的一个小部分上进行垂直整合，而不是在水平层面上逐步推进。

整合一个片段

我们可以将项目分为前端和后端，设计和代码两个层面。尽管从技术上讲，层面比这更多，但在大多数项目中，这两个是主要的整合挑战。

假设项目以大量设计开始。团队可以设计各种屏幕，甚至将其实现为模板或视图。但在它们连接到后端之前，什么都不会发生。工作仍然是假设性和推测性的。

同样，后端也是一样。可以勾选很多任务，但没有任何用户界面，你能用它做什么呢？在没有与之交互的情况下，你如何判断特定业务逻辑的工作是否真的正确呢？

我们想要的是挑选出项目的一个片段进行整合。然后当完成时，团队就有了一些切实可行的东西（或者不起作用并重新考虑）。任何人都可以通过交互进行点击，并查看该功能是否能够如预期般运作，以及它所做的是否是他们想要的。

案例研究：项目中的客户

我们在 Basecamp 3 中构建了一个功能，允许服务公司邀请客户参与他们的项目，并与他们分享选择的文档、消息或待办事项。在提案中定义的概念有各种移动部分：

- 客户访问：在添加此功能之前，Basecamp 的访问模型是全有或全无。我们需要一种方式邀请一些人只查看项目的某些部分。这对后端和缓存有重大影响。
- 客户管理：我们需要一种方式将客户添加到项目中，并能够单独管理客户与团队成员。

- 可见性切换：项目中的每个内容都应该有一个切换按钮，用于向客户公开或隐藏它。

团队中有一个设计师和一个程序员。在他们熟悉现有代码的工作方式后，设计师选择了可见性切换作为首要整合的最佳位置。这是项目中最核心的 UI 元素。这是会出现在演示视频中的元素，并且是客户最常使用的交互。

设计师并没有制作像素级的完美模型。相反，他在应用的 HTML 模板中尝试了不同的使用方式和位置。切换按钮应该是两个单选按钮、一个复选框，还是一个更改状态的自定义按钮？

与此同时，程序员并没有等待。他已经从提案中得到足够的指导，开始解决访问模型的问题。

当设计师对 UI 的基本方向感到自信时，他联系了程序员，并展示了他所设计的初始切换按钮。在一段时间内放下访问问题后，程序员使切换按钮足够工作，以便在所有支持的内容类型上显示，并在点击时更改状态，并将其状态保存在数据库中。

在这一点上，切换按钮实际上并没有改变内容的可见性。但对于服务公司来说它是有效的。设计师可以点击它，感受它，并判断它在暂存服务器上使用实际数据时的效果。

切换按钮仍然需要更多的设计工作。但程序员不再需要参与其中。有了基本的按钮，设计师可以继续尝试文字、位置、颜色、移动端视图渲染等。同时，程序员可以回到访问模型或其他最重要的问题上。

项目开始后的三天左右，设计师向经理演示了可工作的切换按钮。他们的讨论导致了一些微调，然后他们就能够称这个切换按钮为“完成”。项目的一个重要部分被设计、实现、演示并解决。团队展示了切实的进展，团队和管理层通过看到一个工作的部分而对项目感到自信。通过早期的核心交互点击，他们能够验证他们希望在理论上有意义的东西在实践中确实看起来正确并且合理。

这个简短的例子说明了团队如何在短时间内整合，一次完成项目的一部分。

程序员不需要等待

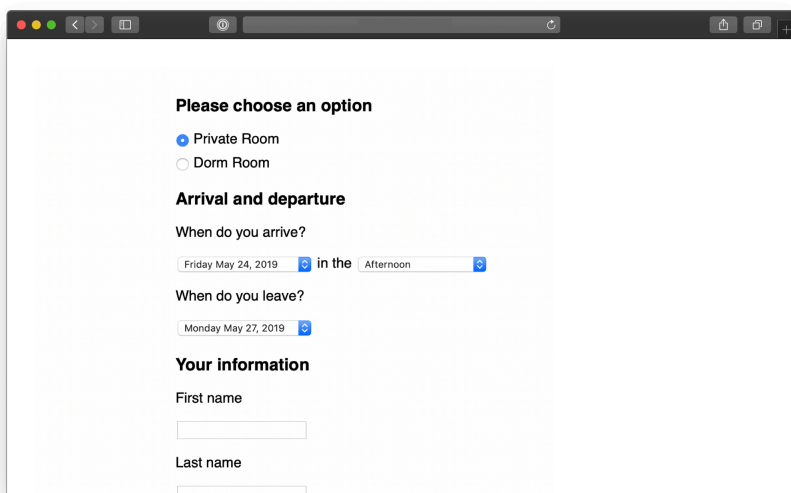
因为在塑造过程中已经定义了重要的移动部分，所以当项目开始时，程序员不需要空等设计。提案中已经有足够的指导，可以让他们从一开始就开始解决后端问题。他们可能无法在不知道前端方向的情况下完成某个功能，但提案中应该有足够的信息来指导基础建模。

像素完美屏幕之前的可承受能力

程序员不需要像素完美的设计来开始实施。他们所需要的只是端点：输入元素、按钮、存储数据应该出现的位置。这些可承受能力是用户界面设计的核心。

对于字体、颜色、间距和布局的问题可以在原始可承受能力就位并在代码中连接后解决。复制写作、基本的可承受能力和一些布线是我们尝试在浏览器或设备上实现实时工作版本所需要的全部。然后我们可以尽早回答基本问题：它是否有意义？它是否容易理解？它是否符合我们的期望？

这意味着设计师给程序员的第一个界面可能看起来非常基础，就像下面的例子一样。它更像是一个面包板，而不是一个视觉设计或精心制作的模型。



Please choose an option

☒ Private Room
☐ Dorm Room

Arrival and departure

When do you arrive?
Friday May 24, 2019 in the Afternoon

When do you leave?
Monday May 27, 2019

Your information

First name

Last name

这个屏幕截图来自一个多日课程的注册应用程序。设计师用 HTML 手工制作。几乎没有样式，只有足够的视觉层次结构，以确保布局可用且适合未来的样式层。

虽然设计看起来简单，但其中反映了许多决定。

- 要求到达时间但不要求离开时间的决定来自于对业务逻辑和定价模型的详细讨论。
- 到达时间下拉选项中的特定选项对应于必须解决的关于何时收费用餐和过夜住宿的规则。

- 设计师的第一张草图使用了日历风格的日期选择器来选择到达和离开日期。但这导致了用户体验问题。某些课程时间很长（多周），有不同的阶段。在标准的日历风格日期选择器中无法在日期框上标记阶段。通过下拉菜单，她可以使用选项组来标记需要时期的日期组。这样用户就不需要在其他地方参考时间表，以确保他们选择了正确的日期。

以下是另一个例子。这是一个从客户访谈中获取数据的应用程序的第一个工作部分。

Projects > Basecamp > Jan

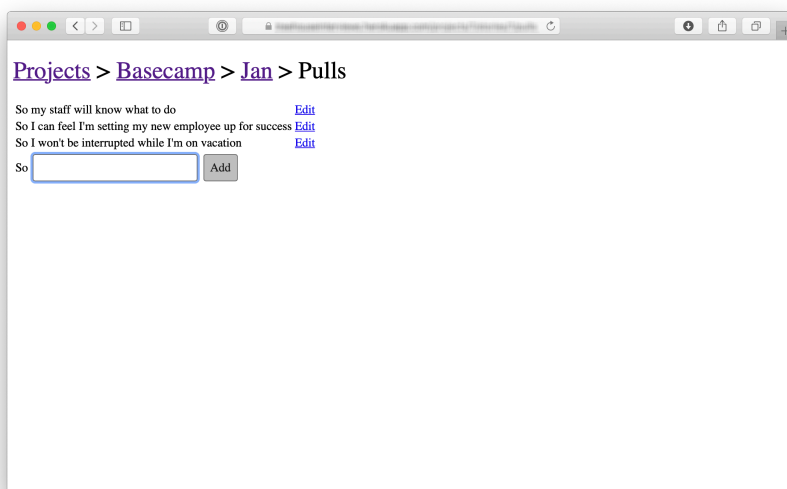
Name: Story #: A: B: [Edit story definition](#)

Setup	Pushes	Pulls	Habits	Anxieties	Observations
	• When I just hired another person • When I'm going to be on vacation • When my staff has been losing track of information	• So my staff will know what to do • So I can feel I'm setting my new employee up for success • So I won't be interrupted while I'm on vacation			
Edit Setup	Add/edit Pushes	Add/edit Pulls	Add/edit Habits	Add/edit Anxieties	Edit Observations

在早期阶段，项目名称（Basecamp）和访谈对象（Jan）都是硬编码的，大部分链接都没有任何作用。

看看这个设计有多原始。操作只是默认的蓝色和紫色浏览器颜色的纯文本链接。包含数据点的框只有简单的黑色边框。尽管它很粗糙，但这个设计测试了一些重要的权衡。设计师选择尽可能多地显示数据以便于查看访谈。这样一来，每个部分内部就没有足够的空

间来添加、编辑或删除数据点的 UI。这导致设计师为每个部分创建了单独的屏幕来添加和编辑数据。



这是为添加和编辑“拉取”而设计的第一个设计——这个访谈技术中的一种数据类型。再看看它有多朴素。这里有足够的设计，可以快速地将其连接起来并测试。团队可以通过点击来判断是否接受在单独的屏幕上记录数据。如果有效，他们可以稍后添加额外的样式。如果不起作用，他们不会浪费大量时间来实施像素完美的设计。

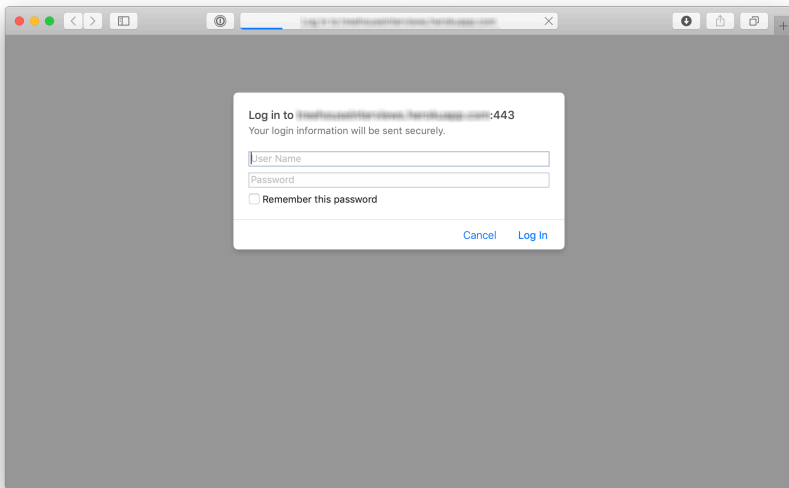
美丽的对齐、颜色和字体在第一次尝试中并不重要。视觉样式在最终产品中很重要，而在早期阶段并不重要。最大的不确定性在于它是否会起作用，它是否有意义，以及实施的难度。元素连线后，它们可以重新排列、重新设计和重新绘制，以改进已经完成的工作。先让它起作用，然后再让它变美观。

为下一步编写足够的程序

对于后端工作也是如此。不必要全盘接受或拒绝。有时设计师只需要一些脚手架——几个保存数据的字段或一些代码来从一个虚拟屏幕导航到另一个。其他时候，她需要在模板中填充一个真实数据集的变量，这样她可以迭代不同的显示方式（行、列、媒体框等），找到最佳设计。

早期的后端工作可以是有策略的零散的。可能会有一个控制器来渲染模板，但没有模型。或者有一个控制器和一些带有模拟数据的模型的部分，但没有支持创建或更新数据。尚未连接的屏幕至少可以通过路由相互连接。

当团队开始测试访谈应用程序的第一个部分时，他们知道其中会包含来自真实访谈的敏感数据。他们需要用某种身份验证保护它。他们只是使用了普通的 HTTPAuth 来硬编码一个密码，而不是构建完整的用户名和密码支持——甚至不是集成第三方解决方案。



这使得团队可以在周期的早期尝试从真实访谈中添加数据，而不必为了连接一些不会教会他们任何关于他们试图解决的问题的身份验证代码而放慢速度。

重点是在产品的同一部分上创建设计和编程之间的来回。而不是一次性交付，轮流逐步添加可承受能力、代码和视觉样式。一步一步，点击通过正在进行中的真实工作功能来判断它的进展情况以及接下来要做什么。

从中间开始

在上面的例子中，团队没有首先构建登录。他们没有在解决添加访谈数据的问题之前构建创建访谈项目和访谈对象的方法。他们直接跳到了有趣的问题所在，并且为了达到目的，对其他事情进行了删节。

为此，以下是选择首先构建的内容时要考虑的三个标准：

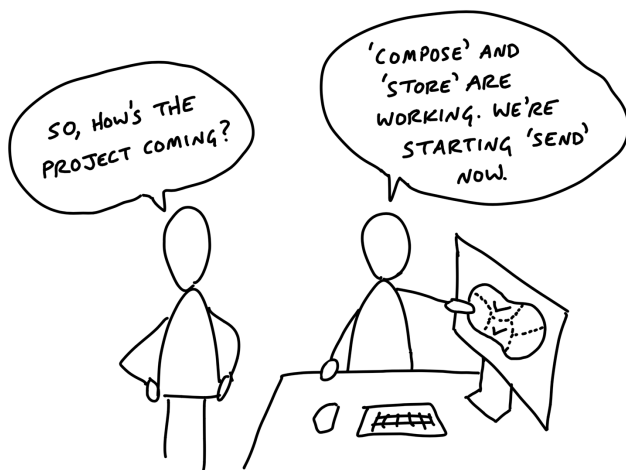
首先，它应该是**核心**。可见性切换对于项目中的客户概念是核心的。如果没有它，其他工作就毫无意义。与项目的更边缘的方面相比，比如更改客户的能力，这两者都是“必需的”，但一个更为核心和重要，需要在周期的早期证明出来。在访谈应用程序中，记录访谈数据比设置新的研究项目更核心——更在中心位置。

其次，它应该是**小**。如果第一个工作部分不够小，那么将其从其余部分中分离出来的好处就不大。关键是要在几天内完成一些有意义的工作，建立动力，有一些真实的点击内容，表明团队正在正确的轨道上。

第三，它应该是**新颖**的。如果项目的两个部分都是核心和小的，那么更倾向于选择你以前从未做过的事情。在项目中的客户功能中，添加客户的 UI 基本上和添加普通用户的 UI 大致相同。开始这项工作会推动项目前进，但它并不会教给团队任何东西。它不会消除

不确定性。从可见性切换开始，能够提高所有人的信心，因为它证明一个新的想法将会行得通。

绘制范围



在上一章中，我们通过提前完成一个整合切片来启动了项目。这种做法属于团队可以在整个项目中使用的更一般的技术。

按结构组织，而不是按人员

当要求为项目组织任务时，人们常常会按人员或角色进行分工：他们会为设计人员创建一个列表，为程序员创建一个列表。这会导致我们在上一章中讨论的问题——人们会完成任务，但任务不会很早期地汇总成项目的一个完成部分。

以软件之外的例子来说明，考虑有人组织一个筹款活动。他们可以为他们的三个志愿者中的每个人创建一个任务列表，并以此跟踪工作。但是这样就无法看到活动进展的整体情况——宏观上哪些已完成，哪些尚未完成。相反，他们应该根据项目的结构创建列表——可以独立完成并独立于彼此的事情。为此，他们将创建食品菜单、场地设置和灯光/声音等列表。然后组织者就可以轻松地看到哪些区域已完成，哪些区域还有待工作。

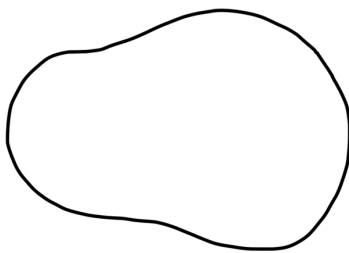
在产品开发中，这些类别并不是预先为我们准备好的。我们通常会构建以前从未构建过的东西。每个项目都是我们必须步行穿过的一片未知领地，然后才能绘制地图。通过深入工作，我们找出了相互依赖的地方、事物的联系以及我们可以分割的内容。

正如我们在上一章中看到的，工作的切片集成了前端和后端任务。这使我们能够完成实际项目的一个切片并明确地继续进行下去。这比在周期结束时有许多——指望着——应该能够组合在一起的碎片要好。

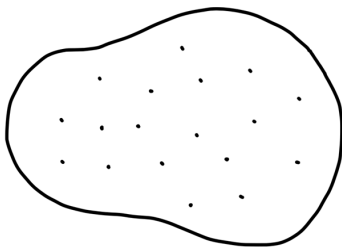
我们将这些集成的项目切片称为 **范围**。我们将项目的整体范围（单数）分解为可以独立完成的多个范围（复数）。在本章中，我们将看到团队如何将项目映射到范围并逐个应对它们。

范围图

想象一下项目的俯视图。一开始，只有来自前期塑造工作的大纲。还没有任务或范围。

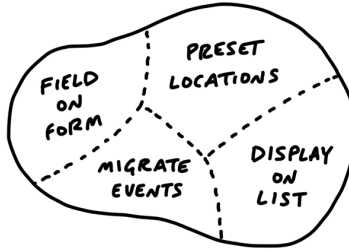


当团队成员接手项目时，他们开始发现任务。任务是一个自然的起点，因为它们的具体和细粒度的。现在将它们组织成更高级别的分类还为时过早。试图人为地将它们分组是不自然的。一开始捕捉需要发生的各种事情就足够了。



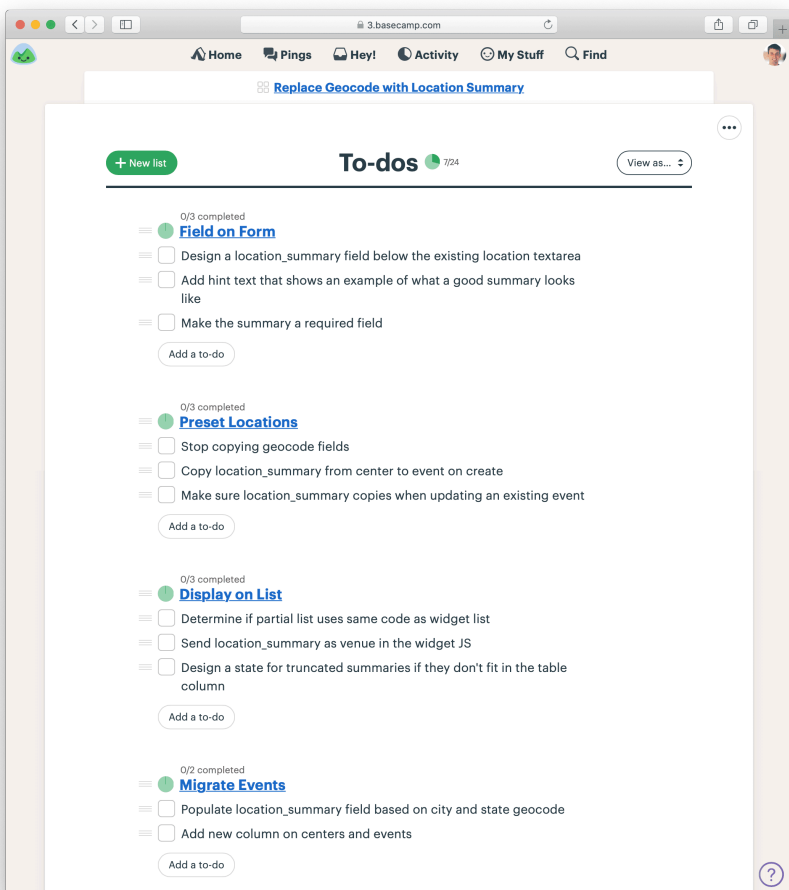
但我们不希望长时间停留在这个画面中。它的层次太低了。高空视角什么也看不到。

随着团队开始在项目上进行真正的工作，他们了解了任务之间的关系以及项目的真实结构。然后他们可以将项目分解为范围。这就像将项目的地图划分为独立的领地。



范围反映了问题的有意义的部分，可以在较短的时间内——几天或更短——独立完成。它们比任务要大得多，但远比整体项目要小得多。

地图是一种心理形象。在实践中，我们将范围定义并跟踪为待办事项列表。每个范围对应一个列表名称。然后该范围的任何任务都放入该列表。



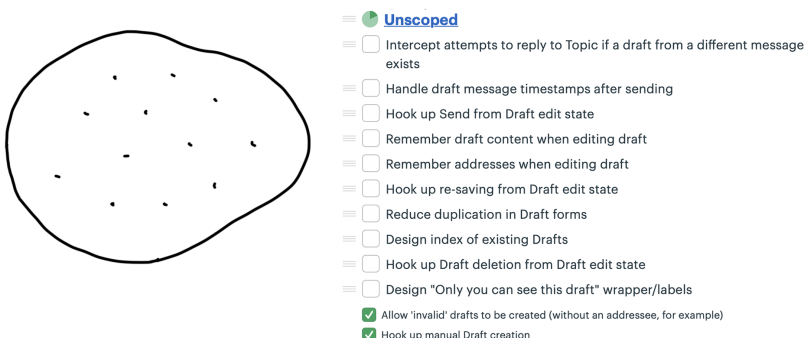
项目的语言

范围不仅仅是切片。它们成为了项目在宏观水平上的语言。当我们构建“项目中的客户”功能时，团队使用了这样的范围语言：“*Bucket Access*完成后，我们可以实现邀请客户。然后当公司内的人翻转可见性切换时，我们将更新录像可见性。”

在汇报状态时，团队使用范围的语言来解释哪些已完成，哪些尚未完成。与其深入到琐事中并辩解各个未完成任务的目的和状态，以宏观水平展示已完成的软件部分会更令人满意。（在下一章中，我们将了解如何使用山图表报告范围的更多信息。）

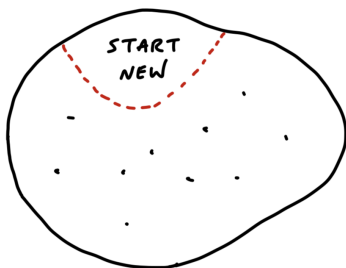
案例研究：消息草稿

一位设计师和一位程序员正在开发一个新应用中创建和保存消息草稿的功能。在启动后，他们确定了一堆任务，他们需要随时完成。



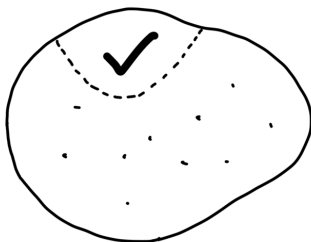
在第一周接近结束时，他们完成了一些任务，但他们没有可以展示他们工作成果的东西。秉持“完成一部分”精神，他们专注于一个关键互动，即创建一个新的草稿。

他们将新的范围命名为“开始新的”，为其创建了一个待办事项列表，并将待办事项移到其中。现在只剩下一个设计任务让他们考虑这个范围是否完成。



- **Start New**
- ☐ Design "Only you can see this draft" wrapper/labels
- ☒ Allow 'invalid' drafts to be created (without an addressee, for example)
- ☒ Hook up manual Draft creation
- **Unscoped**
- ☐ Intercept attempts to reply to Topic if a draft from a different message exists
- ☐ Handle draft message timestamps after sending
- ☐ Hook up Send from Draft edit state
- ☐ Remember draft content when editing draft
- ☐ Remember addresses when editing draft
- ☐ Hook up re-saving from Draft edit state
- ☐ Reduce duplication in Draft forms
- ☐ Design index of existing Drafts
- ☐ Hook up Draft deletion from Draft edit state

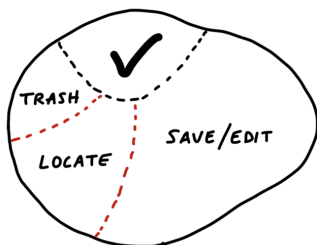
完成了一个设计任务后，范围就完成了。



- **Unscoped**
- ☐ Intercept attempts to reply to Topic if a draft from a different message exists
- ☐ Handle draft message timestamps after sending
- ☐ Hook up Send from Draft edit state
- ☐ Remember draft content when editing draft
- ☐ Remember addresses when editing draft
- ☐ Hook up re-saving from Draft edit state
- ☐ Reduce duplication in Draft forms
- ☐ Design index of existing Drafts
- ☐ Hook up Draft deletion from Draft edit state

剩下的未划定范围中的任务并不代表所有尚未完成的工作。当他们开始处理每个任务时，会发现更多的任务。尽管如此，在工作的多样性足以挖掘出更多的范围。团队在这个时候已经有动力提前拆分范围，因为他们知道他们希望他们的努力很快就能汇总成另一个可见的完成部分。

看看剩下的任务，他们决定将与查找草稿相关的任务分成一个新的范围，称为“定位”，将删除任务分成一个名为“废纸篓”的范围。剩下的工作似乎都与保存和编辑草稿有关，所以他们将其称为“保存/编辑”。

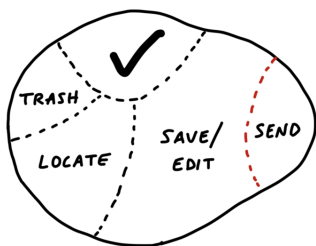


- **Save/Edit**
 - Intercept attempts to reply to Topic if a draft from a different message exists
 - Handle draft message timestamps after sending
 - Hook up Send from Draft edit state
 - Remember draft content when editing draft
 - Remember addresses when editing draft
 - Hook up re-saving from Draft edit state
 - Reduce duplication in Draft forms
- **Trash**
 - Hook up Draft deletion from Draft edit state
- **Locate**
 - Design index of existing Drafts

看看“定位”范围。目前只有一个任务。但肯定会有更多的工作要做，而不仅仅是设计索引。当需要执行任务时，它们将放在这里。

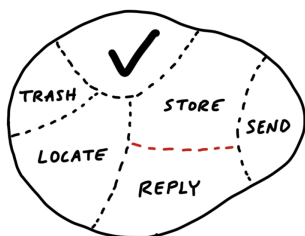
设计师开始了一些定位工作，而程序员则专注于保存/编辑。当她深入研究时，她注意到她可以切割出一些部分以取得更多的可见进展。事实上，它包含了三个范围。

首先，她将与发送草稿消息相关的工作分出来。她称之为“发送”。



- **Send**
 - Hook up Send from Draft edit state
 - Handle draft message timestamps after sending
- **Save/Edit**
 - Intercept attempts to reply to Topic if a draft from a different message exists
 - Remember draft content when editing draft
 - Remember addresses when editing draft
 - Hook up re-saving from Draft edit state
 - Reduce duplication in Draft forms
- **Trash**
 - Hook up Draft deletion from Draft edit state
 - Design a way to trash drafts from the index of drafts
- **Locate**
 - Design index of existing Drafts
 - Design a way to navigate to Drafts via "Inbox..." menu

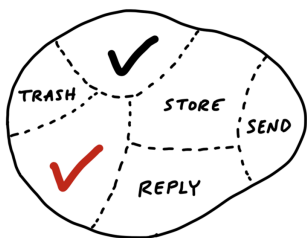
最后，剩下的一些保存/编辑任务是关于存储信息的，而另一个与之实际上无关——它是处理回复另一条消息时的特殊情况。她将这些任务分成两个新的范围：存储和回复。



- **Send**
 - ☐ Hook up Send from Draft edit state
 - ☐ Handle draft message timestamps after sending
- **Store**
 - ☐ Remember draft content when editing draft
 - ☐ Remember addresses when editing draft
 - ☐ Hook up re-saving from Draft edit state
- **Reply**
 - ☐ Intercept attempts to reply to Topic if a draft from a different message exists
- **Trash**
 - ☐ Hook up Draft deletion from Draft edit state
 - ☐ Design a way to trash drafts from the index of drafts
- **Locate**
 - ☐ Design index of existing Drafts
 - ☐ Design a way to navigate to Drafts via "Inbox..." menu

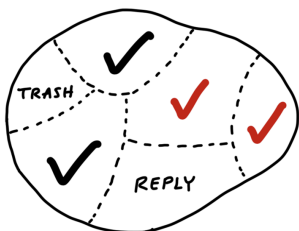
此时，团队突然觉得他们能够从宏观水平上看到整个项目的全貌。作为范围，所有重要的部分都在宏观层面上可见。它们中没有一个太大，以至于重要的或具有挑战性的任务能够在其中不被注意地隐藏起来。

与此同时，设计师在定位方面取得了一些进展。稍微进行了一些布线后，他们就能够标记为已完成。发送和存储的任务也在进行中。



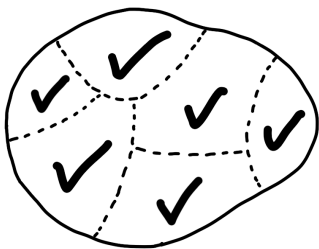
- = ● **Send**
 - = ☐ Handle draft message timestamps after sending
 - ✓ ☒ Hook up Send from Draft edit state
- = ● **Store**
 - = ☐ Remember draft content when editing draft
 - ✓ ☒ Hook up re-saving from Draft edit state
 - ✓ ☒ Remember addresses when editing draft
- = ● **Reply**
 - = ☐ Intercept attempts to reply to Topic if a draft from a different message exists
- = ● **Trash**
 - = ☐ Hook up Draft deletion from Draft edit state
 - ✓ ☒ Design a way to trash drafts from the index of drafts

一旦发送和存储完成后，只剩下一些任务需要处理废纸篓和回复。



- = ● **Reply**
 - = ☐ Intercept attempts to reply to Topic if a draft from a different message exists
- = ● **Trash**
 - = ☐ Hook up Draft deletion from Draft edit state
 - ✓ ☒ Design a way to trash drafts from the index of drafts

然后项目就完成了。



发现范围

范围映射并不是计划。你需要在绘制地图之前走遍整个领域。正确绘制的范围并不是为了整洁而任意分组或分类。它们反映了可以独立完成的真实地面事实——问题中的基本相互依赖和关系。

范围源自相互依赖。部分之间的依赖关系决定了何时可以说某项工作“完成”。你事先不知道工作和相互依赖实际上是什么。我们之前谈到过想象与发现的任务。同样的原则也适用于范围。范围需要通过实际工作的过程中发现，看看事物如何连接或不连接。

这就是为什么在项目开始时，我们不指望看到准确的范围。我们更可能在第一周结束或第二周开始后看到它们，团队有机会做一些真正的工作，找到问题结构中的自然分界线。

在一开始，范围出现一些调整和不稳定是正常的。随着团队摸索出边界的真正位置，线条会被重新绘制或范围会被重命名，就像上面的例子一样。团队专注于保存和编辑草稿的特定问题，所以很容易早期确定该范围。直到他们深入细节，才注意到有关发送草稿的具体任务，并将其作为单独的范围。

如何知道范围是否正确

合理的范围展现了项目的结构。当你感到身体疼痛时，你不必质疑它是在手臂、腿还是头部。你知道身体的部位和名称，这样你就可以解释疼痛的位置。同样，每个项目都有一个自然的结构，这源自你想要的设计、你所在的系统以及你必须解决的问题的相互依赖关系。

以下三个迹象表明范围正确：

1. 你觉得自己可以看到整个项目，没有重要的事情被隐藏在细节中令你担忧。
2. 项目的讨论变得更加流畅，因为范围给了你正确的语言。
3. 当出现新任务时，你知道在哪里放置它们。范围就像容器，你可以轻松地把新任务放进去。

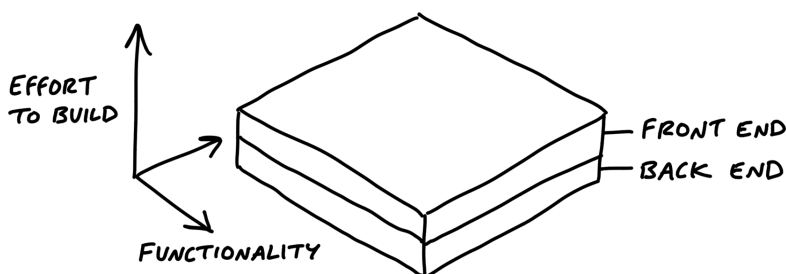
另一方面，以下三个迹象表明应重新绘制范围：

1. 很难说一个范围何时“完成”。这通常发生在范围内的任务是无关联的情况。如果范围内的问题是毫无关联的，完成一个任务并不会让你更接近完成另一个任务。在这种情况下，最好寻找一些你可以分解的部分，就像草稿的例子一样。
2. 名称不是项目独有的，比如“前端”或“错误”。我们把这些称为“杂物箱”和“杂物抽屉”。这说明你的整合还不够，因此你永远无法独立地标记完成一个范围。例如，对于错误，最好将它们归类到特定的范围下，这样你就可以知道，例如，“发送”是否完成，或者在把它忘记之前是否需要先修复几个错误。
3. 太大，无法很快完成。如果一个范围变得太大，包含了太多任务，它就会变成一个长期的主任务清单，带有所有的缺陷。最好将其分解成可以在较短时间内解决的部分，这样在解决问题的过程中就会有胜利，同时也有了问题之间的边界。

让我们以一些处理不同类型任务和范围的小贴士结束本章。

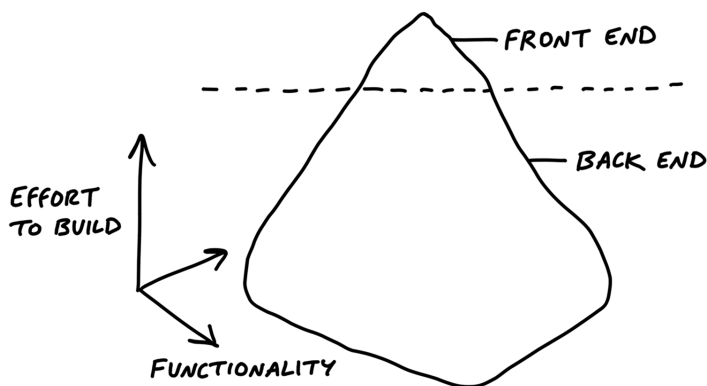
分层蛋糕

大多数软件项目需要一些 UI 设计和薄薄的代码层。想象一个数据库应用程序，你只需要输入信息、保存它，并将其显示出来。这类工作看起来像一个“分层蛋糕”：你可以通过 UI 表面积来评估工作，因为后端工作薄而均匀分布。在这种情况下，你可以将所有设计和程序员任务集成到同一个范围内。对于大多数“信息系统”类型的应用来说，这是一个很好的默认选择。



冰山

但有时后端工作要比 UI 工作多得多，或者相反。例如，一个只需要提交表单的新功能可能需要非常复杂的业务逻辑来返回正确的答案。这种工作就像一座冰山。



对于冰山，将 UI 作为一个独立的工作范围可能会有所帮助（假设 UI 与后端复杂性无关）。如果后端足够复杂，你可以将其拆分为独立的部分，然后将其转化为范围。在这种情况下目标是定义一些不同的事物，可以分阶段完成和集成，而不是等到最后一刻，指望一切都能顺利进行。

有时你也会看到颠倒的冰山，其中 UI 复杂性很大，而后端复杂性较小。例如，日历的数据模型并不复杂，但渲染多天事件并跨越网格单元可能需要大量时间和问题解决。

对于后端和前端冰山，我们总是要质疑它们，然后才接受它们作为事实。复杂性真的是必要的且不可简化吗？我们需要那种花哨的 UI 吗？是否有不同的方式来构建后端流程，使其与系统的其他部分的相互依赖性更少？

杂烩

几乎总会有一些不适合放入范围的任务。我们允许自己有一个“杂烩”清单，用于放置不属于任何范围的松散任务。但我们总是保持怀疑的眼光。如果它的长度超过三到五个项目，那就有些可疑，可能有范围需要绘制。

用~标记好做的事项

随着你深入问题，不断出现新任务。你会发现代码可以整理，需要处理边缘情况，并改进现有功能。处理所有这些改进的一个好方法是将它们记录为范围内的任务，但在前面加上一个~。这样，团队中的每个人都可以不断地将“必须做”的与“可以做”的分开。

在没有截止日期的世界里，我们可以永远改进。但在有限的时间框架内，我们需要一把砍刀来减少不断增长的范围。在一个项目或整个范围的开头加上~是我们的最佳工具。当我们谈到在第 14 章[决定何时停止](#)中删减范围时，我们还会回到这个技巧。

展示进展



善良的经理不喜欢询问项目状态。这很尴尬，感觉像唠叨，而且当他们不得不追问以充分了解情况时，情况变得更糟。

经理们宁愿能够在需要时自己看到项目状态。我们在上一章中看到，将待办事项组织成范围有助于团队掌握工作进展。但是这并不能直接帮助经理。待办事项存在一些问题，这使它们不足以判断状态。

那些不存在的任务

考虑一个列表，其中有一些已完成的项目，没有剩余未完成的项目。这可能意味着所有工作都已完成。但这也可能意味着团队知道还有更多工作，但尚未定义任务。

有时候，团队会在项目早期定义一个“范围”，但尚未填充任务。这标志着需要完成一些工作，但实际任务尚未被发现。

或者考虑在范围末尾进行一些质量保证。所有任务都已完成。没有别的事情可做。然后测试的行为为发现的问题填充了范围的新任务。

这回到了“想象的”与“发现的”任务的概念。在我们天真的计划性列表中，有人逐渐将其填满并逐一勾选。而在现实生活中，通过参与问题的解决，会发现问题。这意味着随着团队取得进展，待办事项实际上会增加。



如果我们试图在 t_2 时判断项目进行了多少，我们将被误导。从外部人员的角度来看，无法知道未完成的任务的数量是减少还是增加。要了解这一点，您需要更多了解范围内工作的上下文，以理解这些特定任务已完成意味着什么，以及是否可能还会出现其他任务。

估算不显示不确定性

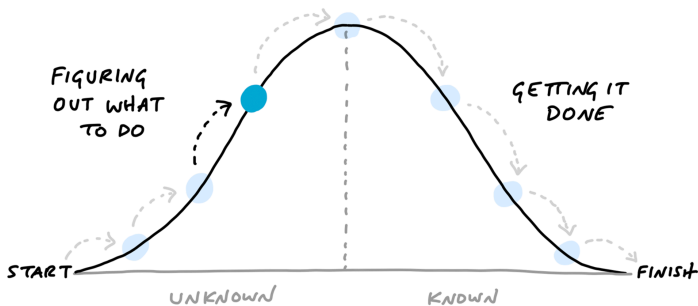
一些团队尝试将估算值附加到其任务或范围以报告状态。估算的问题在于，根据被估算工作的性质，其含义大相径庭。

假设有两个任务，都估计需要四个小时。如果一个任务是团队在过去做过十次的事情，您可以相信这个估算。假设另一个任务是团队从未做过的事情，或者它具有不清晰的相互依赖关系。如果一切都进行得很完美，可能只需要四个小时，但由于其中的未知因素，可能会延长到两三天。将“4 小时，或者可能 3 天”写为估算是没有意义的。

鉴于这一点，我们想出了一种方法，可以在不计数任务和不使用数字估算的情况下查看项目的状态。我们通过将焦点从已完成或未完成转移到未知和已解决来实现这一转变。为了实现这一转变，我们使用了山丘的比喻。

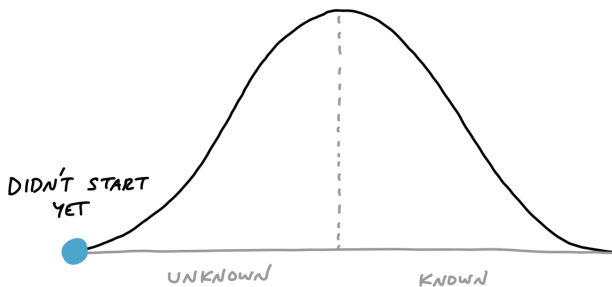
工作就像一座山

每项工作都有两个阶段。首先是上坡阶段，弄清楚我们的方法和我们要做什么。然后，一旦我们能看到所有涉及的工作，就进入了下坡阶段的执行阶段。



让我们用日常例子来感受一下山的感受。

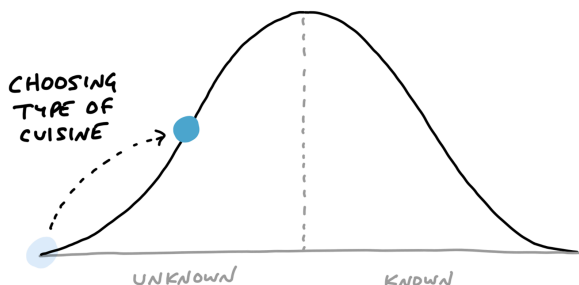
假设您计划举办一个晚宴。您已经确定了日期，但还有几周时间，您还没有考虑要做什么菜。您不知道这顿饭会是哪种菜系，也不知道要做什么菜。这将把您放在山的左下方的起点处。



接下来，您考虑了谁会出席，并注意到有几个素食主义者。这就排除了一些选择（比如烧烤），但仍然剩下很多选择。您考虑意大利菜和印度菜。您认为印度菜可能更有趣，有更多有趣的素食选择。因此，您决定寻找印度菜的食谱。

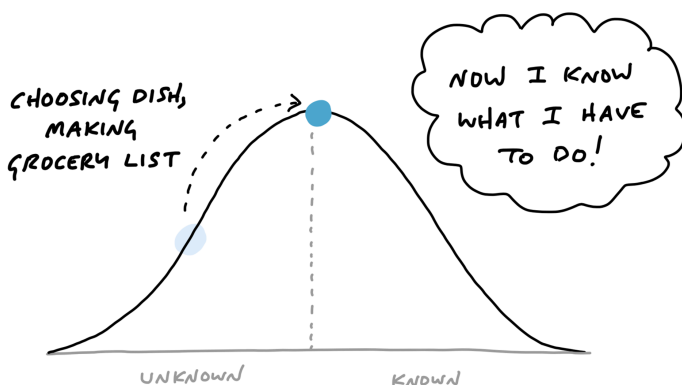
在这一点上，“项目完成了多少百分比”这个问题不合适。如果有人要求您估计购物和准备需要多长时间，您也无法回答，因为您还没有选择菜肴。答案将是：“我已经做了一些工作，弄清楚了菜系，但我还没有将其缩小到一个具体的菜肴。”我们可以通过将您放在

山的“弄清楚”一侧的中间位置来表示这一点。



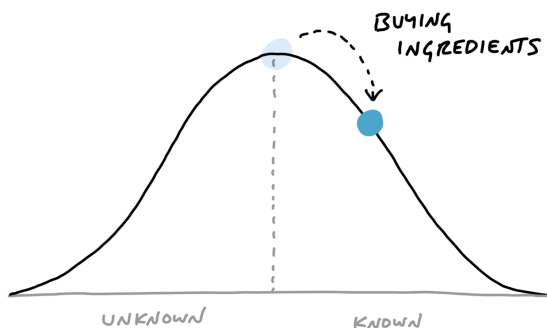
接下来，您在网上搜索并查看您的食谱书。您想找到一个既有趣又不需要太难找到食材的食谱。您选定了一个食谱并准备了购物清单。

现在您的位置与之前完全不同。感觉从“我还不确定我在做什么”变成了“现在我知道该做什么了”。您在山的顶部。

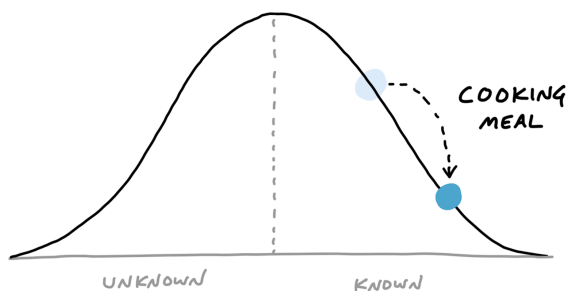


从这个角度，您可以看到剩下的所有步骤。甚至可以估计所有工作需要多长时间（“让我看看...购物需要一个小时，准备需要 30 分钟，烹饪需要 45 分钟...”）。

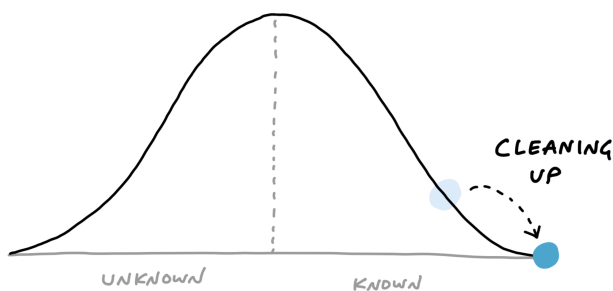
晚宴前一天，您去杂货店购买食材。这将使您向下移动。您离任务完成更近了。



接下来是准备和烹饪晚餐。



晚餐结束后，只剩下一点工作：清理。

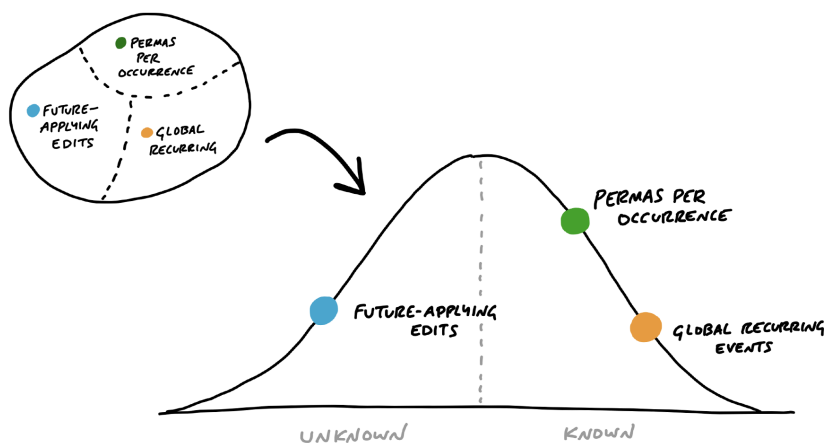


请注意，山显示了不同阶段工作的感觉。上坡阶段充满了不确定性、未知因素和问题解决。下坡阶段则充满了确定性、信心、看清一切和知道该做什么。

山上的范围

我们可以将山与上一章的范围概念相结合。范围为我们提供了项目的语言（“定位”，“回复”），而山描述了每个范围的状态（“上山”，“下山”）。

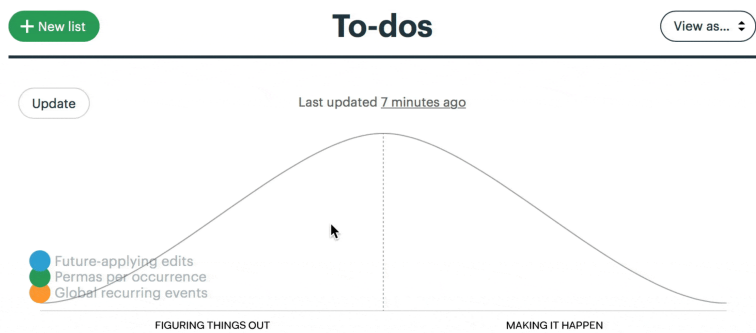
为了了解范围的状态，我们可以在山上用不同的颜色表示每个范围。



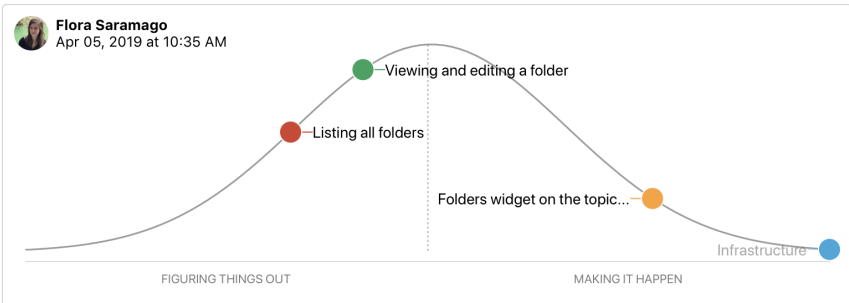
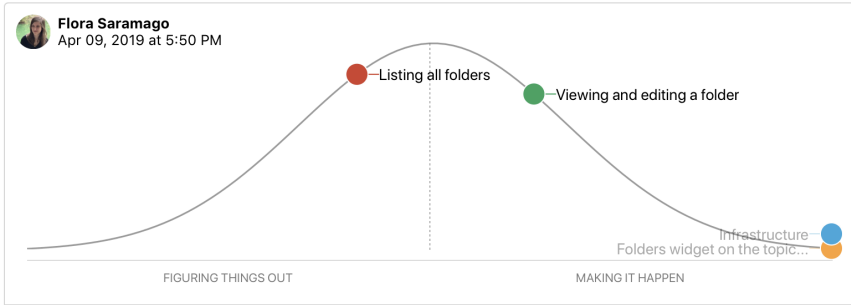
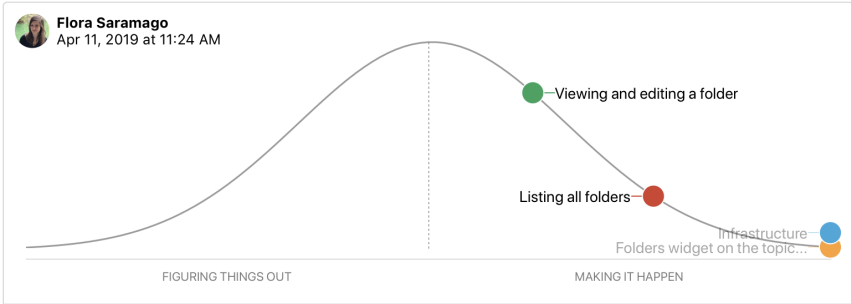
这是 Basecamp 中实现重复事件的项目的快照。在这里，“未来应用编辑”是一个仍在进行中的范围，还有一些重要的未知问题需要解决。其他两个范围没有什么重大的未知问题，“全局重复事件”已经接近完成。

不需要询问的状态

我们为 Basecamp 构建了一个专门用于创建山坡图表并用几次点击更新的功能。团队成员具有工作进展的完整上下文，他们直觉地将范围拖放到位置，并保存一个新的更新，这将记录在项目中（参见[如何在 Basecamp 中实施 Shape Up](#)）。



对于管理人员来说，比较过去状态的能力是关键功能。它不仅显示了工作的进展，还显示了工作的“移动”。



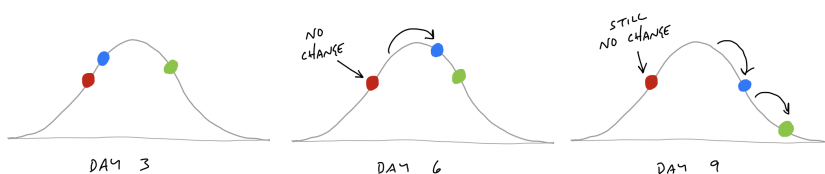
有了这样的二次视图，管理人员可以判断哪些工作正在进行中，哪些工作停滞不前。他们可以看到团队选择解决的问题以及从未知到已知再到完成的每个阶段所花费的时间。

当管理人员对项目感到焦虑时，这份报告成为他们的首选。由于这是自助式的，所以不需要打扰团队提出尴尬的状态问题。而且，在出现问题的情况下，管理人员可以直接就相关工作的问题进行讨论。“看起来‘自动保存’已经上山很久了。是什么未知问题阻碍了它的进展？”管理人员可以就项目的具体部分进行讨论，而不必首先把它们与其他正常进行的工作分开。

没有人说“我不知道”

没有人愿意向管理层举手说“我不知道如何解决这个问题”。这导致团队隐藏不确定性并积累风险。当有人陷入困境或陷入循环时，正是最大的风险和机会所在。如果我们能早日发现这些时刻，就可以通过资深人员的帮助或重新设计概念来解决它们。如果我们没有发现它们，未解决的问题可能会一直持续到周期的后期，从而危及项目。

山坡图表允许每个人看到有人可能陷入困境，而不需要他们亲口说出来。一个不动的点实际上就是举手示意：“这里可能有问题”。

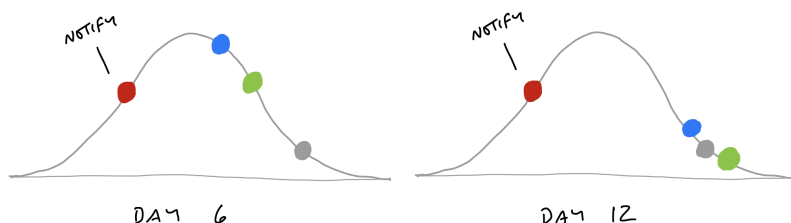


一旦被发现，上山/下山的语言促进了对话。重点不在于人（看起来你陷入困境了！），而在于工作。问题是：我们可以解决什么来让它过山呢？

重构范围的提示

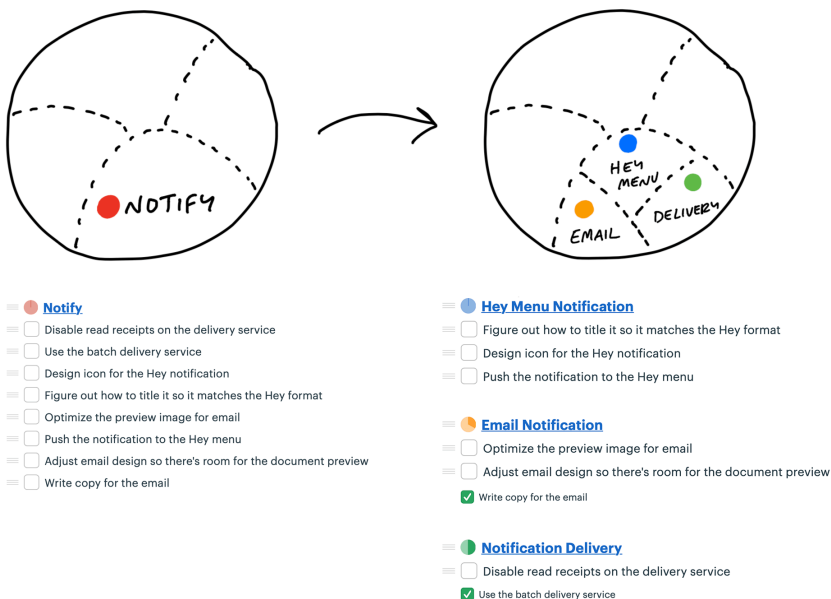
有时候深入研究一个陷入困境的范围会发现它根本没有陷入困境。问题在于范围的划分方式有问题。

下面是一个“通知”范围在山上陷入困境很长时间的案例。

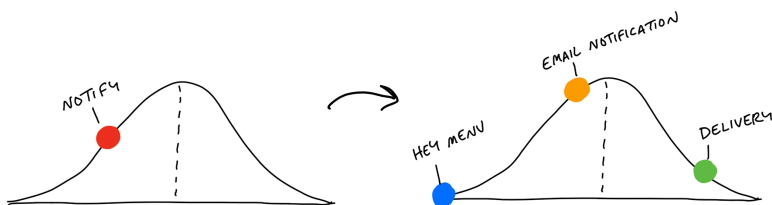


当我们与团队核对时，结果表明工作进展得很顺利。问题在于“通知”并不是一个单一的事物。它有三个不同部分：设计电子邮件，在后端传递电子邮件以及在应用程序菜单中显示通知。团队大部分已经完成了传递电子邮件的代码。电子邮件的设计几乎已经确定。但他们还没有开始处理应用程序内显示的通知。无法确定“通知”作为一个整体是否已经解决了，因为它的部分已经解决了，部分还没有。

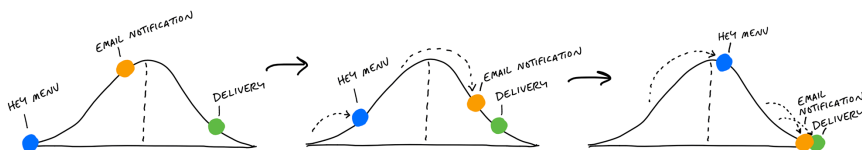
在这种情况下，解决方案是将范围分解成可以独立移动的较小的范围。



现在团队可以移动每个点，准确地显示工作的进展。



这带来的好处在于二次视图。通过将范围分开，它们可以随着时间的推移独立移动。现在团队可以比以前更频繁地展示更多的进展。



向山顶努力

一些团队在首次尝试山坡图表时会遇到倒退的困难。他们认为一个范围已经解决了，将其移到山顶，但后来发现一个意想不到的未知问题。

当这种情况发生时，通常是因为有人在头脑中做了上山的工作，而不是用双手。在头脑中构思方法只是上山的第一步。我们经常对如何解决问题有一个理论——“我只是使用那个 API”，然后实际情况证明更为复杂。将上山的第一阶段视为“我已经考虑过这个问题”，第二阶段是“我已经验证了我的方法”，最后一段到达山顶是“我已经建立的东西让我相信没有其他未知问题了”。

按正确的顺序解决问题

除了看到工作的进展，我们还可以使用山坡图表来安排工作的顺序——解决哪些问题以及以哪种顺序解决。

有些范围比其他范围更具风险。想象两个范围：一个涉及地理编码数据——团队以前从未做过的事情。另一个是设计和实施电子邮件通知。两者都存在未知问题。两者都从山的底部开始。这时团队会问自己：如果在周期结束时时间不够，我们可以很容易地匆匆忙忙地完成其中哪一个——尽管存在未知问题——哪一个可能会证明比我们想象的更困难？

这促使团队优先处理最令人恐惧的工作。一旦他们上山了，他们就会离开那里，然后寻找在完成下山的工作之前是否有任何至关重要的事情。最好在项目的早期就将一些关键的范围移到山顶，留下那些细节调整以后再解决的事情。

工作会膨胀以填满所提供的时间。如果团队首先从电子邮件模板开始，他们可能会轻易地花上几周来反复修改文本或创建最佳的电子邮件设计。但他们并不需要那样做。在最后一周内，可以制定出某个版本的电子邮件模板，它就足够了。另一方面，地理编码器可能会出现新问题，团队可能会为此苦苦挣扎数周。他们不希望这种意外发生在周期的最后。

记者有一个称为“倒金字塔”的概念。这个概念是他们的文章从最重要的信息开始，然后按重要性递减的顺序添加细节和背景信息。这使得印刷报纸的设计人员可以将关键部分放在头版，并根据需要剪裁结尾，而不会丢失任何重要内容。

有效的团队以相同的方式安排他们的解决问题。他们首先选择最重要的、最具未知问题的问题，将其放在山顶，然后将那些最常规或最不令人担忧的问题留到最后解决。

当周期接近尾声时，团队应该已经完成了重要的事情，并留下了各种“不错的事情”和“可能”的事情。这将引出下一章，关于何时停止的决定。

决定何时停止



当周期结束临近时，我们迄今为止所覆盖的技术将使团队处于一个良好的完成和交付位置。这些塑造的工作为他们设立了防护栏，以防止他们四处游荡。他们一次集成一个范围，所以没有半成品的工作横七竖八地散落在那里。因为他们在安排工作时首先优先考虑了这些未知因素，所以所有最重要的问题都已经解决了。

然而，总是有更多的工作而时间有限。按时交付意味着交付不完美的东西。当你审视自己的工作并问自己：“这样可以吗？这准备好发布了么？”时，总会有些不安。

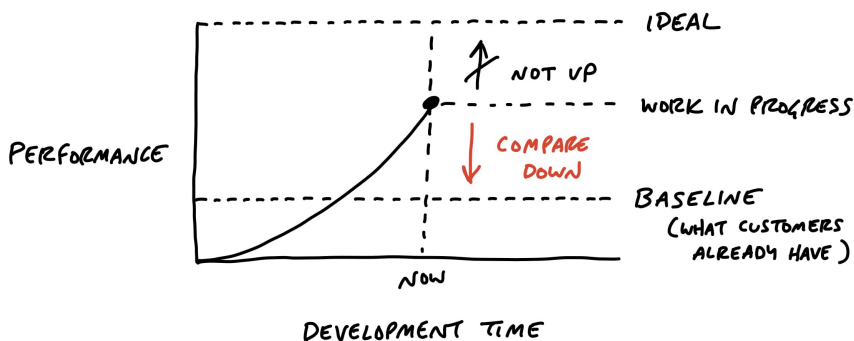
与基线比较

设计师和程序员总是希望做出最好的工作。无论按钮是在着陆页的中心还是在设置屏幕的第二页，设计师都会全力以赴。而最优秀的程序员希望代码库感觉像一个完整的整体，完全符合逻辑，覆盖了每一个边缘情况。

对工作的自豪感对于质量和士气是重要的，但我们需要将其指向正确的目标。如果我们追求理想的完美设计，我们将永远无法达到。同时，我们也不希望降低我们的标准。我们如何做出决定，认为我们已经做得足够好并继续前进呢？

将比较的焦点转移到 **基线** 而不是理想状态有所帮助——即与当前的现实情况相比。客户今天如何解决这个问题，没有这个功能？这个功能消除了哪些令人沮丧的变通方法？客户应该忍受多久某些不起作用的东西或等待解决方案，因为我们不确定设计 A 是否比设计 B 更好？

看到迄今为止我们的工作比当前的替代方案更好，让我们对我们取得的进步感到更满意。这激励我们对那些拖慢我们速度的事情做出决策。这与我们的个人感受无关，更多地关乎客户的价值。这是“永远不够好”和“比他们现在拥有的更好”之间的区别。我们可以说：“好吧，这并不完美，但绝对有效，客户会觉得这对他们来说是一个重大改进。”



通过与基线比较来进行范围裁减，而不是追求某种完美的理想状态

限制激发权衡

请记住，六周的打赌有一个 **断路器** ——如果工作没有完成，项目就不会发生。

这迫使团队进行权衡。当有人说“如果……会不会更好”或者发现另一个边缘情况时，他们应该首先问自己：“这样做有时间吗？”如果没有截止日期，他们很容易因为并不真正值得额外时间的变更而延迟项目。

我们期望我们的团队积极地进行权衡，质疑范围，而不是将任务完成得很勉强。我们为自己创造工作。在我们接受工作之前，我们应该质疑任何新工作是否必要。

范围会像草一样生长

范围会自然增长。范围扩张并不是坏客户、糟糕的经理或糟糕的程序员的错。项目在宏观尺度上是不透明的。直到你深入工作，你才能看到项目的所有微小细节。然后你会发现不仅有你没有预料到的复杂性，而且还有许多可以被修复或改进的事情。

每个项目都充满了我们不需要的范围。产品的每个部分并不需要同样突出、同样快速和同样精致。每个使用案例并不同样常见、同样关键，或者同样与我们试图销售给的市场保持一致。

事实就是如此。与其试图阻止范围的增长，不如给团队提供不断削减范围的工具、权力和责任。

裁减范围并不是在降低质量

在选择要执行的事物以及在多大程度上执行它们时，不会在产品中留下漏洞。做选择会让产品变得更好。它会让产品在某些方面变得更好，而在其他方面变得更糟。对范围挑剔会使产品 *有所不同*。区分核心部分和外围部分的做法使我们在竞争空间中移动，使我们与其他做出不同选择的产品更加相似或不同。

可变的范围并不是在牺牲质量。我们对我们的代码质量、视觉设计、界面中的文案以及我们交互的性能非常挑剔。关键是要问自己哪些事情真的重要，哪些事情推动了进展，哪些事情对我们试图解决的核心使用案例有所影响。

裁减范围

人们经常谈论“裁减”范围。我们使用一个更加强烈的词——**hammering**——来反映反复敲击范围的力量和决心，以使其适应时间范围。

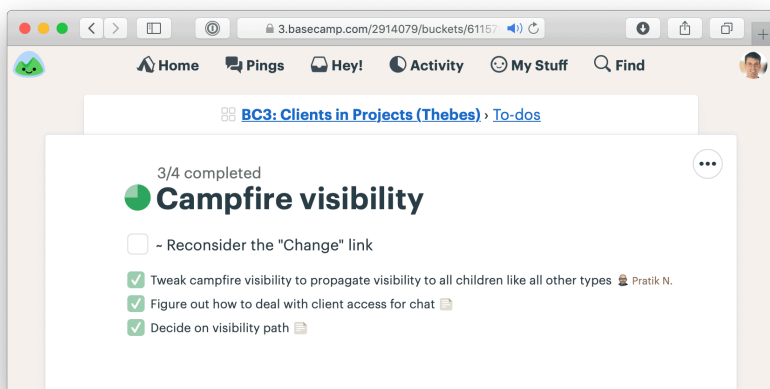
在项目进行中，当我们想要修复、添加、改进或重新设计一些东西时，我们会问自己：

- 这对新功能来说是“必须的”吗？

- 如果没有这个，我们可以进行交付吗？
- 如果我们不做这个会发生什么？
- 这是一个新问题，还是已经存在的问题，客户已经忍受了？
- 这种情况或条件发生的可能性有多大？
- 当这种情况发生时，有哪些客户会看到？它是核心—每个人都使用的—还是更像是边缘情况？
- 如果这种情况或条件真的发生，它的实际影响是什么？
- 当某个事情对特定使用案例表现不佳时，这个使用案例与我们预期的受众有多契合？

固定的截止日期激励我们提出这些问题。可变的范围使我们能够对其进行实际行动。通过不断地削减范围，我们将关注放在只有我们需要做的事情上，以便在时间范围结束时交付出一些有效的东西，让我们可以为此感到自豪。

在整个周期中，当我们的团队发现工作时，你会听到我们的团队谈论 **必须有的** 和 **可有可无的**。必须有的被记录在范围的任务中。在这些任务完成之前，范围不被视为“完成”。可有可无的可能会在范围完成后仍然存在。它们的前面会有一个波浪符号（~）。如果团队在最后有额外的时间，这些任务可能会继续存在，如果没有，它们可能会被裁减。通常它们最终不会被实现。将它们标记为可有可无就是对范围进行裁减的行为。



一个已完成的范围，有一个未完成的 **可有可无** 任务（前面标有“~”）

QA 是为了边缘情况

在当前规模下（数百万用户和产品团队大约有十几人）的 Basecamp 中，我们有一个质量保证人员。他们会在周期结束时加入，并寻找核心功能之外的边缘情况。

QA 可以将他们的注意力限制在边缘情况上，因为设计师和程序员对其基本工作质量负责。程序员编写自己的测试，团队共同努力确保项目按照塑造的要求做到该做的。这源自于给团队整个项目的责任而不是分配给他们个别任务（参见第 9 章，[移交责任](#)）。

多年来，我们没有质量保证角色。然后在我们的用户基数增长到一定规模后，我们发现小的边缘情况开始影响大量用户。增加额外的 QA 步骤帮助我们改善了这些用户的体验，并减少了他们为支持工作带来的不成比例的负担。

因此，我们将 QA 视为一种提升，而不是所有工作必须通过的一个门或检查点。我们与 QA 相比，要好得多。但我们不依赖 QA 来交付符合预期的质量功能。

QA 会产生一些默认都是 **可有可无** 的 **发现的任务**。设计师和程序员团队对它们进行分类，并根据严重程度和可用时间，将其中一些提升为 **必须有的**。最严格的做法是将接收的 QA 问题收集在一个单独的待办事项列表上。然后，如果团队决定某个问题是必须有的，他们会将其拖到它所影响的相关 **范围** 的待办事项列表中。这有助于团队看到，在处理该问题之前，范围并没有完成。

我们将代码审查也同样对待。团队可以在不等待代码审查的情况下进行交付。没有正式的检查点。但代码审查会让事情变得更好，所以如果有时间并且有意义，一些资深人员可能会查看代码并提出反馈。这更多地是利用教学机会，而不是在我们的流程中创建一个必须每次都发生的步骤。

何时延长项目

在非常罕见的情况下，我们会延长一个超过截止日期的项目数周。我们如何决定何时延长一个项目，何时让 **断路器** 履行其职能？

首先，未完成的任务必须是真正的 **必须有的**，经受得住每一次试图将其 **范围裁减** 的尝试。

其次，未完成的工作必须全部是 **下坡的**。没有未解决的问题；没有悬而未决的问题。在周期结束时的 **上坡** 工作指向塑造中的疏忽或概念上的漏洞。未知因素太冒险了。如果工作处于上坡状态，最好是在下一个周期中做其他事情，并将陷入困境的项目放回塑造阶

段。如果找到了修补漏洞的可行方法，那么可以考虑在将来再投入更多时间。

即使满足了考虑延长项目的条件，我们仍然更喜欢遵循大多数项目的 **上限**。两周的 **冷静期** 通常为有一些太多的 **必须有的** 的团队提供了足够的余地，使其能够在下一个周期开始之前完成交付。但这不应成为一种习惯。进入冷静期通常会指向塑造过程中的问题或团队的表现问题。

迈出下一步

让风暴过去

如果不小心，发货实际上可能会带来新的工作。功能发布会带来功能请求。客户会说：“好的，那很棒，但我们一直在要求的另一件事呢？”错误会出现。改进建议会涌现。每个人都专注于新事物并做出反应。

如果你发布的功能改变了现有的工作流程，反馈可能会特别强烈。即使仅仅是视觉上的改变有时也会引发强烈的强烈反对。少数客户可能会反应过度，说出像“你毁了它！改回去！”这样的话。

保持冷静，避免下意识的反应很重要。给它几天时间，让事情平息下来。要坚定，并记住你为什么要做出这个改变以及这个改变对谁有帮助。

保持无债状态

很诱人去对反馈做出改变的承诺，但那样你就不再拥有下一个周期的清白之地了。记住：这些只是刚刚提出的 **原始想法**。处理它们的方式是委婉地说“不”。说“不”并不妨碍你继续思考它们，或许将它们塑造成将来的项目。另一方面，说“是”会剥夺你将来的自由。这就像负债一样。

记住，你刚刚发布的東西是一个六周的 **赌注**。如果产品的这一部分需要更多时间，那就需要一个新的赌注。让刚刚出现的请求或错误与下一个 **赌注桌** 上的其他事物竞争，以确保它们在战略上的重要性。

反馈需要被塑造

我们又回到了起点。刚刚来自客户反馈的原始想法还不具备可操作性。它们需要被塑造。它们是在塑造流程的第一步中谈到的原始输入：**设定边界**。

如果一个请求确实很重要，你可以在下一个周期的塑造轨道上把它作为首要任务。为团队另外选择一个项目进行构建，并利用那段时间来适当地塑造这个新想法。然后，在六周结束时，你可以在赌注桌上提出理由，并安排经过塑造的项目版本，以获得最大的成功机会。

结论

关键概念

本书中介绍的“Shape Up”方法紧密相连。可能需要一些思考和实验，才能提炼出正确的要点并将其调整到您的团队中。

无论您的团队能否一次性采纳该方法，我希望本书中的语言和概念能立即给您带来一些收获：

- 成形与未成形的工作
- 设定期望而非估算
- 以正确的抽象级别进行设计
- 用面包板和粗笔素描进行概念构思
- 通过有上限的下行赌注（断路器）进行打赌，并以不间断的时间予以尊重
- 选择正确的周期长度（六周）
- 周期之间的冷却期
- 将项目分解为范围
- 下坡与上坡工作，以及关于未知的沟通

- 范围锤炼，将必须做的与可选做的分开

联系我们

我们很乐意听取您的意见，以便使“Shape Up”方法更易于采用。我们遗漏了什么？仍然不清楚的地方有哪些？您希望我们谈论而未涉及的内容是什么？我们也很乐意听取您在尝试将其应用到团队和项目中时取得的成功和挑战。

请给我们发送电子邮件至 shapeup@basecamp.com。

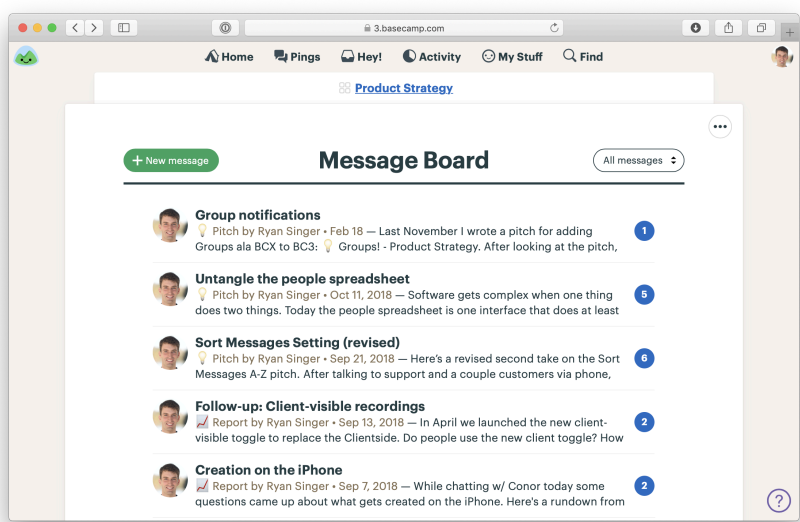
如何在 Basecamp 中实施 Shape Up

我们建立了[Basecamp](#)来实施 Shape Up 方法。与将工作分散在多个工具中不同，Basecamp 将所有项目沟通、任务管理和文档编制集中在一个地方。以下是我们使用它的方式。

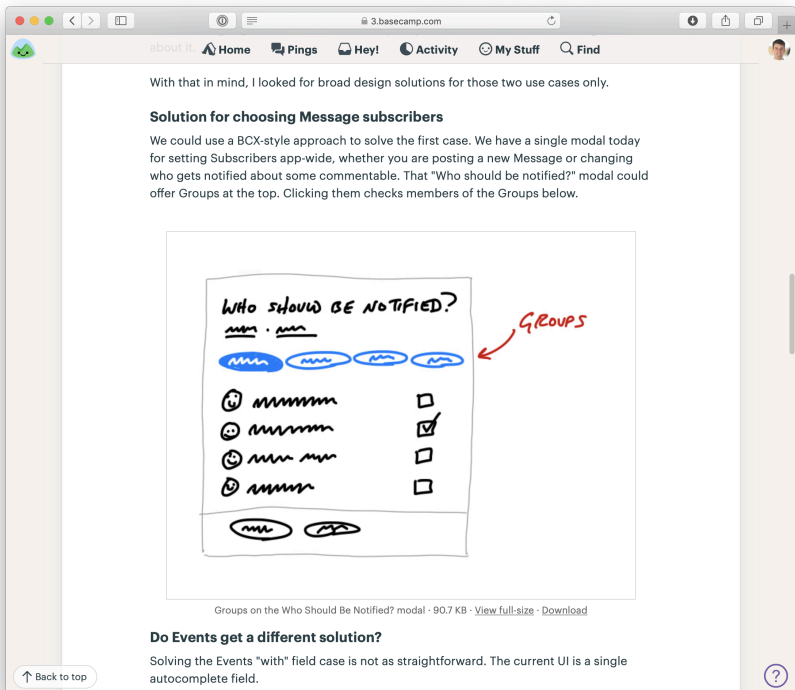
用于塑形的 Basecamp 团队

1. 为塑形创建一个 Basecamp 团队。我们称之为“产品策略”。
2. 添加进行塑形的人员，任何对提案提供反馈的可信任的人员以及在投注表上下注的人员。保持小组规模，并在其他地方更广泛地宣布下注（我们在 Basecamp 的总部进行），当该周期启动时。
3. 在消息板上将提案发布为消息。我们创建了一个名为“提案”的消息类别，并使用了灯泡表情作为图标。

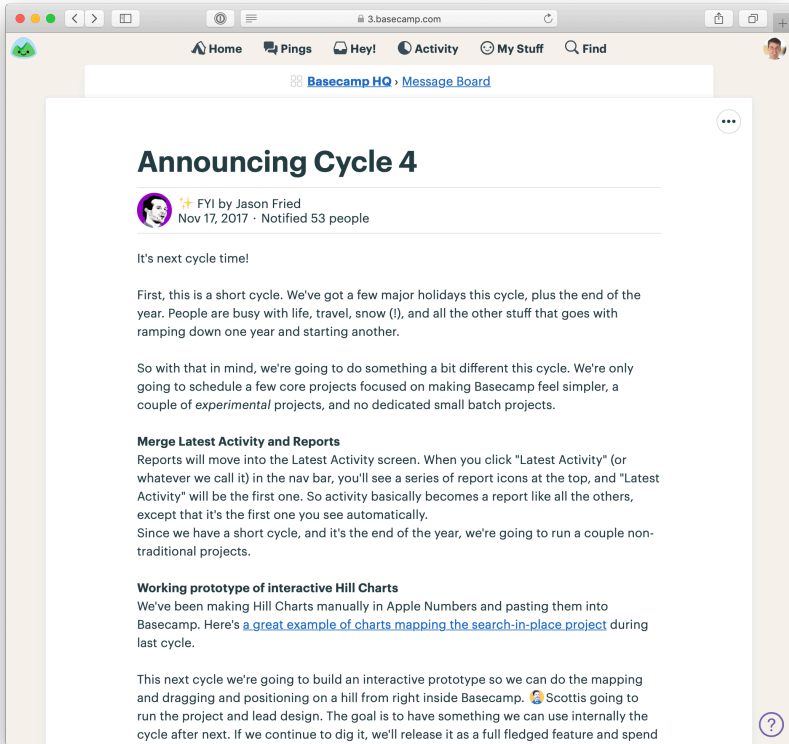
4. 使用 Campfire 聊天室来交换想法，并在周期之间协调投注表。我们通过视频聊天进行实际的投注表会议。



Basecamp 中产品策略团队的消息板上的提案



在提案中绘制的 iPad 上的草图

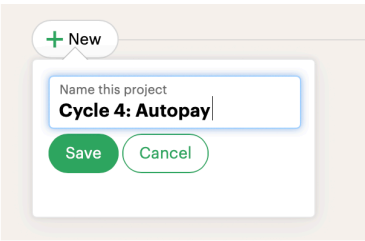


Jason 在总部 - 一个包括整个公司的 Basecamp 团队中宣布了下一个周期的下注

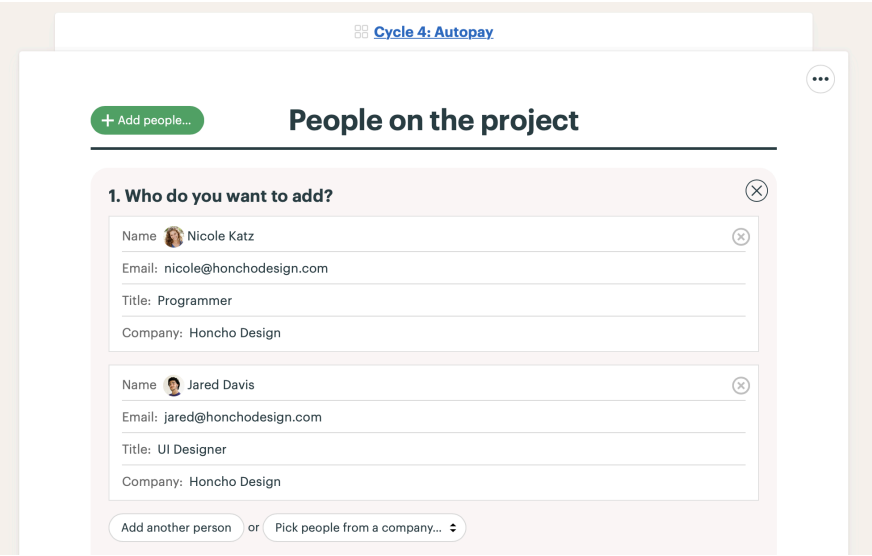
周期项目的 Basecamp 项目

1. 为六周周期中的每个项目创建一个 Basecamp 项目。通常我们会像这样在周期前加上一个名称或编号，比如“第 4 周期：自动支付”。
2. 将设计师和程序员添加到 Basecamp 项目中，他们将在该项目上工作。

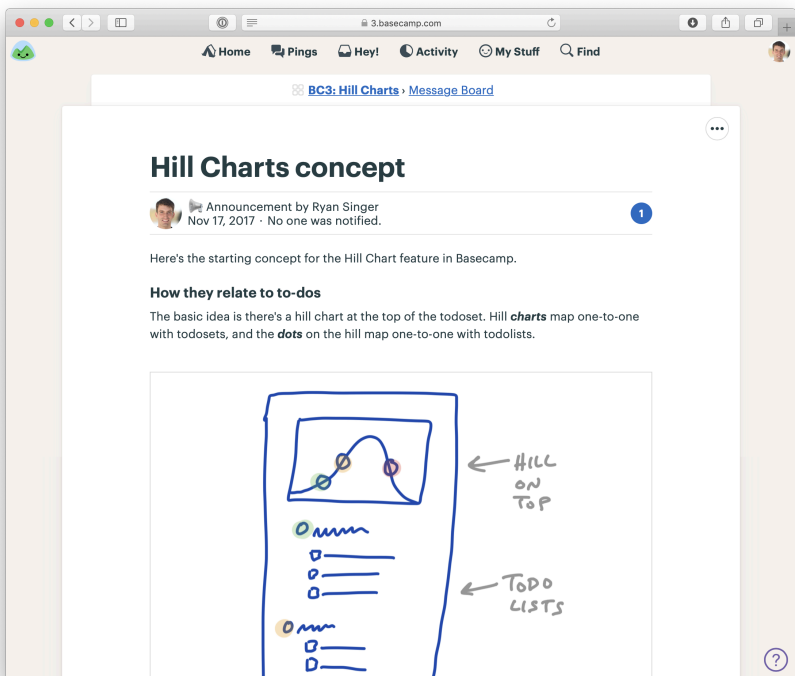
3. 在消息板上发布一个启动消息，其中包含提案或已塑形工作的重新表述，供团队参考。



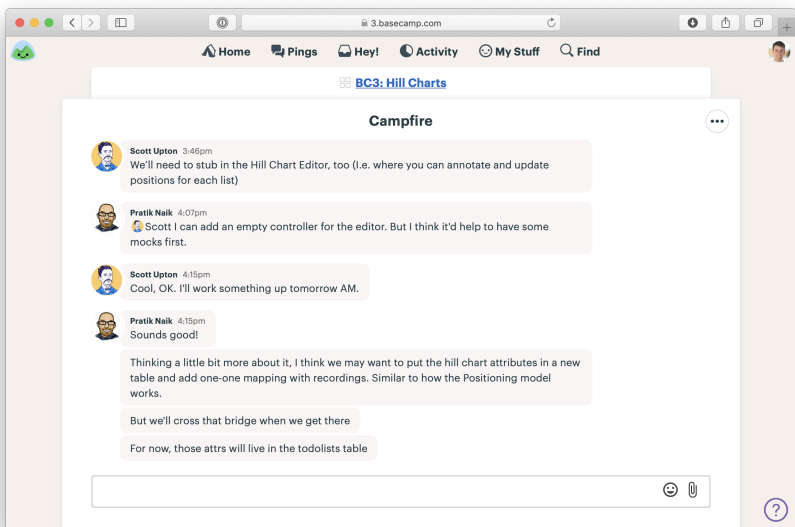
创建项目



添加设计师和程序员



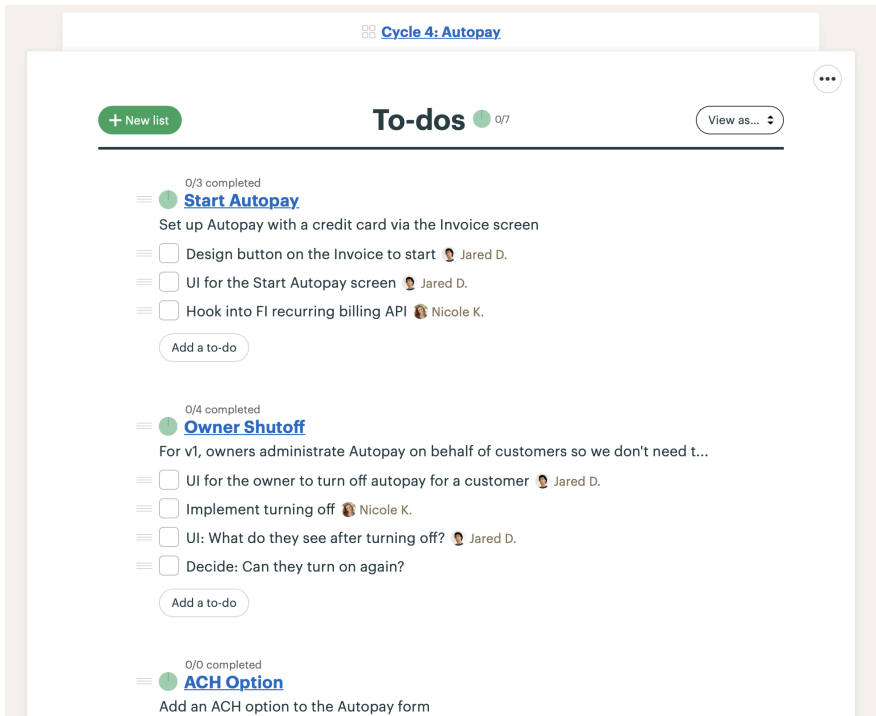
项目中的第一件事是一个包含已塑形概念的启动消息



团队在 Basecamp 项目的聊天室中进行沟通，开始工作

用于作用域的待办事项清单

1. 团队在适应了环境后开始尝试、发现任务，并[将它们映射到作用域中](#)。
2. 在 Basecamp 项目中，团队为每个作用域创建一个待办事项清单，比如“开始自动支付”或“ACH 选项”。有时我们会在待办事项清单的描述字段中总结作用域。
3. 将设计和编程任务添加到每个作用域的待办事项中。例如，“开始自动支付”有一个用于设计 UI 的任务，另一个用于将其与现有的循环计费 API 连接的任务。他们可以使用每个待办事项的讨论线程来发布更新或互相提问。
4. 随着团队发现新的作用域和任务，重复上述步骤。

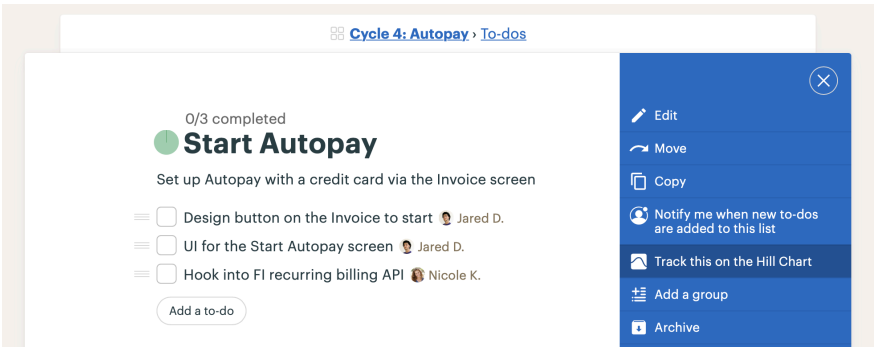


每个作用域的待办事项清单下面有设计和程序员的任务。注意：这些只是迄今为止发现的任务。

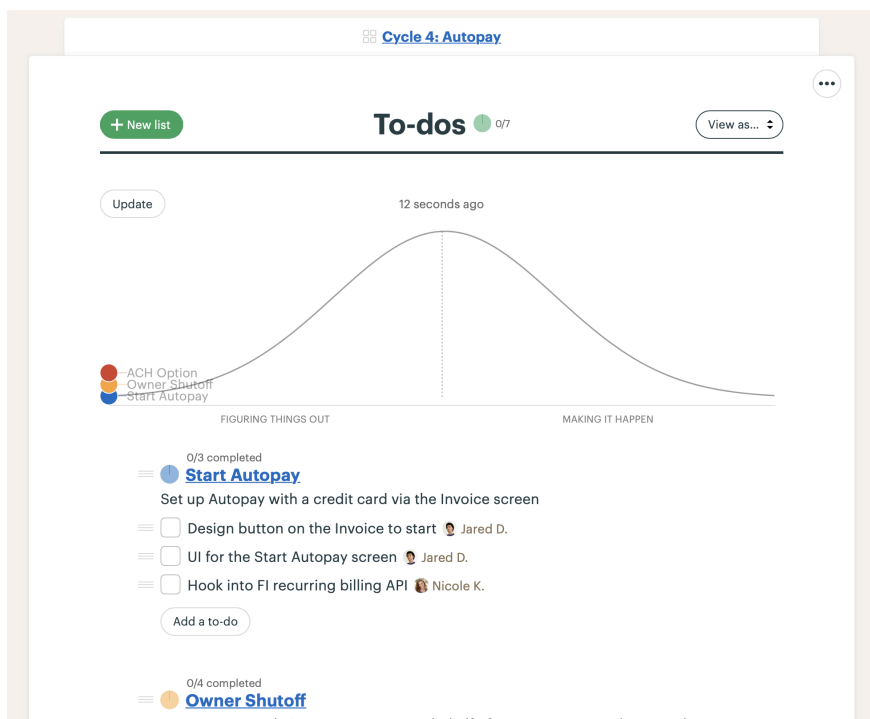
在 Hill 图上跟踪作用域

1. 转到每个作用域的待办事项清单页面，点击右上角的选项菜单（⋮）。点击“在 Hill 图上跟踪此事项”。这将在项目的整体待办事项部分顶部显示一个 Hill 图，并在该作用域（待办事项清单）对应的位置上显示一个点。
2. 对每个作用域（待办事项清单）重复此步骤。
3. 点击 Hill 图上的“更新”，并将点拖动以显示从“未知”到“已知”再到“完成”的进度。必要时使用注释功能添加评论。

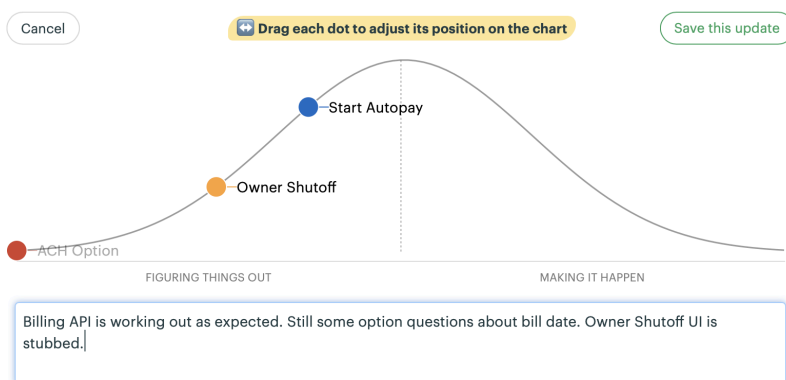
4. 要查看对 Hill 图的更新历史，请点击 Hill 图上方标有“上次更新”的时间戳。



通过待办事项清单页面上的选项菜单跟踪作用域（待办事项清单）到 Hill 图



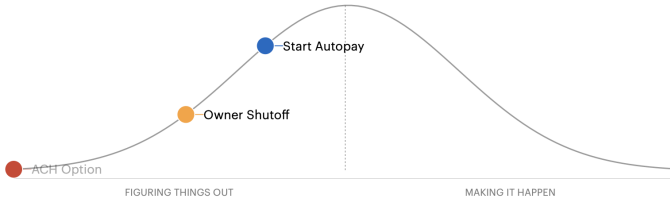
在启用了每个待办事项清单的 Hill 图后，每个作用域都会显示为一个点



拖动点以更新 Hill 图

Update

Last updated 2 minutes ago



2/5 completed

 Start Autopay

Set up Autopay with a credit card via the Invoice screen

☰ ☐ Hook into FI recurring billing API  Nicole K.

☰ ☐ Set correct bill date **NK** Nicole K.

☰ ☐ Move button to action menu on mobile  Jared D.

Add a to-do

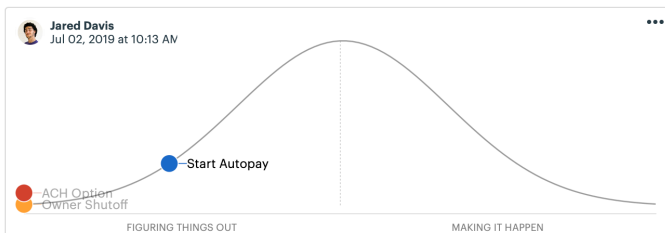
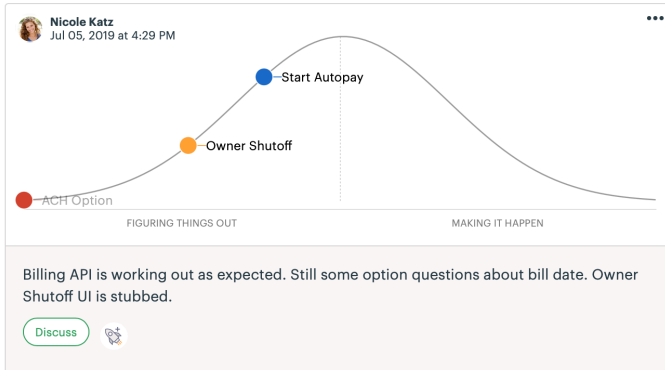
✓ UI for the Start Autopay screen Jared D.

✓ Design button on the Invoice to start  Jared D.

更新后的 Hill 图出现在待办事项页面的顶部

Hill Chart Progress

Today



点击 Hill 图顶部标有“上次更新”时间戳的位置，可以查看更新历史。最新的更新在最上面。

其他工具很难在一个地方看到项目的所有信息。Basecamp 将聊天、消息、待办事项和文档结合在一个界面中，强大到足以满足技术人员的需求，但又友好到可以让其他人使用。程序员、设计师、质量保证人员和经理都感到同样自在，并可以在项目上无缝地协作。[免费试用 Basecamp 30 天。](#)

调整到你的尺寸

基本真理 vs. 具体实践

要将“塑造”方法应用于你的公司，有助于将基本真理与具体实践分开。

工作必须得来自某个地方，找到正确的工作是需要付出努力的。这就是塑造。对工作进行塑造可以为完成工作的人——无论是一个独立团队还是你未来的自己——设定更清晰的界限和期望。如果我们不通过塑造在前期做出权衡，那么在面临截止日期、技术限制或资源约束时，宇宙会迫使我们在疯狂的匆忙中做出权衡。

对于押注也是如此。六周可能不是你的团队确切的时间框架。但是对于每个人来说，做出不明确或不明确结局的承诺所带来的后果是一样的。无论我们押注的具体时间框架是什么，我们都应该谨慎对待我们的押注，并通过风险控制来限制我们的下行风险。

在建设阶段，无论你是否在山形图上追踪它们，都会有一些未知因素需要处理。我们需要区分已知因素和未知因素，以便我们可以按照正确的顺序对工作进行排序，并为未知因素保留容量。

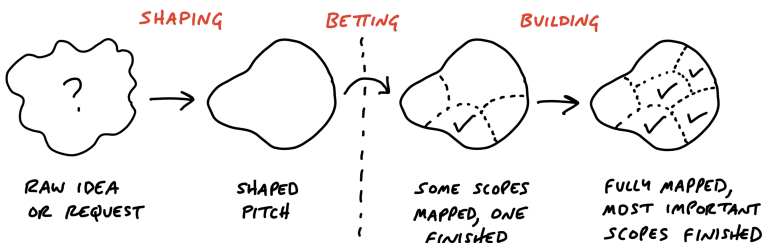
这些基本真理适用于你的组织规模。另一方面，具体实践是依赖规模的。让我们看看在非常小的初创公司和已经发展到足够大以至于需要专业角色和更多结构的组织中实施“塑造”方法意味着什么。

小到可以临时应付

当你的团队只有两三个人时，每个人都会做一点点所有事情。由于少数人承担了许多职责和角色，很难长时间集中精力投入特定项目。进行编程的人可能还要回复客户的请求，同时还要处理基础设施问题。

而且，在规模较小时，进行沟通和调整更加容易。你可以在群聊中提出意见或当面交流，每个人都能立刻理解。

因此，一个小团队可以放弃大部分结构。你不需要一次工作六周的时间。你不需要冷却期，正式的提案或押注表。与其有专门的塑造者和建设者并行轨道，不如同一个人来回切换。要谨慎选择你穿的哪顶帽子，以及你处在哪个阶段。确定一个预期，塑造接下来要做的事情，然后建设，接着再塑造下一个事情。你的押注可能每次都是不同的规模：也许这里两周，那里三周。你仍然在进行塑造、押注和建设，只是不像之前那样僵化地按照周期和冷却期进行。



即使你不按照周期进行工作或没有专门的人来进行塑造和建设，工作的各个阶段仍然成立。

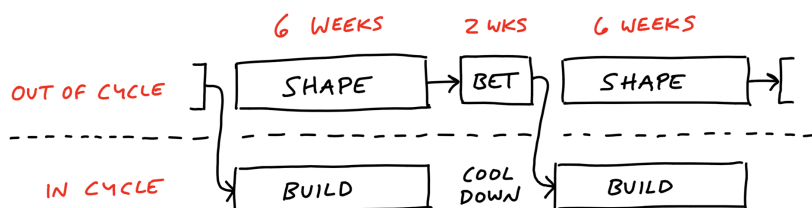
大到可以专业化

当你雇佣了更多人之后，所有这种灵活性从一种优势变成了一种劣势。随意临时开会和在聊天室讨论不再奏效。协调开始占用更多时间，事情开始有所遗漏。

这时候，采用六周周期、冷却期和正式的押注表是有意义的。由于有更多人可以进行建设，就需要有人投入更多时间去弄清楚要建设什么。这可能意味着创始人花更多时间进行塑造而不是建设，或者意味着提升一个员工，从周期内设计工作转向周期外的塑造工作。

在 Basecamp 目前的规模（整个公司大约有 50 人，产品团队大约有十几人），我们已经能够专业化角色，以便设计师和程序员团队能够在周期内无任何干扰地工作。一个名为 SIP（安全、基础设施和性能）的专门团队处理更低层次和更结构化的技术工作。我们的运维团队保持灯火通明。我们在支持团队中有技术人员可以调查客户提出的问题。所有这些意味着在周期内我们不需要打断核心产品团队的设计师和程序员的工作。

有了专门的塑造者和建设者，情况就更有结构化。塑造者在“周期外”轨道上工作。在周期之间的冷却期给了每个人解决错误和处理突发问题的时间。押注表在冷却期举行，然后为下一个周期进行押注。



有了更多的人，塑造和建设发生在不同的轨道上，并且进行押注来填满六周的周期。

如何开始塑造形势

选项 A：进行为期六周的实验

您无需一次性改变一切。如果整个产品团队还没有准备好进行重大改变，只需从进行为期六周的单一实验开始。以下是需要采取的步骤：

1. 选择一个重要的项目，可以在六周内轻松完成。在第一次实验时要保守，给予额外的时间。
2. 抽调一名设计师和两名程序员，在整个六周内的时间。保证在实验期间没有人会打扰他们。
3. 不要使用正式的投注表，只需计划团队完成为这个实验塑造的工作。
4. 通过向团队展示您塑造的工作，并具备演讲的所有要素来开始。设定他们将发现并跟踪自己的任务的期望。[\(移交责任\)](#)
5. 为跨职能团队专门指定一个物理空间或聊天室，以便他们可以紧密合作。

6. 鼓励他们通过早期将 UI 和代码结合来[完成一部分](#)。

您无需立刻担心[范围映射](#)或[显示进展](#)。只需通过投入不间断的时间、提前塑造工作，并让团队解决细节，您应该能看到进展的巨大飞跃。

一旦团队习惯于[完成一部分](#)，舞台就会为以后适当地映射范围做好准备。这与之前的想法相同，只是重复了一次。在以后，当他们擅长定义范围时，您可以使用山图表来[显示进展](#)。

这种方法让您能够在团队和一个为期六周的承诺中展示成功。凭借良好结果获得的信誉，将更容易游说进行更大的改变，并使更广泛的团队转变为这种工作方式。

选项 B：从塑造开始

有时候不可能让一个团队一起工作六周，因为其他人，比如 CTO，控制着程序员的时间。在这种情况下，您可以通过塑造一个有比以往项目更清晰边界的吸引人的项目来开始。提出这个项目，并将其纳入公司现有的调度流程中（即使是使用[碎纸机](#)）。更好地塑造的工作可以为工程团队带来启示，并帮助他们接受更长的周期或更深思熟虑的投注过程。

选项 C：从周期开始

另一种方法是以六周的周期工作开始。对于以前使用两周冲刺的团队来说，这样可以减少不断进行计划会议的开销，并给程序员更多的时间来建立动力并进入状态。一旦团队有了更多的自由空间，就会自然会更多地考虑如何塑造工作，以利用这种新的能力。

首先解决发货问题

在过多担心改进研究或发现流程之前，先建立起发货能力。您可以拥有世界上最好的客户洞察力，但如果无法将其转化为项目并进行交付，那就没有意义。首先让团队养成完成事情的节奏。一旦具备了发货能力，然后才开始改进您的塑造过程的输入。

关注最终结果

有时让团队更自由地设置自己的任务和日程安排可能有点可怕。您可能会想：如果他们沒有利用我们为周期分配的所有时间怎么办？如果其中一名程序员或设计师在周期的某个时刻闲置不做事怎么办？

要克服这些担忧，要将思维方式从微观转移到宏观。问自己：如果我们在六周后交付了这个项目，我们会有什么感受？我们会对我们取得的成就感到满意吗？当项目按时交付，每个人都觉得他们取得了进步，那就是成功。在途中发生的每小时或每天的确切情况并不重要，重要的是结果。

术语表

食欲

我们想要在项目上花费的时间，与估计相对。

基线

顾客在我们目前正在构建的东西之外所做的事情。

赌注

承诺团队在一个周期内不受打扰地完成项目并期望完成的决定。

下注会议

在冷却期间举行的会议，在此期间利益相关者决定在下一个周期下注哪些提案。

大批量

占用团队整个周期并在最终发布的一个项目。

面包板

一个 UI 概念，定义了各种功能和它们之间的连接，但没有视觉样式。

断路器

一种风险管理技术：默认情况下取消在一个周期内没有完成的项目，而不是默认情况下延期。

清理模式

构建新产品的最后阶段，在这个阶段我们不会对任何特定的项目进行形状设计或下注，而是分配非结构化的时间来在发布之前修复所需的任何问题。

冷却期

两周的休息时间，用于处理临时任务、修复错误并举行下注会议，以连接两个周期。

周期

一个六周的时间段，在此期间团队不受干扰地工作在形状设计的项目上。

降低风险

通过形状设计和消除坑洞，提高在一个周期内完成项目的几率。

发现的任务

团队开始参与实际工作后发现需要做的任务。

下坡

任务、范围或项目的阶段，在此阶段所有未知问题都已解决，只剩下执行阶段。

粗线素描

用粗线绘制的 UI 概念的简图，精度非常低。

山图

显示工作状态的图表，从未知到已知到完成的范围。

冰山

工作范围，其中后端工作比 UI 或反之复杂得多。

想象的任务

团队在仅考虑项目后决定需要完成的工作。请参见发现的任务。

层状蛋糕

通过查看 UI 的表面积来估算的工作范围。

抽象级别

在描述问题或解决方案时我们留下或排除的详细程度。

必须完成的任务

必须完成以使范围被认为完成的任务。

可选完成的任务

留到周期末尾的任务。如果没有时间完成，它们会被削减。以“~”开头标记。

提案

介绍经过形状设计的项目想法的文档，以供在下注会议上考虑。

生产模式

构建新产品的阶段，在此阶段核心架构已经确定，我们应用标准的形状设计流程。

兔子洞

项目的一部分，太不明确、复杂或不确定，无法下注。

研发模式

构建新产品的阶段，高级团队快速实现核心功能以定义核心架构。

原始想法

以文字表达的请求或功能想法，尚未进行形状设计。

范围

项目的部分，可以独立于项目的其余部分进行构建、集成和完成。

范围锤炼

强制性地质疑设计、实施或用例，以减少范围并在固定的时间框内完成。

形状设计

通过在下注之前定义解决方案的关键元素，使抽象的项目想法更加具体。

六周

我们周期的长度。六周足够长，可以完成一些有意义的事情，又短到足以从一开始就感受到截止日期的压力。

小批量

一组 1-2 周的项目，由一个团队在六周周期结束时发布。

时间视野

我们可以感受到从一开始就对我们施加压力的最长时间段。六周。

上坡

任务、范围或项目的阶段，在此阶段仍然存在未知问题或未解决的问题。请参见下坡。

作者简介

Ryan Singer 曾在软件开发的各个层面工作过，从 UI 设计到后端编程再到战略规划。

在 [Basecamp](#) 工作了 17 年以上，他设计的功能被数百万人使用，并发明了团队用于设计、开发和发布正确事物的流程。

如今，他专注于产品策略：了解 Basecamp 的客户想要做什么，以及如何使产品更好地满足他们的需求。

谢谢阅读。 我很想听听您在尝试应用“Shape Up”方法时的问题和经验。请通过 [Twitter](#) 联系我，或发送邮件至 shapeup@basecamp.com。